

венных или искусственных языках. Поэтому для обнаружения случайных или умышленных искажений информационной последовательности требуется специальный режим, называемый выработкой *имитоприставки* или *кода аутентификации сообщений* *MAC* (Message Authentication Code).

Имитоприставка - это добавляемая к зашифрованным данным контрольная комбинация относительно небольшой разрядности, зависящая от открытых данных и ключевой информации. Свойства этого контрольного кода:

- имитоприставка имеет фиксированную разрядность, не зависящую от разрядности входных данных;
- * имитоприставка зависит от всех без исключения битов исходного массива данных, а также от их взаимного расположения.

Для противника две следующие задачи вычислительно неразрешимы:

- в вычисление имитоприставки для заданной незашифрованной информационной последовательности;
- # подбор открытых данных под заданную имитоприставку.

В качестве имитоприставки чаще всего используется часть последнего блока шифротекста, полученного в режиме *CFB*. В общем случае вероятность успешного навязывания ложных данных равна 2^{-N} , где N - разрядность контрольного кода.

Литература к главе 1

1. Брассар Ж. Современная криптология: Пер. с англ. - М.: ПОЛИМЕД, 1999.
2. Варфоломеев А. А., Жуков А. Е., Мельников А. Б., Устюжанин Д. Д. Блочные криптосистемы. Основные свойства и методы анализа стойкости. - М.: МИФИ, 1998.
3. Винокуров А. Страничка классических блочных шифров.
<http://www.enlight.ru/crypto/>
4. Жуков И. Ю., Иванов М. А., Осмоловский С. А. Принципы построения криптостойких генераторов псевдослучайных кодов // Проблемы информационной безопасности. Компьютерные системы. - 2001. - № 1. - С. 55-65.
5. Иванов М. А. Криптографические методы защиты информации в компьютерных системах и сетях. М.: КУДИЦ-ОБРАЗ, 2001.
6. Schneier B. Applied cryptography, 2nd Edition, John Wiley & Sons (1996). [Имеется перевод: Шнайер Б. Прикладная криптография. - <http://www.ssl.stu.neva.ru/psw/crypto.html>].

ГЛАВА 2

Стандарт криптографической защиты XXI века - Advanced Encryption Standard (*AES*)

2.1. История конкурса на новый стандарт криптозащиты

В 1997 г. Национальный институт стандартов и технологий США (NIST) объявил о начале программы по принятию нового стандарта криптографической защиты - стандарта XXI в. для закрытия важной информации правительственного уровня на замену существующему с 1974 г. алгоритму *DES*, самому распространенному криптоалгоритму в мире. *DES* считается устаревшим по многим параметрам: длине ключа, удобству реализации на современных процессорах, быстродействию и др., за исключением самого главного - стойкости. За 25 лет интенсивного криптоанализа не было найдено методов вскрытия этого шифра, существенно отличающихся по эффективности от полного перебора по ключевому пространству.

Требования к кандидатам были следующие:

- криптоалгоритм должен быть открыто опубликован;
- * криптоалгоритм должен быть симметричным блочным шифром, допускающим размеры ключей в 128, 192 и 256 бит;
- * криптоалгоритм должен быть предназначен как для аппаратной, так и для программной реализации;
- * криптоалгоритм должен быть доступен для открытого использования в любых продуктах, а значит, не может быть запатен-

тован, в противном случае патентные права должны быть аннулированы;

- криптоалгоритм подвергается изучению по следующим параметрам: *стойкости, стоимости, гибкости, реализуемости в smart-картах.*

Стойкость. Это самый важный критерий в оценке алгоритма. Оценивались: способность шифра противостоять различным методам криптоанализа, статистическая безопасность и относительная защищенность по сравнению с другими кандидатами.

Стоимость. Не менее важный критерий, учитывая одну из основных целей NIST, - широкая область использования и доступность *AES*. Стоимость зависит от вычислительной эффективности (в первую очередь быстродействия) на различных платформах, удобства программной и аппаратной реализации, низких требований к памяти, простоты (простые алгоритмы легче реализовывать, они более прозрачны для анализа).

Гибкость. Гибкость включает способность алгоритма обрабатывать ключи больше оговоренного минимума (128 бит), надежность и эффективность выполнения в разных средах, возможность реализации других криптографических функций: комбинированного шифрования, хеширования и т. д.

Другими словами, *AES* должен быть существенно более эффективным с точки зрения практической реализации (в первую очередь скорости шифрования и формирования ключей), иметь больший запас прочности, чем *TripleDES*, при этом не уступая ему в стойкости.

Реализуемость в smart-картах. Важная область использования *AES* в будущем - smart-карты, при этом главной проблемой является небольшой объем доступной памяти. NIST исходил из допущения, что некоторые дешевые карты могут иметь всего 256 байт *RAM* (для вычисляемых данных) и 2000 *ROM* (для хранения алгоритмов и констант). Существует два основных метода формирования раундовых ключей:

- * вычисление на начальном этапе работы криптоалгоритма и хранение в памяти;
- * вычисление раундовых ключей "на лету".

Ясно, что второй вариант уменьшает затраты *RAM*, и поэтому наличие такой возможности в криптоалгоритме является его несомненным достоинством.

На открытый конкурс были приняты 15 алгоритмов, разработанных криптографами 12 стран – Австралии, Бельгии, Великобритании, Германии, Израиля, Канады, Коста-Рики, Норвегии, США, Франции, Южной Кореи и Японии.

В финал конкурса вышли следующие алгоритмы: *MARS*, *TWOFISH* и *RC6* (США), *RIJNDAEL* (Бельгия), *SERPENT* (Великобритания, Израиль, Норвегия). По своей структуре *TWOFISH* является классическим шифром Фейстеля; *MARS* и *RC6* можно отнести к модифицированным шифрам Фейстеля, в них используется новая малоизученная операция циклического "прокручивания" битов слова на число позиций, изменяющихся в зависимости от шифруемых данных и секретного ключа; *RIJNDAEL* и *SERPENT* являются классическими *SP-сетями*. *MARS* и *TWOFISH* имеют самую сложную конструкцию, *RIJNDAEL* и *RC6* – самую простую.

Финалисты будут описаны по единой схеме, данной в документе NIST. Сначала описываются обнаруженные "слабости" алгоритма (если таковые имеются), затем преимущества и, наконец, недостатки.

MARS выставлен на конкурс фирмой IBM, одним из авторов шифра является Д. Копперсмит, участник разработки *DES*. В алгоритме не обнаружено слабостей в защите.

Преимущества:

- * высокий уровень защищенности;
- « высокая эффективность на 32-разрядных платформах, особенно поддерживающих операции умножения и циклического сдвига;
- * потенциально поддерживает размер ключа больше 256 бит.

Недостатки:

- сложность алгоритма затрудняет анализ его надежности;
- снижение эффективности на платформах без необходимых операций;
- * сложность защиты от временного анализа и анализа мощности.

RC6 предложен фирмой RSA Lab, одним из его авторов является Р. Ривест. В алгоритме не обнаружено слабостей в защите.

Преимущества:

- высокая эффективность на 32-битовых платформах, особенно поддерживающих операции умножения и циклических сдвигов;
- простая структура алгоритма упрощает анализ его надежности;
- наличие хорошо изученного предшественника - *RC5*;
- « быстрая процедура формирования ключа;
- « потенциально поддерживает размер ключа больше 256 бит;
- длина ключа и число раундов могут быть переменными.

Недостатки:

- * относительно низкий уровень защищенности;
 - * снижение эффективности на платформах, не имеющих необходимых операций;
- В сложность защиты от временного анализа и анализа мощности;
- * невозможность генерации раундовых ключей "на лету".

RIJNDAEL большинством участников конкурса назван как лучший выбор, если будет отвергнут их собственный шифр. Основан на шифре *SQUARE* тех же авторов. В алгоритме не обнаружено слабостей в защите.

Преимущества:

- * высокая эффективность на любых платформах;
- высокий уровень защищенности;
- хорошо подходит для реализации в smart-картах из-за низких требований к памяти;
- * быстрая процедура формирования ключа;
- * хорошая поддержка параллелизма на уровне инструкций;
- поддержка разных длин ключа с шагом 32 бита.

Недостатки:

- » уязвим к анализу мощности.

SERPENT – разработка профессиональных криптоаналитиков Р. Андерсона, Э. Бихэма и Л. Кнудсена. Создал шифр, успешно противостоящий всем известным на сегодня атакам, разработчики затем удвоили количество его раундов. В алгоритме не обнаружено слабостей в защите.

Преимущества:

- * высокий уровень защищенности;
- * хорошо подходит для реализации в smart-картах из-за низких требований к памяти.

Недостатки:

- * самый медленный алгоритм среди финалистов;
- * уязвим к анализу мощности.

TWOFISH основан на широко используемом шифре *BLOWFISH*; один из авторов разработки - Б. Шнайер. Главная особенность шифра - изменяющиеся в зависимости от секретного ключа таблицы замен. В алгоритме не обнаружено слабостей в защите.

Преимущества:

- * высокий уровень защищенности;
 - * хорошо подходит для реализации в smart-картах из-за низких требований к памяти;
- В высокая эффективность на любых платформах, в том числе на ожидаемых в будущем 64-разрядных архитектурах фирм Intel и Motorola;
- « поддерживает вычисление раундовых ключей "на лету";
- в поддерживает распараллеливание на уровне инструкций;
- * допускает произвольную длину ключа до 256 бит.

Недостатки:

- * особенности алгоритма затрудняют его анализ;
- * высокая сложность алгоритма;
- * применение операции сложения делает алгоритм уязвимым к анализу мощности и временному анализу.

В табл. 2.1-2.4 представлены сравнительные характеристики финалистов конкурса *AES* по данным работы [11].

Таблица 2.1. Скорость шифрования (число процессорных циклов, требуемое для шифрования одного 128-разрядного блока с использованием 128-разрядного ключа)

Криптоалгоритм	Процессор Intel Pentium Pro	Процессор Intel Pentium Pro	Процессор Intel Pentium Pro	Процессор TMS320c50 DSP
MARS	390	320	2960	8908
RC6	260	250	1870	8231
RUNDAEL	440	291	2230	3518
SERPENT	1030	900	3180	14703
TWOFISH	400	258	5280	4672

Таблица 2.2. Параллелизм теоретическая производительность

Криптоалгоритм	Число циклов на критическом пути по данным двух источников		Максимальное число процессоров, эффективно работающих параллельно
MARS	214	258	2
RC6	181	185	2
RUNDAEL	71	86	7
SERPENT	526	556	2
TWOFISH	162	166	3

Таблица 2.3. Время формирования ключа при реализации на Си (число процессорных циклов при шифровании одного 128-разрядного блока с использованием 128-разрядного ключа)

Криптоалгоритм	Процессор Intel Pentium Pro	Процессор TMS320c541 DSP
MARS	4400	54427
RC6	1700	40011
RUNDAEL	850	26642
SERPENT	2500	28913
TWOFISH	8600	88751

Таблица 2.4. Требуемая память при реализации на Java, байт

Криптоалгоритм	RAM	ROM
MARS	456	19719
RC6	480	7800
RIJNDAEL	2000	18405
SERPENT	1248	38900
TWOFISH	8000	19181

В октябре 2000 г. конкурс завершился. Победителем был признан бельгийский шифр *RIJNDAEL*, как имеющий наилучшее сочетание стойкости, производительности, эффективности реализации и гибкости. Его низкие требования к объему памяти делают его идеально подходящим для встроенных систем. Авторами шифра являются Йон Дэмен (Joan Daemen) и Винсент Рюмен (Vincent Rijmen), начальные буквы фамилий которых и образуют название алгоритма - *RIJNDAEL*.

После этого NIST начал подготовку предварительной версии Федерального Стандарта Обработки Информации (Federal Information Processing Standart – FIPS) и в феврале 2001 г. опубликовал его на сайте <http://csrc.nist.gov/encryption/aes/>. В течение 90-дневного периода открытого обсуждения предварительная версия FIPS пересматривалась с учетом комментариев, после чего начался процесс исправлений и утверждения. Наконец 26 ноября 2001 г. была опубликована окончательная версия стандарта FIPS-197, описывающего новый американский стандарт шифрования *AES*. Согласно этому документу стандарт вступил в силу с 26 мая 2002 г.

2.2. Блочный криптоалгоритм *RIJNDAEL* и стандарт *AES*

Материалы данного раздела в значительной степени основываются на авторском описании алгоритма *RIJNDAEL* [8] и документе FIPS-197 [9]. Везде в дальнейшем изложении, где стандарт *AES* совпадает с алгоритмом *RIJNDAEL*, упоминается только последний. Все отличия принятого стандарта от криптоалгоритма *RIJNDAEL* оговариваются особо.

2.2.1. Математические предпосылки

Алгоритм оперирует байтами, которые рассматриваются как элементы конечного поля $GF(2^8)$.

Элементами поля $GF(2^8)$ являются многочлены степени не более 7, которые могут быть заданы строкой своих коэффициентов. Если представить байт в виде

$$\{a_7, a_6, a_5, a_4, a_3, a_2, a_1, a_0\}, a_i \in \{0, 1\}, i = \overline{0, 7},$$

то элемент поля описывается многочленом

$$a_7x^7 + a_6x^6 + a_5x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

Например, байту $\{11001011\}$ (или $\{cb\}$ в шестнадцатеричной форме) соответствует многочлен $x^7 + x^6 + x^3 + x + 1$.

Для элементов конечного поля определены аддитивные и мультипликативные операции.

Сложение суть операция поразрядного *XOR* и поэтому обозначается как $\odot$. Пример выполнения операции сложения:

$$(x^6 + x^4 + x^2 + x + 1) \odot (x^7 + x + 1) = x^7 + x^6 + x^4 + x^2$$

(в виде многочленов)

$$\{01010111\} \odot \{10000011\} = \{11010100\}$$

(двоичное представление)

$$\{57\} \oplus \{83\} = \{d4\}$$

(шестнадцатеричное представление)

В конечном поле для любого ненулевого элемента a существует обратный элемент $-a$, при этом $a + (-a) = 0$, где нулевой элемент — это $\{00\}$. В $GF(2^8)$ справедливо $a + a = 0$, т. е. каждый ненулевой элемент является своей собственной аддитивной инверсией.

Умножение, обозначаемое далее как $\bullet$, более сложная операция. Умножение в $GF(2^8)$ — это операция умножения многочленов со взятием результата по модулю неприводимого многочлена $\phi(x)$ восьмой степени и с использованием операции *XOR* при приведении подобных членов. В *RIJNDAEL* выбран $\phi(x) = x^8 + x^4 + x^3 + x + 1$, или в шестнадцатеричной форме $1\{1b\}^1$. Пример операции умножения:

$$\{57\} \bullet \{83\} = \{c1\},$$

¹ Запись $1\{1b\}$ обозначает, что присутствует "лишний" девятый бит.

так как

$$(x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1) \bmod (x^8 + x^4 + x^3 + x + 1) = x^7 + x^6 + 1,$$

поскольку

$$x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1 = (x^5 + x^3)(x^8 + x^4 + x^3 + x + 1) \oplus (x^7 + x^6 + 1).$$

Для любого ненулевого элемента a справедливо $a \cdot 1 = a$. Мультипликативной единицей в $GF(2^8)$ является элемент $\{01\}$.

Операция умножения может быть описана и реализована по-другому. Умножая произвольный многочлен седьмой степени

$$a_7x^7 + a_6x^6 + a_5x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

на x , получим

$$a_7x^8 + a_6x^7 + a_5x^6 + a_4x^5 + a_3x^4 + a_2x^3 + a_1x^2 + a_0x.$$

Приводя полученный многочлен по модулю $\phi(x) = 1\{1b\}$, получим результат произведения в конечном поле $GF(2^8)$. Для этого при $a_7 = 1$ достаточно вычесть (применить операцию поразрядного XOR) $\phi(x)$ из полученного выше многочлена. Если же $a_7 = 0$, то результат уже получается приведенным. Тогда умножение на x (т. е. $\{00000010\}$ в двоичной или $\{02\}$ в шестнадцатеричной форме) получается сдвигом влево и возможным последующим применением XOR с $\{1b\}$.

Пусть функция $xtime()$ осуществляет операцию умножения на x вышеописанным способом. Применяя функцию и раз можно получить результат умножения на x^n , а суммируя различные степени x , можно получить любой элемент поля. Например:

$$\{57\} \cdot \{13\} = \{fe\}, \text{ так как}$$

$$\{57\} \cdot \{02\} = xtime(\{57\}) = \{ae\}$$

$$\{57\} \cdot \{04\} = xtime(\{ae\}) = \{47\}$$

$$\{57\} \cdot \{08\} = xtime(\{47\}) = \{8e\}$$

$$\{57\} \cdot \{10\} = xtime(\{8e\}) = \{07\}, \text{ откуда}$$

$$\begin{aligned} \{57\} \cdot \{13\} &= \{57\} \cdot (\{01\} \text{ в } \{02\} \odot \{10\}) = \\ &= \{57\} \oplus \{ae\} \odot \{07\} = \{fe\}. \end{aligned}$$

Раундовые преобразования *RIJNDAEL* оперируют 32-разрядными словами. Четырехбайтовому слову может быть поставлен

в соответствии многочлен $a(x)$ с коэффициентами из $GF(2^8)$ степени не более трех:

$$a(x) = a_3x^3 + a_2x^2 + a_1x + a_0,$$

где $a_i \in GF(2^8)$, $i = \overline{0, 3}$. Сложение двух многочленов с коэффициентами из $GF(2^8)$ суть операция сложения многочленов с приведением подобных членов в поле $GF(2^8)$, т. е.

$$a(x) + b(x) = (a_3 \oplus b_3)x^3 + (a_2 \oplus b_2)x^2 + (a_1 \oplus b_1)x + (a_0 \oplus b_0).$$

Таким образом, сложение двух 4-байтовых слов суть операция поразрядного XOR.

Умножение - более сложная операция. Предположим, мы перемножаем два многочлена

$$a(x) = a_3x^3 + a_2x^2 + a_1x + a_0 \text{ и } b(x) = b_3x^3 + b_2x^2 + b_1x + b_0.$$

Результатом умножения

$$c(x) = a(x)b(x)$$

будет многочлен

$$c(x) = c_6x^6 + c_5x^5 + c_4x^4 + c_3x^3 + c_2x^2 + c_1x + c_0,$$

$$c_0 = a_0b_0$$

$$c_1 = a_1b_0 \oplus a_0b_1$$

$$c_2 = a_2b_0 \oplus a_1b_1 \oplus a_0b_2$$

$$c_3 = a_3b_0 \oplus a_2b_1 \oplus a_1b_2 \oplus a_0b_3$$

$$c_4 = a_3b_1 \oplus a_2b_2 \oplus a_1b_3$$

$$c_5 = a_3b_2 \oplus a_2b_3$$

$$c_6 = a_3b_3.$$

Для того чтобы результат умножения мог быть представлен 4-байтовым словом, необходимо взять результат по модулю многочлена степени не более 4. Авторы шифра выбрали многочлен

$$x^4 + 1,$$

для которого справедливо

$$x^i \bmod (x^4 + 1) = x^{i \bmod 4}.$$

Таким образом, результатом $d(x)$ умножения $\otimes$ двух многочленов по модулю $x^4 + 1$

$$d(x) = a(x) \otimes b(x)$$

будет многочлен

$$d(x) = d_3x^3 + d_2x^2 + d_1x + d_0,$$

где

$$d_0 = a_0b_0 \oplus a_3b_1 \oplus a_2b_2 \oplus a_1b_3$$

$$d_1 = a_1b_0 \oplus a_0b_1 \oplus a_3b_2 \oplus a_2b_3$$

$$d_2 = a_2b_0 \oplus a_1b_1 \oplus a_0b_2 \oplus a_3b_3$$

$$d_3 = a_3b_0 \oplus a_2b_1 \oplus a_1b_2 \oplus a_0b_3.$$

В матричной форме это может быть записано следующим образом

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}.$$

Пусть

$$b(x) = b_3x^3 + b_2x^2 + b_1x + b_0.$$

Умножению на x многочлена $b(x)$ с коэффициентами из $GF(2^8)$ по модулю $x^4 + 1$, учитывая свойства последнего, соответствует циклический сдвиг байтов в пределах слова в сторону старшего байта, так как

$$x \otimes b(x) = b_2x^3 + b_1x^2 + b_0x + b_3.$$

2.2.2. Описание криптоалгоритма

2.2.2.1. Формат блоков данных и число раундов

RIJNDAEL - это итерационный блочный шифр, имеющий архитектуру "Квадрат". Шифр имеет переменную длину блоков и различные длины ключей. Длина ключа и длина блока могут быть равны независимо друг от друга 128, 192 или 256 битам. В стандарте *AES* определена длина блока данных, равная 128 битам.

Промежуточные результаты преобразований, выполняемых в рамках криптоалгоритма, называются *состояниями* (*State*). Состояние можно представить в виде прямоугольного массива байтов (рис. 2.1). При размере блока, равном 128 битам, этот 16-байтовый массив (рис. 2.2, а) имеет 4 строки и 4 столбца (каждая строка и каждый столбец в этом случае могут рассматриваться как 32-разрядные слова). Входные данные для шифра обозначаются как байты состояния в порядке $s_{00}, s_{10}, s_{20}, s_{30}, s_{01}, s_{11}, s_{21}, s_{31}, \dots$

После завершения действия шифра выходные данные получают- ся из байтов состояния в том же порядке. В общем случае число столбцов N_b равно длине блока, деленной на 32.

State

a_0	a_4	a_8	a_{12}
a_1	a_5	a_9	a_{13}
a_2	a_6	a_{10}	a_{14}
a_3	a_7	a_{11}	a_{15}

Рис. 2.1. Пример представления 128-разрядного блока данных в виде массива *State*, где a_i - байт блока данных, а каждый столбец - одно 32-разрядное слово

s_{00}	s_{01}	s_{02}	s_{03}
s_{10}	s_{11}	s_{12}	s_{13}
s_{20}	s_{21}	s_{22}	s_{23}
s_{30}	s_{31}	s_{32}	s_{33}

a

k_{00}	k_{01}	k_{02}	k_{03}
k_{10}	k_{11}	k_{12}	k_{13}
k_{20}	k_{21}	k_{22}	k_{23}
k_{30}	k_{31}	k_{32}	k_{33}

b

Рис. 2.2. Форматы данных:

a - пример представления блока ($N_b = 4$);

b - ключа шифрования ($N_k = 4$), где s_{ij} и k_{ij} - соответственно байты массива *State* и ключа, находящиеся на пересечении i -й строки и j -го столбца

Ключ шифрования также представлен в виде прямоугольного массива с четырьмя строками (рис. 2.2, *b*). Число столбцов N_k этого массива равно длине ключа, деленной на 32. В стандарте определены ключи всех трех размеров - 128 бит, 192 бита и 256 бит,

то есть соответственно 4, 6 и 8 32-разрядных слова (или столбца - в табличной форме представления). В некоторых случаях ключ шифрования рассматривается как линейный массив 4-байтовых слов. Слова состоят из 4 байтов, которые находятся в одном столбце (при представлении в виде прямоугольного массива).

Число раундов N_r в алгоритме *RIJNDAEL* зависит от значений N_b и N_k , как показано в табл. 2.5. В стандарте *AES* определено соответствие между размером ключа, размером блока данных и числом раундов шифрования, как показано в табл. 2.6.

Таблица 2.5. Число раундов N_r как функция от длины ключа N_k и длины блока N_b

N_r	$N_b = 4$	$N_b = 6$	$N_b = 8$
$N_k = 4$	10	12	14
$N_k = 6$	12	12	14
$N_k = 8$	14	14	14

Таблица 2.6. Соответствие между длиной ключа, размером блока данных и числом раундов в стандарте *AES*

Стандарт	Длина ключа (N_k слов)	Размер блока данных (N_b слов)	Число раундов (N_r)
<i>AES-128</i>	10	12	14
<i>AES-192</i>	12	12	14
<i>AES-256</i>	14	14	14

2.2.2.2. Раундовое преобразование

Раунд состоит из четырех различных преобразований:

- замены байтов *SubBytes()* - побайтовой подстановки в *S*-блоках с фиксированной таблицей замен размерностью 8 x 256;
- сдвига строк *ShiftRows()* - побайтового сдвига строк массива *State* на различное количество байт;
- перемешивания столбцов *MixColumns()* - умножения столбцов состояния, рассматриваемых как многочлены над $GF(2^8)$, на многочлен третьей степени $g(x)$ по модулю $x^4 + 1$;
- сложения с раундовым ключом *AddRoundKey()* - поразрядного XOR с текущим фрагментом развернутого ключа.

Замена байтов (*SubBytes*). Преобразование *SubBytes()* представляет собой нелинейную замену байтов, выполняемую независимо с каждым байтом состояния. Таблицы замены *S*-блока являются инвертируемыми и построены из композиции следующих двух преобразований входного байта:

- 1) получение обратного элемента относительно умножения в поле $GF(2^8)$, нулевой элемент $\{00\}$ переходит сам в себя;
- 2) применение преобразования над $GF(2)$, определенного следующим образом:

$$\begin{bmatrix} b'_0 \\ K \\ b'_2 \\ b'_3 \\ b'_4 \\ b'_5 \\ b'_6 \\ b'_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} \vee \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ \perp \end{bmatrix} + \begin{bmatrix} \Gamma \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}.$$

Другими словами суть преобразования может быть описана уравнениями:

$$b'_i = b_i \otimes b_{(i+4) \bmod 8} \odot b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \odot b_{(i+7) \bmod 8} \odot c_i$$

где $c_0 = c_1 = c_5 = c_6 = 1$, $c_2 = c_3 = c_4 = c_7 = 0$, b_i и b'_i — соответственно исходное и преобразованное значение i -го бита, $i = \overline{0, 7}$.

Применение описанного 5-блока ко всем байтам состояния обозначается как *SubBytes(State)*. Рис. 2.3. иллюстрирует применение преобразования *SubBytes()* к состоянию. Логика работы 5-блока при преобразовании байта {ху} отражена в табл. 2.7. Например, результат {ed} преобразования байта {53} находится на пересечении 5-й строки и 3-го столбца.

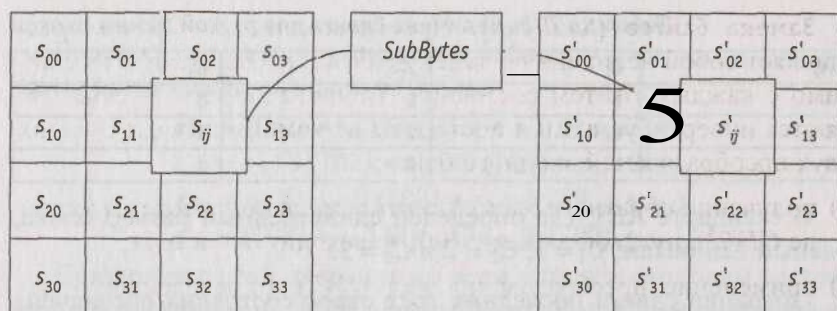


Рис. 2.3. *SubBytes* () действует на каждый байт состояния

Таблица 2.7. Таблица замен 5-блока

x	y															
	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	И	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	d	1d	9e
e	e1	ff1	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

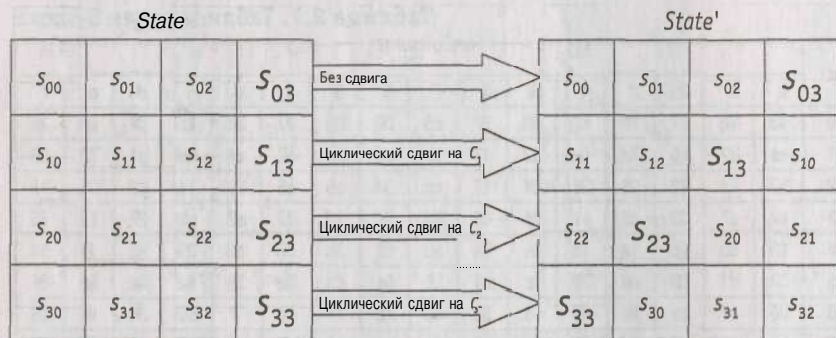
Преобразование сдвига строк (*ShiftRows*). Последние 3 строки состояния циклически сдвигаются влево на различное число байтов. Строка 1 сдвигается на C_1 байт, строка 2 - на C_2 байт, и строка 3 - на C_3 байт. Значения сдвигов C_1 , C_2 и C_3 в *RIJNDAEL* зависят от длины блока N_b . Их величины приведены в табл. 2.8.

Таблица 2. 8. Величина сдвига для разной длины блоков

N_b	C_1	4	C_3
4	1	2	3
6	1	2	3
8	1	3	4

В стандарте *AES*, где определен единственный размер блока, равный 128 битам, $C_1 = 1$, $C_2 = 2$ и $C_3 = 3$.

Операция сдвига последних трех строк состояния обозначена как *ShiftRows(State)*. Рис. 2.4 показывает влияние преобразования на состояние.

Рис. 2.4. *ShiftRows()* действует на строки состояния

Преобразование перемешивания столбцов (*MixColumns*). В этом преобразовании столбцы состояния рассматриваются как многочлены над $GF(2^8)$ и умножаются по модулю $x^4 + 1$ на многочлен $g(x)$, выглядящий следующим образом:

$$g(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}.$$

Это может быть представлено в матричном виде следующим образом:

$$\begin{bmatrix} s'_{0c} \\ s'_{1c} \\ s'_{2c} \\ s'_{3c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0c} \\ s_{1c} \\ s_{2c} \\ s_{3c} \end{bmatrix}, \quad 0 \leq c \leq 3,$$

где c - номер столбца массива *State*.

В результате такого умножения байты столбца $s_{0c}, s_{1c}, s_{2c}, s_{3c}$ заменяются соответственно на байты

$$s'_{0c} = (\{02\} \cdot s_{0c} \oplus (\{03\} \cdot s_{1c}) \oplus s_{2c} \oplus s_{3c},$$

$$s'_{1c} = s_{0c} \oplus (\{02\} \cdot s_{1c} \oplus (\{03\} \cdot s_{2c}) \oplus s_{3c},$$

$$s'_{2c} = s_{0c} \oplus s_{1c} \oplus (\{02\} \cdot s_{2c} \oplus (\{03\} \cdot s_{3c}),$$

$$s'_{3c} = (\{03\} \cdot s_{0c} \oplus s_{1c} \oplus s_{2c} \oplus (\{02\} \cdot s_{3c}).$$

Применение этой операции ко всем четырем столбцам состояния обозначено как *MixColumns(State)*. Рис. 2.5 демонстрирует применение преобразования *MixColumns()* к столбцу состояния.

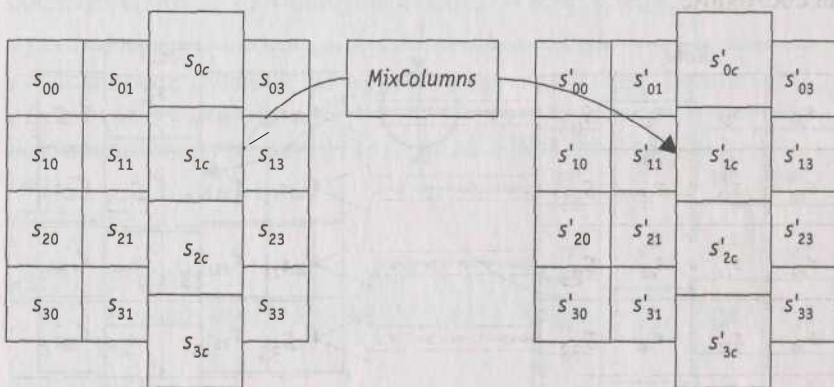


Рис. 2.5. *MixColumns()* действует на столбцы состояния

Добавление раундового ключа (*AddRoundKey*). В данной операции раундовый ключ добавляется к состоянию посредством простого поразрядного XOR. Раундовый ключ вырабатывается из ключа шифрования посредством алгоритма выработки ключей (key schedule). Длина раундового ключа (в 32-разрядных словах) равна длине блока N_b . Преобразование, содержащее добавление посредством XOR раундового ключа к состоянию (рис. 2.6), обозначено как *AddRoundKey(State, RoundKey)*.

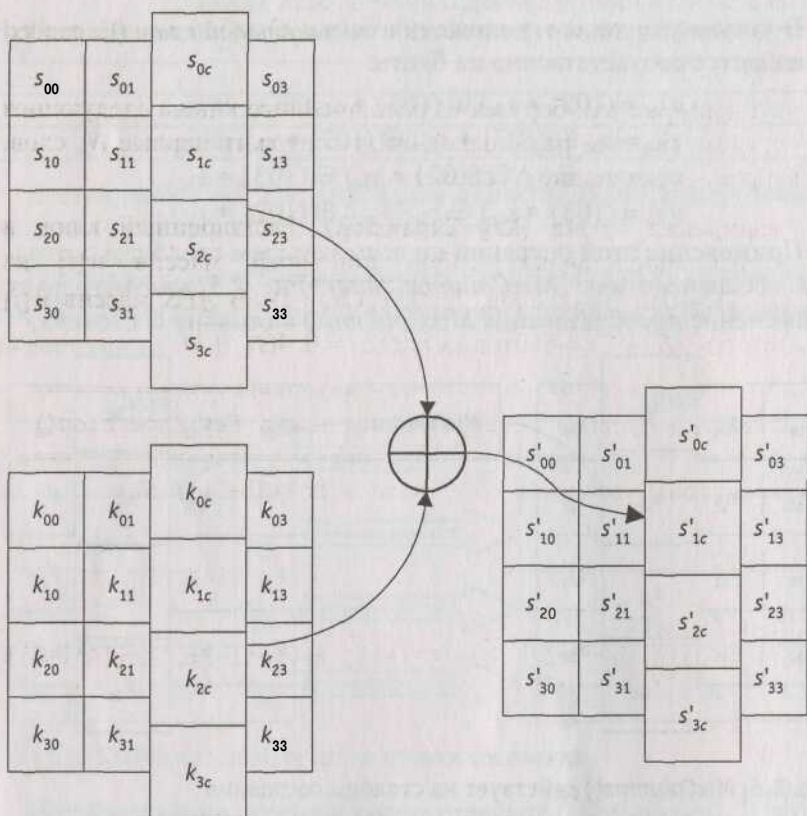


Рис. 2.6. При добавлении ключа раундовый ключ складывается посредством операции *XOR* с состоянием

2.2.2.3. Алгоритм выработки ключей (*Key Schedule*)

Раундовые ключи получаются из ключа шифрования посредством алгоритма выработки ключей. Он содержит два компонента: *расширение ключа* (Key Expansion) и *выбор раундового ключа* (Round Key Selection). Основополагающие принципы алгоритма выглядят следующим образом:

- * общее число битов раундовых ключей равно длине блока, умноженной на число раундов, плюс 1 (например, для длины блока 128 бит и 10 раундов требуется 1408 бит раундовых ключей);

- * ключ шифрования расширяется в *расширенный ключ* (Expanded Key);
- * раундовые ключи берутся из расширенного ключа следующим образом: первый раундовый ключ содержит первые N_b слов, второй - следующие N_b слов и т. д.

Расширение ключа (Key Expansion). Расширенный ключ в *RIJNDAEL* представляет собой линейный массив $w[i]$ из $N_b(N_r+1)$ 4-байтовых слов, $i = 0, N_b(N_r+1)$. В *AES* массив $w[i]$ состоит из $4(N_r+1)$ 4-байтовых слов, $i = 0, 4(N_r+1)$.

```
//=====
// Псевдокод функции разворачивания ключа Key Expansion()
//=====
KeyExpansion(byte key[4*Nk], word w[Nb*(Nr+1)], Nk)
begin
word temp
i = 0
while (i < Nk)
    w[i] = word(key[4*i], key[4*i+1], key[4*i+2], key[4*i+3])
    i = i+1
end while
i = Nk
while (i < Nb * (Nr+1))
    temp = w[i-1]
    if (i mod Nk = 0)
        temp = SubWord(RotWord(temp)) xor Rcon[i/Nk]
    else if (Nk > 6 and i mod Nk = 4)
        temp = SubWord(temp)
    end if
    w[i] = w[i-Nk] xor temp
    i = i + 1
end while
end
//=====
```

Примечание. Необходимо учитывать, что с целью полноты описания здесь приводится алгоритм для всех возможных длин ключей, на практике же полная его реализация нужна не всегда.

Первые N_k слов содержат ключ шифрования. Все остальные слова определяются рекурсивно из слов с меньшими индексами. Алгоритм выработки ключей зависит от величины N_k .

Как можно заметить (рис. 2.7, а), первые N_k слов заполняются ключом шифрования. Каждое последующее слово $w[i]$ получается посредством XOR предыдущего слова $w[i-1]$ и слова на N_k позиций ранее $w[i - N_k]$:

$$w[i] = w[i - 1] \oplus w[i - N_k].$$

Для слов, позиция которых кратна N_k , перед XOR применяется преобразование к $w[i-1]$, а затем еще прибавляется раундовая константа $Rcon$. Преобразование реализуется с помощью двух дополнительных функций: $RotWord()$, осуществляющей побайтовый сдвиг 32-разрядного слова по формуле $\{a_0 a_1 a_2 a_3\} \rightarrow \{a_1 a_2 a_3 a_0\}$, и $SubWord()$, осуществляющей побайтовую замену с использованием S -блока функции $SubBytes()$. Значение $Rcon[j]$ равно 2^{j-1} . Значение $w[i]$ в этом случае равно

$$w[i] = SubWord(RotWord(w[i - 1])) \oplus Rcon[i/N_k] \oplus w[i - N_k].$$



а

Раундовый ключ 0	Раундовый ключ 1	Раундовый ключ 2	
---------------------	---------------------	---------------------	--

б

Рис. 2.7. Процедуры:

- а - расширения ключа (светло-серым цветом выделены слова расширенного ключа, которые формируются без использования функций $SubWord()$ и $RotWord()$; темно-серым цветом выделены слова расширенного ключа, при вычислении которых используются преобразования $SubWord()$ и $RotWord()$);
- б - выбора раундового ключа для $N_k = 4$

Выбор раундового ключа (Round Key Selection). Раундовый ключ i получается из слов массива раундового ключа от $W[N_b i]$ и до $W[N_b(i + 1)]$, как показано на рис. 2.7.

Примечание. Алгоритм выработки ключей можно осуществлять и без использования массива $w[i]$. Для реализаций, в которых существенно требование к занимаемой памяти, раундовые ключи могут вычисляться "на лету" посредством использования

буфера из N_k слов. Расширенный ключ должен всегда получаться из ключа шифрования и никогда не указывается напрямую. Нет никаких ограничений на выбор ключа шифрования.

2.2.2.4. Функция зашифрования

Шифр *RIJNDAEL* состоит (рис. 2.8):

- из начального добавления раундового ключа;
- $N_r - 1$ раундов;
- заключительного раунда, в котором отсутствует операция *MixColumns()*.

На вход алгоритма подаются блоки данных *State*, в ходе преобразований содержимое блока изменяется и на выходе образуется шифротекст, организованный опять же в виде блоков *State*, как показано на рис. 2.9, где $N_b - 4$, in_m и $out_m - m$ -е байты соответственно входного и выходного блоков, $m = 0, 15$, s_{ij} - байт, находящийся на пересечении i -й строки и j -го столбца массива *State*, $i = 0, 3$, $j = 0, 3$.

Перед началом первого раунда происходит суммирование по модулю 2 с начальным ключом шифрования, затем - преобразование массива байтов *State* в течение 10, 12 или 14 раундов в зависимости от длины ключа. Последний раунд несколько отличается от предыдущих тем, что не задействует функцию перемешивания байт в столбцах *MixColumns()*.

```

//=====
//=== Процедура зашифрования в псевдокоде =====
//=====
Cipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*(Nr+1)])
begin
    byte state[4,Nb]
    state = in
    AddRoundKey(state, w[0, Nb-1])
    for round = 1 step 1 to Nr-1
        SubBytes(state)
        ShiftRows(state)
        MixColumns(state)
        AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])
    end for
    SubBytes(state)

```



```

ShiftRows(state)
AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1])
out = state
end
//=====

```

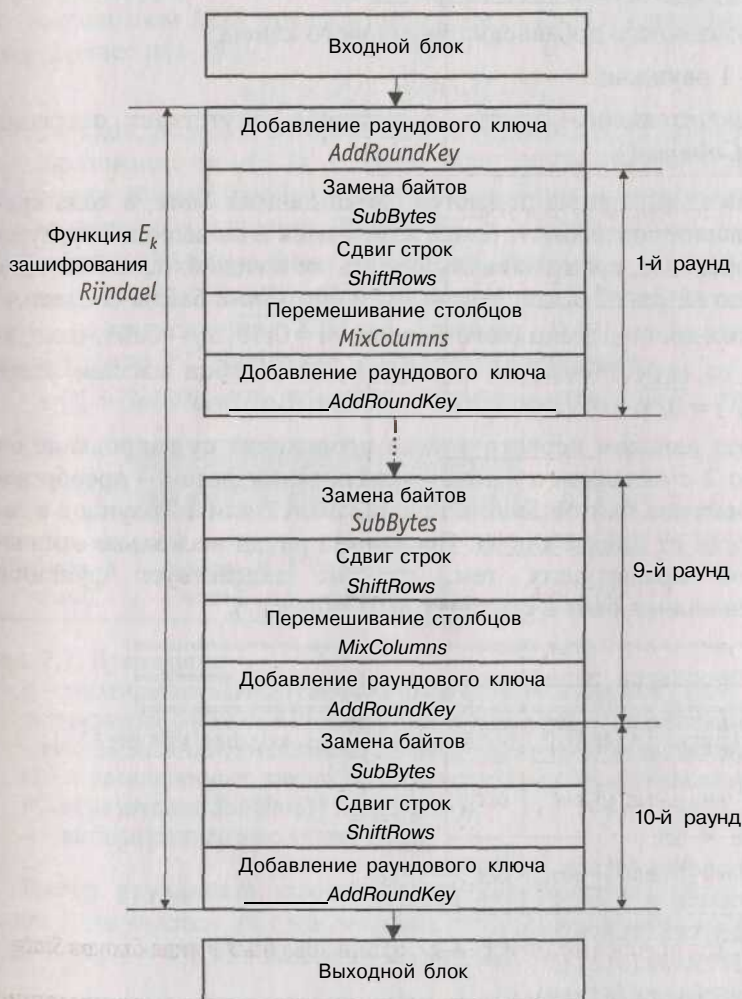


Рис. 2.8. Схема функции E_k зашифрования криптоалгоритма *RIJNDAEL* при $N_k = N_b = 4$



Рис. 2.9. Ход преобразования данных, организованных в виде блоков *State*

Рис. 2.10 демонстрирует и рассеивающие и перемешивающие свойства шифра. Видно, что два раунда обеспечивают полное рассеивание и перемешивание.

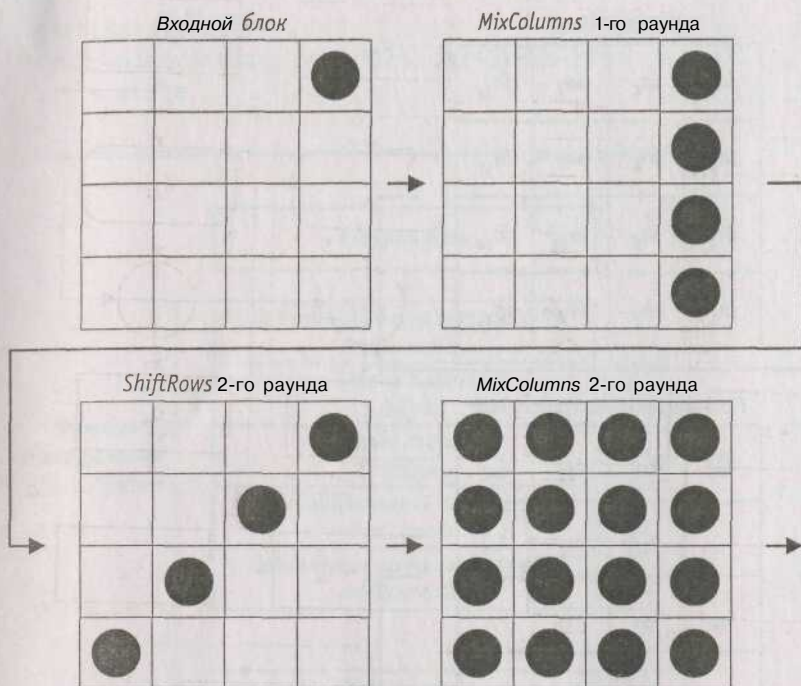


Рис. 2.10. Принцип действия криптоалгоритма *RIJNDAEL*;

● - измененный байт

2.2.2.5. Функция обратного расшифрования

Если вместо *SubBytes()*, *ShiftRows()*, *MixColumns()* и *AddRoundKey()* в обратной последовательности выполнить инверсные им преобразования, можно построить функцию обратного расшифрования. При этом порядок использования раундовых ключей является обратным по отношению к тому, который используется при зашифровании.

```
//=====
// Процедура обратного расшифрования в псевдокоде
//=====
InvCipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*(Nr+1)])
begin
    byte state[4,Nb]
    state = in
```



```

AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1])
for round = Nr-1 step -1 downto 1
    InvShiftRows(state)
    InvSubBytes(state)
    AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])
    InvMixColumns(state)
end for
InvShiftRows(state)
InvSubBytes(state)
AddRoundKey(state, w[0, Nb-1])
out = state
end
//=====

```

Далее приводится описание функций, обратных используемым при прямом зашифровании.

Функция *AddRoundKey()* обратна сама себе, учитывая свойства используемой в ней операции *XOR*.

Преобразование *InvSubBytes*. Логика работы инверсного *S*-блока при преобразовании байта {*xy*} отражена в табл. 2.9.

Таблица 2.9. Таблица замен инверсного *S*-блока

x	y															
	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
1	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
2	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
3	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
4	72	га	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
5	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
6	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
7	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
8	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	ra	b4	e6	73
9	96	ac	74	22	e7	ad	35	85	e2	Я	37	e8	1c	75	df	6e
a	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
b	fc	56	3e	4b	c6	d2	79	20	9a	db	00	fe	78	cd	5a	f4
c	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
d	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
e	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
f	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

Преобразование *InvShiftRows*. Последние 3 строки состояния циклически сдвигаются вправо на различное число байтов. Строка 1 сдвигается на C_1 байт, строка 2 - на C_2 байт, и строка 3 - на C_3 байт. Значения сдвигов C_1 , C_2 и C_3 зависят от длины блока N_b . Их величины приведены в табл. 2.8.

Преобразование *InvMixColumns*. В этом преобразовании столбцы состояния рассматриваются как многочлены над $GF(2^8)$ и умножаются по модулю $x + 1$ на многочлен $g^{-1}(x)$, выглядящий следующим образом:

$$g^{-1}(x) = \{0b\}x^3 + \{0d\}x^2 + \{09\}x + \{0e\}.$$

Это может быть представлено в матричном виде следующим образом:

$$\begin{bmatrix} s'_{0c} \\ s'_{1c} \\ s'_{2c} \\ s'_{3c} \end{bmatrix} = \begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix} \begin{bmatrix} s_{0c} \\ s_{1c} \\ s_{2c} \\ s_{3c} \end{bmatrix}, 0 \leq c \leq 3.$$

В результате на выходе получаются следующие байты:

$$\begin{aligned} s'_{0c} &= (\{0e\} \bullet s_{0c}) \oplus (\{0b\} \bullet s_{1c}) \oplus (\{0d\} \bullet s_{2c}) \oplus (\{09\} \bullet s_{3c}), \\ s'_{1c} &= (\{09\} \bullet s_{0c}) \circledast (\{0e\} \bullet s_{1c}) \circledast (\{0b\} \bullet s_{2c}) \circledast (\{0d\} \bullet s_{3c}), \\ s'_{2c} &= (\{0d\} \bullet s_{0c}) \circledast (\{09\} \bullet s_{1c}) \circledast (\{0e\} \bullet s_{2c}) \circledast (\{0b\} \bullet s_{3c}), \\ s'_{3c} &= (\{0b\} \bullet s_{0c}) \circledast (\{0d\} \bullet s_{1c}) \circledast (\{09\} \bullet s_{2c}) \circledast (\{0e\} \bullet s_{3c}). \end{aligned}$$

2.2.2.6. Функция прямого расширения

Алгоритм обратного расшифрования, описанный выше, имеет порядок приложения операций-функций, обратный порядку операций в алгоритме прямого шифрования, но использует те же параметры (развернутый ключ). Однако некоторые свойства алгоритма шифрования *RIJNDAEL* позволяют применить для расшифрования тот же порядок приложения функций (обратных используемым для зашифрования) за счет изменения некоторых параметров, а именно - развернутого ключа.

Два следующих свойства позволяют применить алгоритм прямого расшифрования.

Порядок приложения функций *SubBytes()* и *ShiftRows()* не играет роли. То же самое верно и для операций *InvSubBytes()*

и *InvShiftRowsQ*. Это происходит потому, что функции *SubBytes()* и *InvSubBytesQ* работают с байтами, а операции *ShiftRows()* и *InvShiftRowsQ* сдвигают целые байты, не затрагивая их значений.

Операция *MixColumnsQ* является линейной относительно входных данных, что означает

$$\begin{aligned} & \text{InvMixColumns}(\text{StateXOR RoundKey}) = \\ & - \text{InvMixColumns}(\text{State}) \text{ XOR } \text{InvMixColumns}(\text{RoundKey}) \end{aligned}$$

Эти свойства функций алгоритма шифрования позволяют изменить порядок применения функций *InvSubBytesQ* и *InvShiftRowsQ*. Функции *AddRoundKeyQ* и *InvMixColumns()* также могут быть применены в обратном порядке, но при условии, что столбцы (32-битные слова) развернутого ключа расшифрования предварительно пропущены через функцию *InvMixColumnsQ*.

Таким образом, можно реализовать более эффективный способ расшифрования с тем же порядком приложения функций как и в алгоритме зашифрования.

```
//=====
//== Псевдокод алгоритма прямого расшифрования ==
//=====
EqInvCipher(byte in[4*Nb], byte out[4*Nb], word
dw[Nb*(Nr+1)])
begin
    byte state[4,Nb]
    state = in
    AddRoundKey(state, dw[Nr*Nb, (Nr+1)*Nb-1])
    for round = Nr-1 step -1 downto 1
        InvSubBytes(state)
        InvShiftRows(state)
        InvMixColumns(state)
        AddRoundKey(state, dw[round*Nb, (round+1)*Nb-1])
    end for
    InvSubBytes(state)
    InvShiftRows(state)
    AddRoundKey(state, dw[0, Nb-1])
    out = state
end
//=====
_1*
```


При формировании развернутого ключа шифрования в процедуру развертывания ключа необходимо добавить следующий код

```
//=====
for i = 0 step 1 to (Nr+1)*Nb-1
  dw[i]=w[i]
end for
for round = 1 step 1 to Nr-1
  InvMixColumns(dw[round*Nb, (round+1)*Nb-1])
end for
//=====
```

Примечание. В последнем операторе (в функции *InvMixColumn()*) происходит преобразование типа, так как развернутый ключ хранится в виде линейного массива 32-разрядных слов, в то время как входной параметр функции - двумерный массив байтов.

В табл. 2.10 приведена процедура зашифрования, а также два эквивалентных варианта процедуры расшифрования при использовании двухраундового варианта *Rijndael*. Первый вариант функции расшифрования суть обычная инверсия функции зашифрования. Второй вариант функции зашифрования получен из первого после изменения порядка следования операций в трех парах преобразований:

InvShiftRows - *InvSubBytes* (дважды)

и *AddRoundKey* - *InvMixColumns*.

Таблица 2.10. Последовательность преобразований в двухраундовом варианте *RIJNDAEL*

Функция зашифрования двухраундового варианта <i>RIJNDAEL</i>	Функция обратного расшифрования двухраундового варианта <i>RIJNDAEL</i>	Эквивалентная функция прямого расшифрования двухраундового варианта <i>RIJNDAEL</i>
AddRoundKey	AddRoundKey	AddRoundKey
SubBytes	InvShiftRows	InvSubBytes
ShiftRows	InvSubBytes	InvShiftRows
MixColumns	AddRoundKey	InvMixColumns
AddRoundKey	InvMixColumns	AddRoundKey
SubBytes	InvShiftRows	InvSubBytes
ShiftRows	InvSubBytes	InvShiftRows
AddRoundKey	AddRoundKey	AddRoundKey

Очевидно, что результат преобразования при переходе от исходной к обратной последовательности выполнения операций в указанных парах не изменится.

Видно, что процедура зашифрования и второй вариант процедуры расшифрования совпадают с точностью до порядка использования раундовых ключей (при выполнении операций *AddRoundKey*), таблиц замен (при выполнении операций *SubBytes* и *InvSubBytes*) и матриц преобразования (при выполнении операций *MixColumns* и *InvMixColumns*). Данный результат легко обобщить и на любое другое число раундов.

2.2.3. Основные особенности *RIJNDAEL*

В заключение сформулируем *основные особенности RIJNDAEL*:

- * новая архитектура "Квадрат", обеспечивающая быстрое рас-сеивание и перемешивание информации, при этом за один ра-унд преобразованию подвергается весь входной блок;
- » байт-ориентированная структура, удобная для реализации на 8-разрядных МК;
- * все раундовые преобразования суть операции в конечных по-лях, допускающие эффективную аппаратную и программную реализацию на различных платформах.

2.3. Аспекты реализации шифра

2.3.1. Программная реализация зашифрования

Алгоритм шифрования *RIJNDAEL* эффективно реализуется на наиболее распространенных 8- и 32-разрядных платформах. Далее рассматриваются приемы, оптимизирующие программную реализацию на 8-разрядных (применяются в smart-картах) и 32-разрядных (наиболее широко используемых в настоящее время) процессорах.

2.3.1.1.8-разрядные процессоры

На 8-разрядных процессорах алгоритмы шифрования реализуются прямым выполнением операций, входящих в состав соответствующих преобразований. Так, побайтовый сдвиг строк *ShiftRows()* и сложение по модулю 2 с раундовым ключом *AddRoundKey()* реализуются напрямую тривиальными операциями. Для реализации преобразования *SubBytes()* необходима таблица замен размером 256 байт.

Операции преобразований *AddRoundKey()*, *ShiftRows()* и *SubBytes()* могут быть объединены в одно преобразование и выполняться последовательно по мере поступления блоков *State* открытых данных.

Преобразование *MixColumns()* является перемножением матриц:

$$\begin{bmatrix} s'_{0c} \\ 4 \\ 4 \\ s'_{3c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0c} \\ s_{1c} \\ s_{2c} \\ s_{3c} \end{bmatrix}, 0 \leq c \leq 3,$$

где c - номер столбца, или в более развернутом виде

$$\begin{aligned} s'_{0c} &= (\{02\} \cdot s_{0c}) \oplus (\{03\} \cdot s_{1c}) \oplus s_{2c} \oplus s_{3c} \\ s'_{1c} &= s_{0c} \oplus (\{02\} \cdot s_{1c}) \oplus (\{03\} \cdot s_{2c}) \oplus s_{3c} \\ s'_{2c} &= s_{0c} \oplus s_{1c} \oplus (\{02\} \cdot s_{2c}) \oplus (\{03\} \cdot s_{3c}) \\ s'_{3c} &= (\{03\} \cdot s_{0c}) \oplus s_{1c} \oplus s_{2c} \oplus (\{02\} \cdot s_{3c}) \end{aligned}$$

или

$$\begin{aligned} s'_{0c} &= s_{0c} \oplus (\{02\} \cdot (s_{0c} \oplus s_{1c})) \oplus s_{0c} \oplus s_{1c} \oplus s_{2c} \oplus s_{3c} \\ s'_{1c} &= s_{1c} \oplus (\{02\} \cdot (s_{1c} \oplus s_{2c})) \oplus s_{0c} \oplus s_{1c} \oplus s_{2c} \oplus s_{3c} \\ s'_{2c} &= s_{2c} \oplus (\{02\} \cdot (s_{2c} \oplus s_{3c})) \oplus s_{0c} \oplus s_{1c} \oplus s_{2c} \oplus s_{3c} \\ s'_{3c} &= s_{3c} \oplus (\{02\} \cdot (s_{3c} \oplus s_{0c})) \oplus s_{0c} \oplus s_{1c} \oplus s_{2c} \oplus s_{3c} \end{aligned}$$

где $\oplus$ и $\cdot$ - соответственно операции сложения (поразрядный XOR) и умножения в поле $GF(2^8)$.

Полученное выражение для байтов одного столбца может быть реализовано с помощью описанной выше функции умножения *xtime()* следующим образом:

$$Tmp = s[0] \oplus s[1] \oplus s[2] \oplus s[3],$$

где $s[i]$ - вектор из 4 байт i -го столбца;

$$\begin{aligned}
 Tm &= s[0] \oplus s[1]; Tm = xtime(Tm); s[0] = s[0] \oplus Tm \oplus Tmp; \\
 Tm &= s[1] \oplus s[2]; Tm = xtime(Tm); s[1] = s[1] \oplus Tm \oplus Tmp; \\
 Tm &= s[2] \oplus s[3]; Tm = xtime(Tm); s[2] = s[2] \oplus Tm \oplus Tmp; \\
 Tm &= s[3] \oplus s[0]; Tm = xtime(Tm); s[3] = s[3] \oplus Tm \oplus Tmp,
 \end{aligned}$$

Чтобы исключить возможность *временного анализа*, основанного на оценке времени выполнения процессором определенных операций (в нашем случае операции умножения), зависящего от значений операндов, необходимо функцию *xtime()* запрограммировать так, чтобы она осуществлялась за одинаковое число машинных циклов независимо от значения умножаемого байта. Например, можно создать таблицу результатов функции и выбирать из нее значения на основании конкретного входного байта.

Очевидно, что развертывание ключа сразу, т. е. перед началом процедур шифрования или расшифрования, требует сравнительно много (в масштабах smart-карт) оперативной памяти. Более того, в большинстве случаев, таких как *дебитовые* карточки, электронные деньги, количество обрабатываемой информации невелико (порядка нескольких 128-разрядных блоков), а значит, выигрыш от единовременного развертывания ключа для одного сеанса шифрования невелик. В то же время алгоритм развертывания ключа позволяет реализовать его в буфере размером всего $4N_k$ байт. Таким образом, если разворачивать ключ побайтно непосредственно перед каждым раундом, то за счет некоторого увеличения количества операций можно получить значительную экономию памяти.

2.3.1.2. 32-разрядные процессоры

Раундовое преобразование. Все преобразования, выполняемые за один раунд шифрования, могут быть объединены в несколько обращений к вычисляемым таблицам замены, что позволяет добиться значительной эффективности реализации алгоритма на 32-битных процессорах.

Если обозначить через *e* выходные данные (результат преобразования *AddRoundKey()*), получающиеся на выходе одного раунда преобразований (иначе говоря, одной итерации цикла преобразований) блока открытого текста *a*, тогда имеем следующие соотношения для *AddRoundKey()* и *MixColumns()*, записанные в матричной форме:

$$\begin{bmatrix} e_{0j} \\ e_{1j} \\ e_{2j} \\ e_{3j} \end{bmatrix} = \begin{bmatrix} d_{0j} \\ d_{1j} \\ d_{2j} \\ d_{3j} \end{bmatrix} \oplus \begin{bmatrix} k_{0j} \\ k_{1j} \\ k_{2j} \\ k_{3j} \end{bmatrix}, j = \overline{0,3}, \quad \begin{bmatrix} d_{0j} \\ d_{1j} \\ d_{2j} \\ d_{3j} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} c_{0j} \\ c_{1j} \\ c_{2j} \\ c_{3j} \end{bmatrix},$$

где d - результат преобразования $MixColumnsQ$, c - результат преобразования $ShiftRowsQ$.

Для преобразований $ShiftRowsQ$ и $SubBytesQ$ можем записать следующее соотношение

$$\begin{bmatrix} c_{0j} \\ c_{1j} \\ c_{2j} \\ c_{3j} \end{bmatrix} = \begin{bmatrix} b_{0j} \\ b_{1j-C_1} \\ b_{2j-C_2} \\ b_{3j-C_3} \end{bmatrix},$$

где b - результат преобразования $SubBytesQ$, $b_{ij} = S[a_{ij}]$, $C_1 = 1$, $C_2 = 2$, $C_3 = 3$ и номер столбца вычисляется по модулю Nb , то есть 4.

Подставляя последовательно промежуточные результаты в формулу выходных данных, получим:

$$\begin{bmatrix} e_{0j} \\ e_{1j} \\ e_{2j} \\ e_{3j} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} S[a_{0j}] \\ S[a_{1j-C_1}] \\ S[a_{2j-C_2}] \\ S[a_{3j-C_3}] \end{bmatrix} \odot \begin{bmatrix} k_{0j} \\ k_{1j} \\ k_{2j} \\ k_{3j} \end{bmatrix},$$

что может быть переписано в следующем виде:

$$\begin{bmatrix} "e_{0j}" \\ e_{1j} \\ e_{2j} \\ e_{3j} \end{bmatrix} = S[a_{0j}] \begin{bmatrix} "02" \\ 01 \\ 01 \\ 03 \end{bmatrix} \oplus S[a_{1j-C_1}] \begin{bmatrix} "03" \\ 02 \\ 01 \\ 01 \end{bmatrix} \odot \begin{bmatrix} k_{0j} \\ k_{1j} \\ k_{2j} \\ k_{3j} \end{bmatrix} \oplus S[a_{2j-C_2}] \begin{bmatrix} 01 \\ 03 \\ 02 \\ 01 \end{bmatrix} \oplus S[a_{3j-C_3}] \begin{bmatrix} 01 \\ 01 \\ 03 \\ 02 \end{bmatrix} \odot \begin{bmatrix} k_{0j} \\ k_{1j} \\ k_{2j} \\ k_{3j} \end{bmatrix},$$

где $S[a_{ij}]$ - подстановки соответствующих байт, получаемых из таблицы замен S -блока.

Таким образом, мы можем определить четыре таблицы подстановки:

$$T_0[a] = \begin{bmatrix} S[a] \bullet 02 \\ \text{ад} \\ \text{ад} \\ S[a] \bullet 03 \end{bmatrix}, \quad T_1[a] = \begin{bmatrix} S[a] \bullet 03 \\ \bullet 02 \\ S[a] \\ S[a] \end{bmatrix},$$

$$T_2[a] = \begin{bmatrix} S[a] \\ S[a] \bullet 03 \\ S[a] \bullet 02 \\ S[a] \end{bmatrix}, \quad T_3[a] = \begin{bmatrix} S[a] \\ S[a] \\ S[a] \bullet 03 \\ S[a] \bullet 02 \end{bmatrix}.$$

Все четыре таблицы (каждая из которых служит для выборки одного 4-байтового слова из 256 возможных на основании значения байта из обрабатываемого столбца) в сумме занимают 4 Кбайт памяти. Путем использования этих таблиц один раунд процедуры зашифрования может быть выражен следующим образом:

$$e_j = T_0[a_{0j}] \oplus T_1[a_{1j-C_1}] \oplus T_2[a_{2j-C_2}] \oplus T_3[a_{3j-C_3}] \oplus k_j.$$

Это преобразование проиллюстрировано на рис. 2.11.

Таким образом, посредством использования таблиц общим объемом 4 Кбайт, один раунд процедуры шифрования осуществляется всего шестнадцатью обращениями к T -таблицам (по четыре на каждый столбец входного массива данных $State$) и четырьмя операциями сложения по модулю 2.

Можно также заметить, что таблицы отличаются лишь циклическим сдвигом на 1 байт вправо (или вниз, если смотреть на вышеприведенную формулу), что делает возможным реализовать преобразование, используя 1 Кбайт вместо 4 Кбайт памяти за счет добавления операции циклического побайтового сдвига. Такая экономия памяти оправдана в апплетах, где вообще можно строить T -таблицы "на лету" вместо хранения их в коде апплета.

Тот факт, что в последнем раунде отсутствует операция $MixColumns()$, ведет лишь к тому, что вместо T -таблиц нужно использовать S -блок.

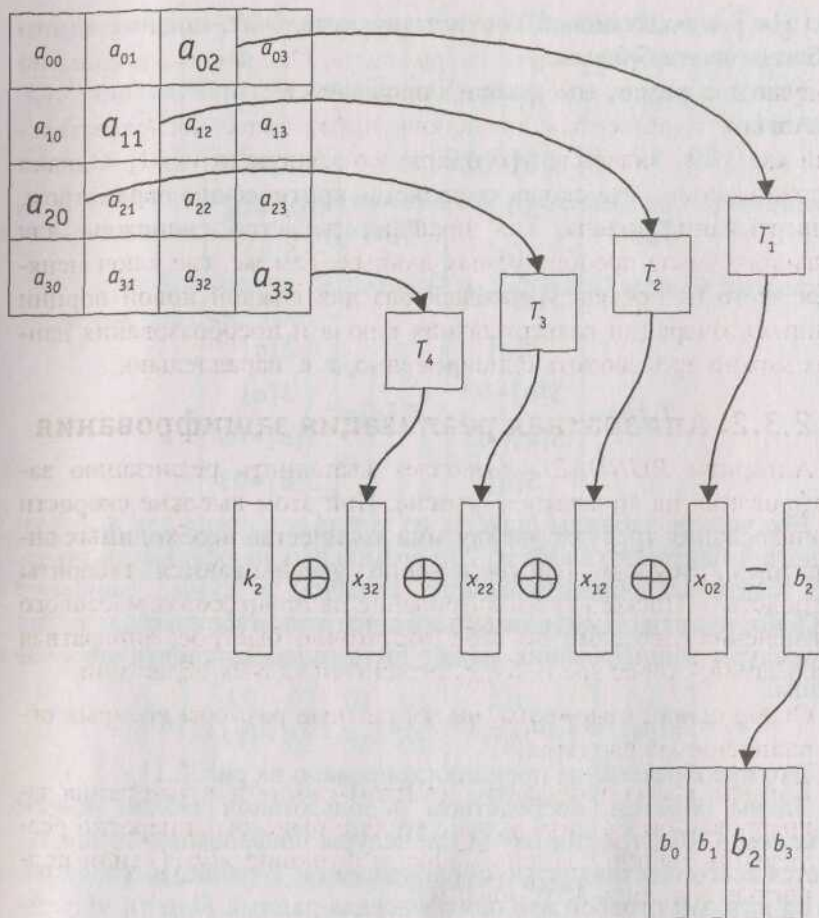


Рис. 2.11. Раундовое преобразование блока данных с использованием T -таблиц

Параллелизм. Исходя из определенных в одном раунде преобразований, можно утверждать, что в алгоритме заложена высокая степень параллелизма. Действительно, все четыре основные операции преобразования могут работать параллельно либо с несколькими байтами (функции *SubBytes()* и *AddRoundKey()*), либо со строками массива *State* (функция *ShiftRows()*), либо со столбцами массива *State* (функция *MixColumns()*).

При использовании T -таблиц все четыре обращения к ним можно также осуществлять параллельно. Операция сложения по модулю два также легко распараллеливается.

Алгоритм развертывания ключа имеет более последовательный характер: значение $w[i]$ напрямую зависит от $w[i-1]$. Однако в приложениях, где скорость является критическим параметром, развертывание ключа, как правило, делается единожды для большого числа преобразуемых данных. Там же, где ключ меняется часто (в пределе - каждый раз для каждой новой порции данных), операции развертывания ключа и преобразования данных можно производить одновременно, т. е. параллельно.

2.3.2. Аппаратная реализация зашифрования

Алгоритм *RIJNDAEL* позволяет выполнить реализацию зашифрования на аппаратном уровне. При этом высокие скорости зашифрования требуют увеличения количества необходимых аппаратных ресурсов (соответственно увеличиваются габариты устройства). Поскольку зашифрование на процессорах массового применения уже само по себе достаточно быстрое, аппаратная реализация, скорее всего, будет ограничена двумя областями.

- Особо скоростные чипы, на габаритные размеры которых ограничения не вводятся.
- Компактные сопроцессоры на smart-картах для ускорения зашифрования. На этом уровне удобно было бы аппаратно реализовать таблицу замен S -блока и функцию *xtime()* (или полностью функцию *MixColumns()*).

2.3.3. Программная реализация расшифрования

Для алгоритма обратного расшифрования неприменим метод таблиц подстановки. При зашифровании можно взять байты из соответственно сдвинутых столбцов и, используя их как индексы, определить 32-разрядные значения в T -таблицах, которые затем просто складываются по модулю 2 с раундовым ключом. При расшифровании преобразования осуществляются в обратном порядке. А это означает, что перед тем как к входным данным (при расшифровании это блок шифротекста) будет приложена функ-

ция перемешивания байт *InvMixColumnsQ*, необходимо эти данные сложить по модулю 2 с неизвестным заранее раундовым ключом, что исключает возможность заранее подготовить *T*-таблицы. Таким образом становится невозможным заменить вычислительно громоздкую операцию *InvMixColumnsQ* на простую подстановку значений из *T*-таблиц. Но для алгоритма прямого расшифрования применение *T*-таблиц также возможно, как и для алгоритма зашифрования.

Необходимо отметить, что выбор коэффициентов многочлена

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$$

в функции *MixColumns()* был отчасти продиктован оптимизацией алгоритма зашифрования. Поэтому, хотя прямое расшифрование аналогично по порядку применения операций-функций алгоритму зашифрования, применение других коэффициентов многочлена

$$a^{-1}(x) = \{0b\}x^3 + \{0d\}x^2 + \{09\}x + \{0e\}$$

в функции *InvMixColumnsQ* и изменения в алгоритме развертывания ключа (добавление преобразования *InvMixColumnsQ* при формировании всех раундовых ключей за исключением первого и последнего) ведет к некоторому уменьшению производительности. Это уменьшение (приблизительно 30%) наблюдается при реализации на 8-разрядных процессорах.

Такая асимметрия объясняется тем, что производительность расшифрования рассматривается как менее важный параметр по сравнению с производительностью зашифрования, поскольку во многих случаях расшифрование вообще не предполагается. Это верно, например, при вычислении кода аутентификации сообщения (*MAC* - Message Authentication Code), а также для поточных режимов шифрования с обратной связью по шифротексту (*CFB* - Cipher FeedBack), обратной связью по выходу (*OFB* - Output FeedBack) и счетчика (*Counter*).

2.3.3.1. 8-разрядные процессоры

Как было отмечено в описании реализации алгоритма зашифрования на 8-разрядных процессорах, операция *MixColumnsQ* выполняется достаточно эффективно благодаря значениям коэффициентов $\{01\}$, $\{02\}$ и $\{03\}$. Умножение при таких коэффициентах можно легко производить с использованием функции *xtime()*. По-

скольку для *InvMixColumns()* коэффициенты равны {09}, {0e}, {0b} и {0d}, то использование функции *xtime()* уже не столь эффективно. Значительного ускорения можно добиться применением таблиц подстановки результатов *xtime()*, хотя такое решение и приводит к использованию дополнительной памяти.

Развертывание ключа определено таким образом, что имея последние *NIC* 32-разрядных слов развернутого ключа (ключ, использовавшийся в последнем раунде), мы всегда можем вернуться назад к начальному ключу шифрования. Иначе говоря, пошаговое вычисление раундового ключа для расшифрования вполне осуществимо.

2.3.3.2. 32-разрядные процессоры

Каждый раунд алгоритма прямого расшифрования может быть реализован аналогично зашифрованию посредством четырех обращений к таблицам подстановки и сложению по модулю 2 с раундовым ключом. Таблицы подстановки, разумеется, будут отличаться от *T*-таблиц, применяемых в зашифровании, в остальном же отличий нет, поэтому никакого уменьшения производительности быть не должно. Оно происходит только за счет изменения в алгоритме разворачивания ключа, так как для всех раундов овых ключей, кроме первого и последнего, необходимо еще приложить функцию *InvMixColumns()*.

2.3.4. Аппаратная реализация прямого расшифрования

Применение различных преобразований при зашифровании и расшифровании не позволяет произвести обе процедуры на базе одной и той же схемы. Однако некоторые цепи все же могут быть общими для обоих режимов.

Рассмотрим, например, нелинейное преобразование подстановки

$$S(x) = f(g(x)),$$

где $g(x)$ - замена $x \rightarrow x^{-1}GF(2^8)$, а $f(x)$ - аффинное преобразование. Замена $g(x)$ обратна самой себе, откуда

$$S^{-1}(x) = g^{-1}(f^{-1}(x)) = g(f^{-1}(x)).$$

Таким образом, для S и S^{-1} достаточно реализовать g, f и $/^{-1}$. Поскольку обе функции f и f^{-1} являются тривиальными битовыми преобразованиями, не требующими много аппаратуры для реализации, то общее количество аппаратуры можно значительно уменьшить по сравнению с полной реализацией двух S -блоков.

То же самое применимо для реализации функции $xtime()$ при перемешивании байт в столбцах.

2.4. Характеристики производительности

В табл. 2.11 и 2.12 приведены данные по времени выполнения *RIJNDAEL* при его реализации на ассемблере двух 8-разрядных МП, наиболее типичных для применения в smart-картах, а именно, Intel 8051 и Motorola 68HC08.

Таблица 2.11. Реализация *RIJNDAEL* на МП Intel 8051

Длина ключа / размер блока, бит	Число циклов (1 цикл равен 12 периодам тактовой частоты)	Размер кода, байт
(128, 128) Вариант 1	4065	768
(128, 128) Вариант 2	3744	826
(128, 128) Вариант 3	3168	1016
(192, 128)	4512	1125
(256, 128)	5221	1041

Таблица 2.12. Реализация *RIJNDAEL* на МП Motorola 68HC08

Длина ключа / размер блока, бит	Число циклов	Объем RAM, байт	Размер кода, байт
(128, 128) Вариант 1	8390	36	919
(192, 128)	10780	44	1170
(256, 128)	12490	52	1135

В табл. 2.13 приведены данные Б. Гладмана по времени выполнения *RIJNDAEL* при его реализации на процессорах семейств Pentium Pro и Pentium II. Результаты были получены с использованием компилятора Visual C++ (версия 6).

Таблица 2.13. Реализация *RIJNDAEL* на 32-разрядных процессорах Pentium Pro и Pentium II

Длина ключа / размер блока, бит	Количество циклов, необходи- мых для разворачивания ключа		Производительность шифрования	
	<i>RIJNDAEL</i>	<i>RIJNDAEL</i> ⁻¹	Скорость, Мбит/сек	Число циклов / блок
(128, 128)	305	1389	70, 5	363
(192, 128)	277	1595	59, 3	432
(256, 128)	374	1960	51, 2	500

2.5. Обоснование выбранной структуры шифра

В данном разделе приводится обоснование выбора преобразований, констант и структуры шифра *RIJNDAEL*, сделанное авторами криптоалгоритма. Этот выбор обеспечивает, по их убеждению, отсутствие возможностей реализации "черного хода" (или, иначе говоря, ловушек, люков).

2.5.1. Неприводимый многочлен $\phi(x)$

Многочлен $\phi(x) = x^8 + x^4 + x^3 + x + 1$, используемый для построения поля $GF(2^8)$, является первым неприводимым многочленом восьмой степени, упоминающимся в большинстве справочников. То есть его выбор достаточно произволен.

2.5.2. S-блок, используемый в преобразовании *SubBytes()*

Блоки замен в шифре *RIJNDAEL* играют важную роль. Согласно основополагающим принципам, сформулированным еще К. Шенноном, преобразования данных, используемые в шифре, должны придавать последнему два основных свойства - рассеивание и перемешивание. Рассеивание предполагает распространение влияния каждого бита открытого текста, а также каждого бита ключа на значительное количество битов шифротекста. Перемешивание же приводит к потере в процессе шифрования всяческих зависимостей между битами открытого текста. То есть на выходе мы получаем данные, как если бы мы выбрали их совер-

шенно случайным образом, а не как значение функций преобразования шифрующего алгоритма. Именно эти два свойства обеспечивают защиту от двух возможных угроз: подделки сообщения и его раскрытия.

За обеспечение этих двух свойств отвечают функции *MixColumns()* и *SubBytes()*. Рассеивание в шифре *RIJNDAEL* обеспечивается в основном функцией *MixColumns()*, перемешивание — в основном функцией *SubBytes()*. Таким образом, критерии выбора и разработки таблицы замен *S*-блоков обусловлены, с одной стороны, учетом возможностей дифференциального и линейного криптоанализа (будут рассмотрены далее), а с другой — учетом возможных алгебраических манипуляций, например, атаки методом интерполяции (будет рассмотрена ниже). Таким образом, основными критериями выбора таблицы замен *S*-блоков стали:

- * обратимость;
- * минимизация наидлиннейшей возможной нетривиальной корреляции между линейными комбинациями бит входных и выходных данных; иначе говоря, для всех случайно выбранных входных данных количество раундов, в которых еще можно проследить зависимости между входными и выходными данными, будет минимально; именно прослеживание таких зависимостей является основой линейного криптоанализа;
- минимизация наибольшего нетривиального значения *XOR* таблицы; характеризует устойчивость к дифференциальному криптоанализу;
- * сложность алгебраического представления в поле $GF(2^8)$; важно для устойчивости к алгебраическим атакам;
- простота описания.

В [18] приведено несколько методов конструирования таблиц *S*-блоков, удовлетворяющих первым трем критериям. Для обратимой байтовой таблицы замен максимальная корреляция входных/выходных данных может быть минимизирована до 2^{-3} , а максимальная величина *XOR* таблицы — минимизирована до 4 (что соответствует коэффициенту дифференциального проникновения 2^{-6}).

Для формирования таблицы замен было выбрано отображение $x \rightarrow x^{-1}$ (мультипликативная инверсия) в поле $GF(2^8)$. Однако очевидно, что выбранное отображение имеет слишком простое алгебраическое представление. Это делает возможным использование алгебраических манипуляций в таких атаках на шифр, как, например, атака методом интерполяции [10]. Поэтому результат преобразования подвергается дополнительному (вполне обратимому) изменению. Это изменение не умаляет свойств S-блока в отношении первых трех критериев, но позволяет удовлетворить требование соответствия таблицы замен четвертому критерию выбора. Было выбрано такое изменение, которое само по себе очень простое в описании, но при этом, в сочетании с нахождением мультипликативного обратного элемента в поле $GF(2^8)$, имеет сложное алгебраическое представление. Оно может быть представлено как перемножение многочленов по модулю $x^8 + 1$ с последующим сложением:

$$b(x) - (x^6 + x^5 + x + 1) + a(x)(x^7 + x^6 + x^5 + x^4 + 1) \bmod (x^8 + 1).$$

Здесь $a(x)$ - преобразуемый байт в виде многочлена, $b(x)$ - результирующий байт. Модуль $x^8 + 1$ был выбран как самый простой из возможных. Сомножитель $x^7 + x^6 + x^5 + x^4 + 1$ был выбран из набора многочленов, взаимно простых с модулем $x^8 + 1$, как имеющий наиболее простое представление. А многочлен $x^6 + x^5 + x + 1$ был выбран таким образом, чтобы полученная в результате таблица замен не имела точек симметрии ($S(a) = a$) и точек обратной симметрии ($S(a) = a^{-1}$).

Примечание. Существуют также и другие S-блоки, удовлетворяющие вышеперечисленным критериям. Так что в случае подозрения на существование в данной таблице "черного хода" она может быть легко заменена на другую. Более того, структура шифра и выбранное количество раундов позволяют применить такую таблицу замен, которая не удовлетворяет критериям 2 и 3. Даже "средняя" в этом отношении таблица замен тем не менее будет обеспечивать стойкость к дифференциальному и линейному криптоанализу.

2.5.3. Функция *MixColumns()*

Функция рассеивания *MixColumnsQ* была выбрана из возможных линейных преобразований 4 bytes $\rightarrow$ 4 bytes исходя из следующих критериев выбора:

- « обратимость;
- » линейность в поле $GF(2)$; это свойство функции *MixColumnsQ* позволяет применять алгоритм прямого расшифрования;
- * достаточно сильное рассеивание;
- * скорость выполнения на 8-разрядных процессорах;
- в симметричность, т. е. *MixColumnsQ* должна работать со всеми данными единообразно;
- простота описания.

Критерии 2, 5 и 6 обусловили выбор произведения многочленов по модулю $x^4 + 1$. Критерии 1, 3 и 4 определили выбор коэффициентов сомножителей. Критерий 4 определял выбор коэффициентов (в порядке предпочтительности) - $\{00\}$, $\{01\}$, $\{02\}$, $\{03\}$... Значение $\{00\}$ не предполагает вообще никакого преобразования, $\{01\}$ - не нужно производить умножения, $\{02\}$ - умножение может быть выполнено при помощи функции *xtimeQ*, и $\{03\}$ - при помощи функции *xtimeQ* и последующего XOR.

Третий критерий определяет более сложные условия выбора коэффициентов. Стратегия разработчиков шифра учитывала следующие требования к преобразованию *MixColumnsQ*. Пусть F будет линейным преобразованием, выполняемым над векторами байт, и пусть *байтовым весом* вектора будет количество ненулевых байт в нем. Тогда *коэффициентом распространения* линейного преобразования, характеризующим его силу рассеивания, будем называть величину

$$\min_{a \neq 0} (W(a) + W(F(a))),$$

где $W(a)$ - байтовый вес. Байт, не равный нулю, будем называть *активным байтом*. Для функции *MixColumnsQ* при одном активном байте входного 32-разрядного массива *State* на выходе будет максимум 4 активных байта, так как эта функция преобразует столбцы по отдельности. Таким образом, верхняя граница коэф-

фициента распространения равна 5. Коэффициенты {01}, {02} и {03} и были подобраны так, чтобы получить это максимально возможное значение коэффициента распространения. То, что оно равно 5, говорит о том, что разница во входных данных в 1 байт рассеивается (распространяется) на 4 байта выходных данных, разница в 2 байтах входных данных отразится, как минимум, на 3 байтах выходных данных. Более того, можно сказать, что линейная зависимость между битами во входных и выходных данных всегда затрагивает биты как минимум пяти различных байт на входе и выходе функции *MixColumns()*.

2.5.4. Смещения в функции *ShiftRows()*

Выбор из возможных комбинаций смещений в строках преобразуемого блока был сделан на основании следующих критериев:

- * все четыре смещения должны быть различны для каждой строки и $C_0 = 0$;
- * стойкость к атакам, использующим сокращенные дифференциалы (см. ниже, а также [13]);
- * стойкость к атаке *Square* (будет описана ниже, также см. [6]);
- простота.

Для некоторых комбинаций значений смещений атаки с использованием сокращенных дифференциалов становятся эффективны для более чем одного раунда. Для других комбинаций более эффективны атаки типа *Square*. Из набора комбинаций смещений, наилучшим образом удовлетворяющих пунктам 2 и 3, была выбрана простейшая.

2.5.5. Алгоритм разворачивания ключа

Алгоритм разворачивания ключа определяет порядок получения раундовых ключей из начального ключа шифрования. Алгоритм разворачивания должен обеспечивать стойкость против следующих типов атак:

- * атак, в которых часть начального ключа шифрования криптоаналитику известна;

- атак, в которых ключ шифрования известен заранее или может быть выбран, например, если шифр применяется для компрессии данных при хешировании;
- * атак "эквивалентных ключей" [3, 12]; необходимым условием стойкости к таким атакам является отсутствие слишком больших одинаковых наборов раундовых ключей, полученных из двух разных начальных ключей шифрования.

Процедура разворачивания ключа также играет важную роль в исключении:

- * симметрии однораундового преобразования, которое обрабатывает все входные байты единообразно; для ее устранения в процедуре разворачивания ключа при получении каждого первого 32-разрядного слова раундового ключа используются функция *SubWord(RotWord())* и раундовая константа;
- » межраундовой симметрии (раундовое преобразование одинаково для всех итераций цикла преобразования); для ее нарушения в алгоритм разворачивания ключа введены константы, значения которых различны для каждого раунда.

Таким образом, алгоритм разворачивания выбран в соответствии со следующим критериями:

- » обратимость используемых преобразований, т. е. из любых N_k последовательных слов развернутого ключа можно однозначно восстановить весь развернутый ключ;
- » хорошая скорость на различных типах процессоров;
- * наличие раундовых констант для уменьшения симметричности;
- хорошее рассеивание изменений в начальном ключе шифрования на формирующийся раундовый ключ;
- * отсутствие возможности на основе знания части бит раундового или начального ключа вычислить значительную часть остальных бит;
- * достаточная нелинейность для предупреждения полного определения межбитовых зависимостей развернутого раундового

ключа лишь на основании знания таких зависимостей в начальном ключе шифрования;

- * простота описания.

Была выбрана байт-ориентированная схема алгоритма, которая может эффективно реализовываться на 8-разрядных процессорах. Применение *SubBytes()* обеспечивает нелинейность преобразования и требует небольшого дополнительного пространства памяти на 8-разрядных процессорах.

2.5.6. Количество раундов

Авторы шифра определили количество раундов, учитывая максимальное количество раундов, для которых еще возможна атака, более эффективная, чем полный перебор по всему ключевому пространству (так называемая сокращенная атака), и прибавили запас в виде дополнительных раундов.

Для упрощенной версии шифра из 6 раундов со 128-битовыми блоком данных и ключом не было выявлено ни одной эффективной сокращенной атаки. Добавление еще 4 раундов с учетом следующих соображений является более чем достаточным запасом прочности.

Два раунда шифра обеспечивают полное рассеивание в том смысле, что каждый бит блока *State* зависит от всех бит этого же блока "двухраундовой давности", или, иначе говоря, изменение одного бита блока *State* с большой вероятностью отразится на половине бит этого блока через 2 раунда. Таким образом, дополнительные 4 раунда могут рассматриваться как добавление шага полного рассеивания в начале и в конце процедуры шифрования. Высокая степень рассеивания шифра обусловлена целостностным характером алгоритма, преобразующего сразу все биты входных данных. Для сетей Фейстеля раундовое преобразование за шаг изменяет только половину бит входных данных, и полное рассеивание на практике достигается не менее, чем за 4 и более раундов.

При линейном и дифференциальном криптоанализе, при проведении атаки методом сокращенных дифференциалов используют зависимости, которые можно проследить сквозь n раундов для того, чтобы атаковать $(n+1)$ -й или $(n+2)$ -й раунд. Это же верно и для *Square-атак*, использующих зависимости сквозь 4 раунд-

да для атаки на шестой раунд. Для последних дополнительные 4 раунда означают удваивание количества раундов, сквозь которые необходимо отслеживать зависимости.

Для версий шифра с более длинным ключом количество раундов увеличивается на единицу для каждого дополнительных 32 разрядов ключа шифрования, исходя из следующих соображений.

Одна из главных целей - невозможность сокращенных атак. Поскольку с ростом длины ключа трудоемкость простого перебора возрастает, то и сокращенная атака может позволить себе быть более трудоемкой.

Атаки по известному (или частично известному) ключу, атаки с использованием "эквивалентных ключей" основаны соответственно на наличии информации о битах начального ключа шифрования и возможности применения нескольких различных ключей шифрования. Если ключ удлиняется, то и возможностей у криптоаналитика становится больше.

Поскольку никаких эффективных атак по известному ключу или с использованием "эквивалентных ключей" для шифра с шестью раундами найдено не было, то остальные раунды могут рассматриваться как дополнительный запас стойкости.

Для других версий шифра *RIJNDAEL* (не включенных в стандарт) с блоками входных данных, размер которых больше 128 разрядов, количество раундов также увеличивается на единицу с каждым дополнительным 32-разрядным словом, исходя из следующих соображений.

Для блоков размером более 128 разрядов полное рассеивание наступает не ранее, чем после проведения третьего раунда преобразований, т. е. относительное рассеивание раунда уменьшается с ростом размера блока входных данных.

С ростом размера блока входных данных также увеличивается количество возможных комбинаций зависимостей входных/выходных данных. А это, в свою очередь, иногда позволяет продлить эффективность атаки еще на один раунд.

Авторы шифра утверждают тем не менее, что угроза распространения эффективности атаки еще на 1 раунд даже для 256-разрядного блока маловероятна. Поэтому все раунды, номер которых больше шести, можно рассматривать как дополнительный запас стойкости шифра.

2.6. Стойкость к известным атакам

2.6.1. Свойства симметричности и слабые ключи

Учитывая значительную степень присущей шифру симметричности (однородности) в работе с данными, были предприняты специальные меры, позволившие уменьшить ее влияние. Это достигается благодаря различным для каждого раунда константам (см. обоснование выбора алгоритма разворачивания ключа). Тот факт, что алгоритмы зашифрования и расшифрования состоят из различных операций-функций, практически исключает возможность существования слабых и полуслабых ключей шифрования (что имело место в *DES*). Нелинейность процедуры разворачивания ключа также практически исключает возможность получения эквивалентных раундовых ключей после разворачивания различных начальных ключей шифрования.

2.6.2. Дифференциальный и линейный криптоанализ

Дифференциальный криптоанализ был впервые описан Э. Бихамом и А. Шамиром [4]. Линейный криптоанализ был впервые описан М. Мацуи [16]. В [5] приводится детальное объяснение таких ключевых понятий, как дифференциальное проникновение и корреляция. Все эти методы будут подробно рассмотрены в четвертой книге настоящей серии.

2.6.2.1. Дифференциальный криптоанализ

Дифференциальный криптоанализ (ДК) - это атака на шифр, базирующаяся на возможности свободного подбора открытых текстов для последующего их зашифрования. Анализуются зависимости между парами блоков данных до и после применения шифра. ДК-атаки становятся возможны тогда, когда можно установить проникновение таких зависимостей сквозь почти все (за исключением 2-3) раунды шифрующего преобразования. При этом относительное количество фиксированных пар бит данных (назовем это число *коэффициентом проникновения*), для которых можно установить зависимость различий на выходе

от различий на входе при зашифровании, должно быть значительно выше, чем 2^{1-n} , где n - длина входных данных в битах. Попробуем объяснить этот критерий успешности дифференциального криптоанализа следующим образом. Представим себе, что мы можем произвольно выбирать два различных блока открытого текста длиной по n бит каждый для зашифрования и последующего анализа двух полученных блоков шифротекста. Напомним, мы будем искать пары бит во входных данных, различия между которыми подсказали бы нам, каково будет различие между битами какой-то определенной пары бит в выходных данных. Таким образом, мы выбираем по одному определенному биту из каждого из двух блоков открытого текста и анализируем результаты зашифрования при *всех* возможных значениях входных данных. Поскольку теперь в каждом блоке осталось $n - 1$ битов, которые могут принимать различные значения, то нам нужно проанализировать всего 2^{n-1} значений каждого блока входных данных. И если обнаружится, что в выходных данных существует пара бит такая, что различия между этими битами можно предсказать, исходя из различий между битами выбранной пары во входных данных, и это событие не случайно (т. е. его вероятность больше, чем $1/2^{n-1} = 2^{1-n}$), то можно считать, что преобразование зашифрования имеет явную слабость и может быть подвергнуто дифференциальному криптоанализу.

Раунд зашифрования при дифференциальном криптоанализе удобно рассматривать как отдельную *разностную сессию* преобразования данных со своими входными и выходными данными. Полная разностная сессия, таким образом, состоит из отдельных сессий - раундов зашифрования, а полный коэффициент проникновения при зашифровании является произведением коэффициентов проникновения составляющих сессий.

Таким образом, для того чтобы обеспечить стойкость к ДК-атакам, достаточно чтобы коэффициент проникновения после некоторого числа раундов стал не больше 2^{1-n} . Далее будет приведено доказательство того, что уже после четырех раундов преобразования коэффициент проникновения становится не больше, чем 2^{-150} (а после 8 раундов - менее 2^{-300}), что учитывая данные табл. 2.5, доказывает стойкость шифра при различных n .

2.6.2.2. Линейный криптоанализ

Линейный криптоанализ (ЛК) также применяется в атаках с возможностью подбора открытых данных для их последующего зашифрования. Он основан на анализе *линейных зависимостей* между битами входных данных, которые обуславливают вполне определенные зависимости между битами выходных данных.

Пусть

$$A[i_1, i_2, i_3, \dots, i_a]$$

- сумма по модулю 2 бит массива данных A , индексами которых являются значения $i_1, i_2, i_3, \dots, i_a$, то есть

$$A[i_1, i_2, i_3, \dots, i_a] = A[i_1] \oplus A[i_2] \oplus A[i_3] \oplus \dots \oplus A[i_a].$$

Тогда линейной зависимостью будем называть выражение вида

$$P[i_1, i_2, i_3, \dots, i_a] = C[j_1, j_2, j_3, \dots, j_b] \oplus K[k_1, k_2, k_3, \dots, k_c],$$

где P , C и K означают, соответственно, массивы бит входных данных, выходных данных и ключа. Таким образом, если для достаточно большого количества различных P и C удастся подобрать K , удовлетворяющее данному выражению, то можно говорить о раскрытии одного бита информации о ключе на основании существования линейной зависимости входных и выходных данных. Существуют также методы определения более чем одного бита информации о ключе на основании анализа линейных зависимостей.

ЛК-атаки становятся эффективны тогда, когда можно установить существование таких линейных зависимостей (иначе - *корреляции*) после почти всех (за исключением 2-3) раундов шифрующего преобразования. При этом относительное число (коэффициент) таких корреляций в данных должно быть значительно выше чем $2^{-n/2}$ (то есть более, чем одна пара бит). Корреляции имеют знак, то есть могут быть отрицательными или положительными, в зависимости от того, совпадает ли зависимость во входных данных с зависимостью в выходных или же обратна ей. Это дает ключ (игра слов) к нахождению бит раундового ключа. Считается, что общая корреляция при шифровании является композицией (или суммой) всех корреляций, наблюдаемых в каждой из отдельных *линейных сессий* (т. е. в каждом раунде

преобразования), где есть предсказуемые комбинации бит во входных и выходных данных.

Достаточным условием стойкости к ЛК-атаке является отсутствие каких-либо линейных сессий с коэффициентом общей корреляции выше, чем $2^{-n/2}$. Далее будет приведено доказательство того, что уже после четырех раундов преобразования коэффициент корреляции становится меньше, чем 2^{-5} (а после 8 раундов меньше 2^{-150}).

2.6.2.3. Эффективность разностных и линейных сессий

В [5] показано, что:

- * полный коэффициент проникновения разностной сессии может быть аппроксимирован к произведению коэффициентов проникновения всех активных в данной разностной сессии таблиц замен S -блоков;
- * общая корреляция линейной сессии может быть аппроксимирована к произведению коэффициентов корреляции между входными и выходными данными всех активных в данной линейной сессии таблиц замен S -блоков.

Таким образом, стратегия сессии преобразования любого шифра должна заключаться в следующем:

- * выбрать S -блок такой, что максимальный коэффициент проникновения и максимальный коэффициент корреляции между входными и выходными данными были бы как можно меньше; для S -блоков *RIJNDAEL* они равны соответственно 2^{-6} и 2^{-3} ;
- сконструировать рассеивание таким образом, чтобы не было многораундовых преобразований с малым числом активных S -блоков.

Далее будет показано, что минимальное число активных S -блоков в любой 4-раундовой разностной или линейной сессии равно 25. Это дает коэффициент проникновения 2^{-150} для любой 4-раундовой разностной сессии и самое большее 2^{-75} для коэффициента корреляции любой 4-раундовой линейной сессии. Это верно для всех размеров блоков шифра и не зависит от значения раундового ключа.

Примечание. Нелинейность 5-блока, выбранного наугад из набора возможных обратимых 8-разрядных блоков замены, будет, скорее всего, меньше оптимальной. Типичны значения от 2^{-5} до 2^{-4} для максимального коэффициента проникновения и 2^{-2} для коэффициента корреляции между входными и выходными данными таблиц замены.

2.6.2.4. Проникновение образов активности

Для дифференциального криптоанализа количество *активных S-блоков* в одном раунде определяется количеством ненулевых байт, задающих различия во входных данных раунда (количество ненулевых байт после применения операции *XOR* к двум массивам входных данных). Пусть образ (pattern) массива *State*, определяющий позиции активных 5-блоков, обозначается термином (*разностный*) *образ активности*, и пусть (*разностный*) *байтовый вес* обозначает число активных (ненулевых) байт в образе.

Для линейного криптоанализа количество активных 5-блоков определяется количеством ненулевых байт в массиве входных данных. Пусть образ, определяющий позиции активных 5-блоков, обозначается термином (*корреляционный*) *образ активности*, и пусть (*корреляционный*) *байтовый вес* $W(a)$ будет количеством активных (ненулевых) байт в образе *a*. Более того, пусть столбец в образе активности называется активным, если он содержит хотя бы один активный байт. Пусть $W_c(a)$ - количество активных столбцов в *a*, т. е., по сути, *столбцовый вес* этого образа. *Байтовый вес столбца j* образа *a*, обозначаемый $W_j(a)$, задает количество активных байт в этом столбце.

Общий вес сессии равен сумме весов образов активности входных данных для каждого раунда, входящего в состав этой сессии.

Можно рассматривать *проникновение разностных* (линейных) *образов активности* сквозь раунды преобразования как проникновение зависимостей *входных/выходных* данных в разностной (линейной) сессии. Это проиллюстрировано на рис. 2.12, где серым цветом выделены активные байты.

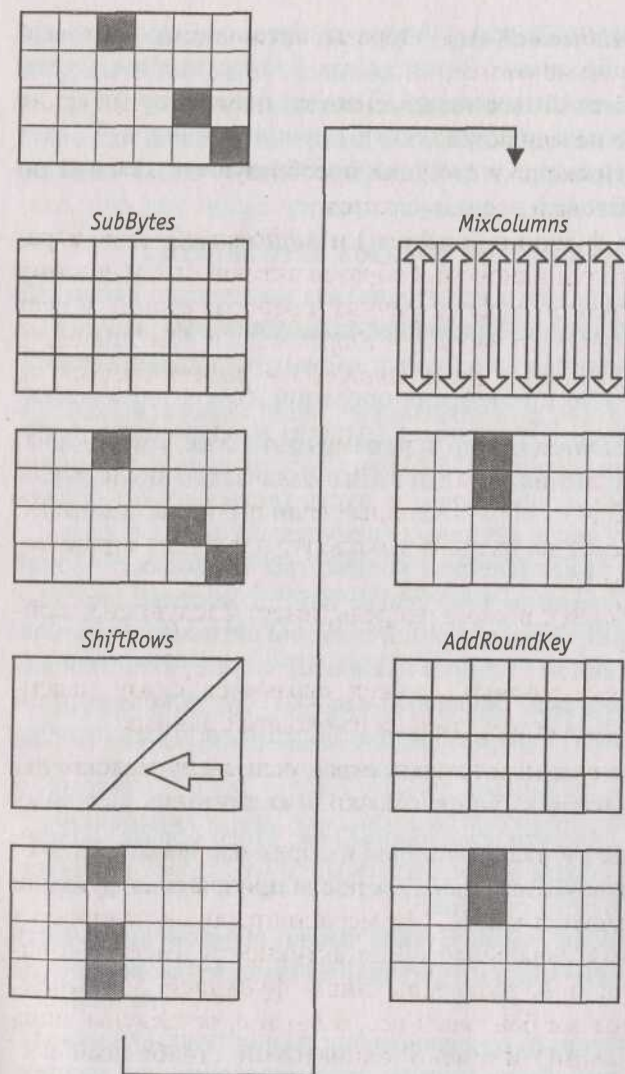


Рис. 2.12. Проникновение образов активности сквозь однораундовое преобразование

Функции, входящие в состав одного раунда преобразования, оказывают следующее влияние на образы активности.

SubBytesQ и *AddRoundKey()*. Образы активности, байтовый и столбцовый веса не меняются.

ShiftRowsQ. Байтовый вес не изменяется, поскольку значения самих байт вообще не меняются.

MixColumnsQ. Поскольку столбцы преобразуются каждый по отдельности, столбцовый вес не меняется.

Таким образом, функции *SubBytes()* и *AddRoundKey()* не играют никакой роли в проникновении образов активности, и поэтому в нашем рассмотрении эффект раунда преобразований может быть сведен к эффекту от функций *ShiftRowsQ* и *MixColumnsQ*. В дальнейшем *SubBytesQ* и *AddRoundKey()* в расчет браться не будут.

Функция *MixColumnsQ*, как было отмечено выше, имеет коэффициент распространения, равный 5. Это означает, что для любого активного столбца из образа входных (или выходных) данных, сумма байтовых весов на входе и выходе раунда будет ограничена снизу значением 5.

Функция *ShiftRows()*, в свою очередь, имеет следующие свойства:

- * столбцовый вес выходных данных ограничен снизу максимальным байтовым весом столбца из входных данных;
- столбцовый вес входных данных ограничен снизу максимальным байтовым весом столбца из выходных данных.

Итак, пусть далее в нашем описании образ активности на входе g' -го раунда обозначается как a_{i-1} , а после приложения функции *ShiftRowsQ* обозначается как b_{i-1} . Нумерация раундов начинается с единицы, поэтому начальный образ активности имеет обозначение a_0 . Тогда a_i и b_i разделены лишь функцией *ShiftRowsQ* и имеют один и тот же байтовый вес, а b_{i-1} и a_i разделены лишь функцией *MixColumnsQ* и имеют одинаковый столбцовый вес. Общий вес m -раундовой сессии равен сумме весов образов от a_0 до a_{m-1} . Свойства проникновения образов активности проиллюстрированы на рис. 2.13, где темно-серым цветом выделены активные байты, а светло-серым - активные столбцы.

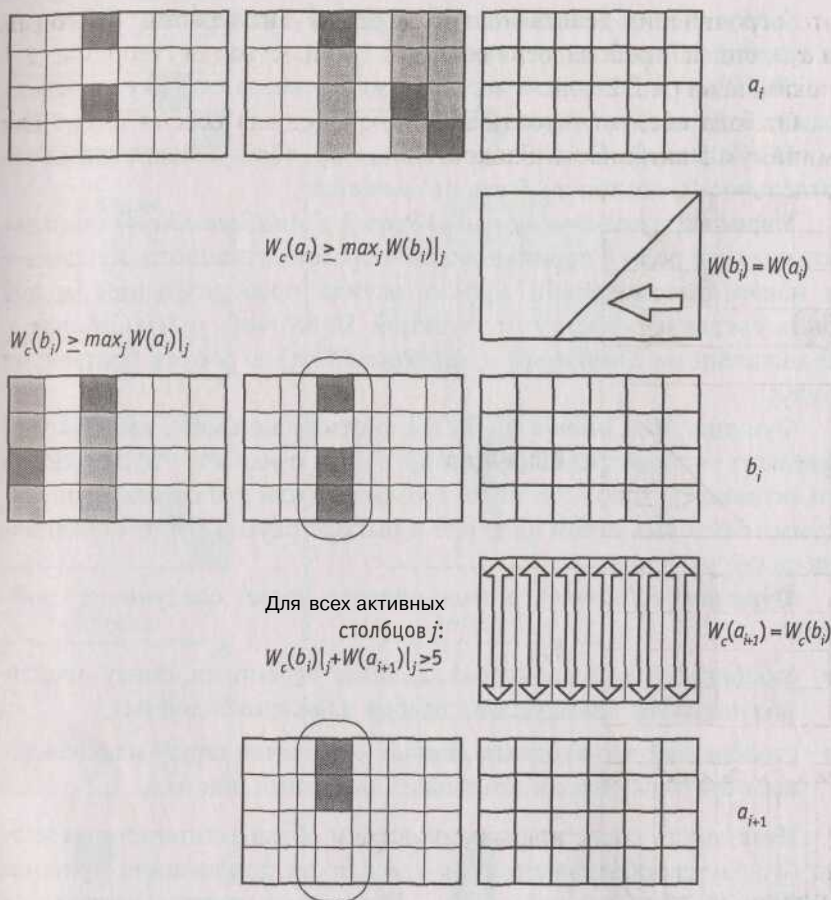


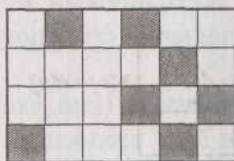
Рис. 2.13. Проникновение образов в преобразованиях одного раунда

Теорема 2.1. Общий байтовый вес двухраундовой сессии с Q активными столбцами на входе второго раунда ограничен снизу значением $5Q$.

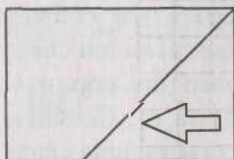
Доказательство. Тот факт, что функция *MixColumns()* имеет коэффициент распространения, равный 5, означает, что сумма байтовых весов в столбцах образов b_0 и a_1 для каждого столбца ограничена снизу значением 5. Поэтому, если столбцовый вес $a \setminus$ равен Q , то это дает ограничение снизу $5Q$ для суммы байтовых весов b_0 и $a \setminus$. Поскольку a_0 и b_0 имеют одинаковый байтовый вес,

это ограничение действительно и для суммы байтовых весов a_0 и a_1 , что и требовалось доказать. Иллюстрация теоремы 2.1 показана на рис. 2.14.

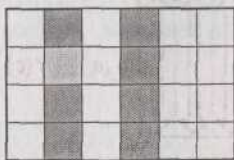
Отсюда следует, что любая двухраундовая сессия имеет как минимум 5 активных S-блоков.



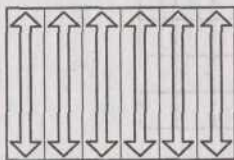
a_0



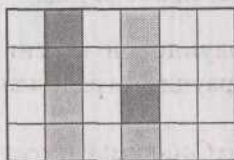
$W(b_0) = W(a_0)$



b_0



$W(a_1) + W(b_0) \geq 5W_c(a_1)$



a_1

Рис. 2.14. Иллюстрация теоремы 2.1 при $Q = 2$

Лемма 2.1. В двухраундовой сессии сумма активных столбцов во входных данных и активных столбцов в выходных данных не

меньше 5. Другими словами сумма столбцовых весов образов a_0 и a_2 не меньше 5.

Доказательство. Функция *ShiftRows()* сдвигает байты любого столбца из образа a_i в различные столбцы образа b_i и наоборот. Отсюда следует, что столбцовый вес образа a_i ограничен снизу байтовым весом отдельных столбцов b_i . Так же и столбцовый вес образа b_i ограничен снизу байтовым весом отдельных столбцов образа a_i .

В сессии активен как минимум один столбец образа a_1 (или, что тоже самое, образа b_0). Обозначим этот столбец как "столбец g ". Поскольку коэффициент распространения функции *MixColumns()* равен 5, то сумма байтовых весов столбца g в образе b_0 и в образе a_1 будет не менее 5. Но столбцовый вес образа a_0 ограничен снизу байтовым весом столбца g образа b_0 . А столбцовый вес образа b_1 ограничен снизу байтовым весом столбца g образа a_1 . Следовательно, сумма столбцовых весов образов a_0 и b_1 ограничена снизу значением 5. Поскольку столбцовый вес образа a_2 равен столбцовому весу образа b_1 , то лемму можно считать доказанной. Иллюстрация леммы 2.1 приведена на рис. 2.15.

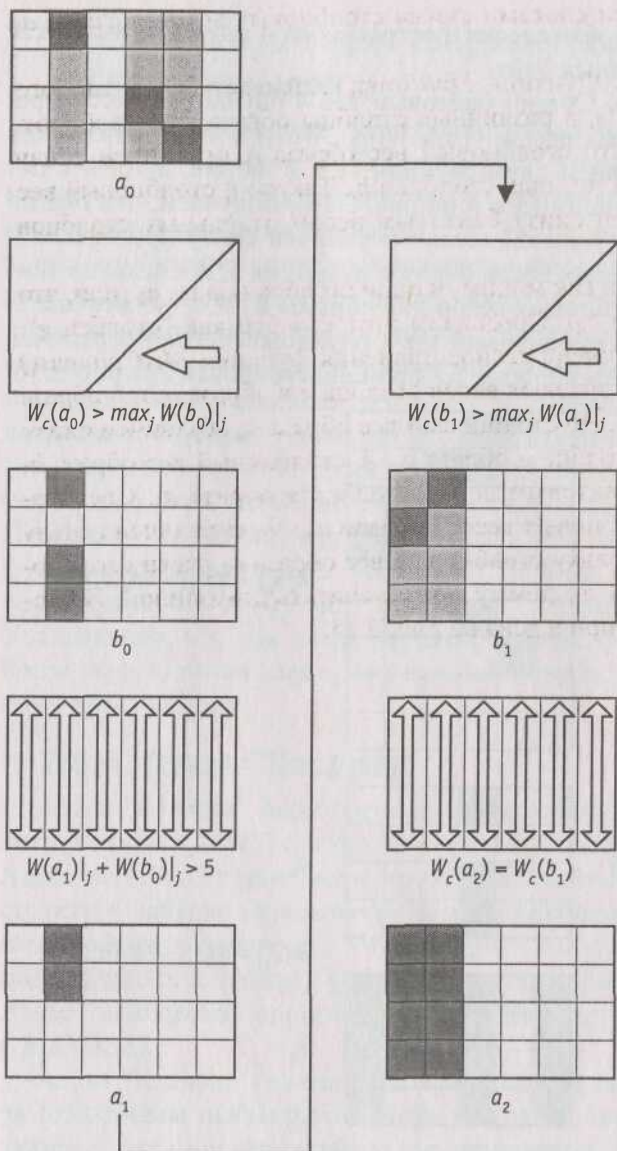


Рис. 2.15. Иллюстрация леммы 2.1 с одним активным столбцом в образе a_1

Теорема 2.2. Любая сессия, состоящая из 4 раундов, имеет как минимум 25 активных байт.

Доказательство. Общий байтовый вес 4-раундовой сессии равен сумме байтовых весов $W(a_0)$, $W(a_1)$, $W(a_2)$ и $W(a_3)$. Применив теорему 2.1 к первым двум раундам (1 и 2 на рис. 2.16) и к последним двум раундам (3 и 4 на рис. 2.16), получаем, что общий байтовый вес 4-раундовой сессии ограничен снизу произведением суммы столбцовых весов образов a_1 и a_3 на 5. А согласно лемме 2.1 сумма столбцовых весов для образов a_1 и a_3 не меньше 5. Отсюда следует, что байтовый вес 4-раундовой сессии ограничен снизу значением 25. Теорема 2.2 проиллюстрирована на рис. 2.16.

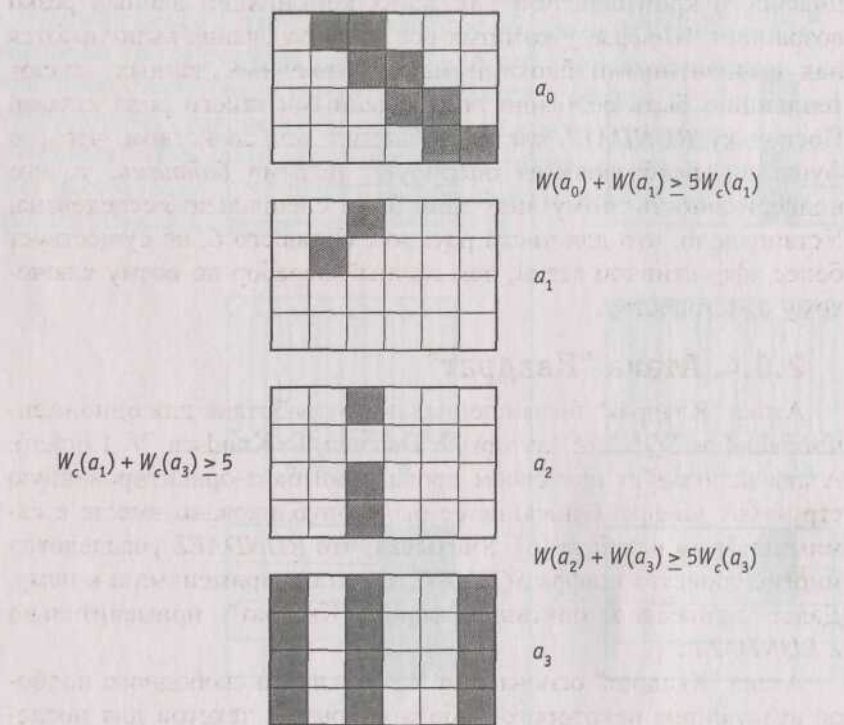


Рис. 2.16. Иллюстрация теоремы 2.2

2.6.3. Атака методом сокращенных дифференциалов

Концепция сокращенных дифференциалов была впервые опубликована в работе Л. Кнудсена [13]. Атаки этого класса основаны на том, что для некоторых шифров разностные сессии имеют тенденцию *кластеризации* [5]. Это означает, что для некоторых комбинаций линейных зависимостей во входных и выходных данных количество раундов, сквозь которые можно отследить значительное число таких зависимостей, становится слишком большим. Другими словами эффективность дифференциального криптоанализа для таких комбинаций данных резко возрастает. Шифры, у которых все преобразования выполняются над выровненными блоками массива входных данных, имеют тенденцию быть особенно подверженными такого рода атакам. Поскольку *RIJNDAEL* как раз обладает тем свойством, что его функции преобразования оперируют целыми *байтами*, то его подверженность этому виду атак была специально исследована. Установлено, что для числа раундов, большего 6, не существует более эффективной атаки, чем полный перебор по всему ключевому пространству.

2.6.4. Атака "Квадрат"

Атака "Квадрат" была специально разработана для одноименного шифра *SQUARE* (авторы J. Daemen, L. Knudsen, V. Rijmen). Атака использует при своем проведении байт-ориентированную структуру шифра. Описание ее было опубликовано вместе с самим шифром в работе [6]. Учитывая, что *RIJNDAEL* унаследовал многие свойства шифра *SQUARE*, эта атака применима и к нему. Далее приведено описание атаки "Квадрат" применительно к *RIJNDAEL*.

Атака "Квадрат" основана на возможности свободного подбора атакующим некоторого набора открытых текстов для последующего их зашифрования. Она независима от таблиц замен *S*-блоков, многочлена функции *MixColumns()* и способа разворачивания ключа. Эта атака для 6-раундового шифра *RIJNDAEL*, состоящего из 6 раундов, эффективнее, чем полный перебор по всему ключевому пространству. После описания базовой атаки на

4-раундовый *RIJNDAEL*, будет показано, как эту атаку можно продлить на 5 и даже 6 раундов. Но уже для 7 раундов "Квадрат" становится менее эффективным, чем полный перебор.

2.6.4.1. Предпосылки

Пусть Л-набор - такой набор из 256 входных блоков (массивов *State*), каждый из которых имеет байты (назовем их *активными*), значения которых различны для всех 256 блоков. Остальные байты (будем называть их *пассивными*) остаются одинаковыми для всех 256 блоков из Л-набора. То есть:

$$\forall x, y \in \Lambda : \begin{cases} x_{ij} \neq y_{ij}, & \text{если байт с номером } y \text{ активный;} \\ x_{ij} = y_{ij} & \text{в противном случае.} \end{cases}$$

Будучи подвергнутыми обработке функциями *SubBytes()* и *AddRoundKey()* блоки Л-набора дадут в результате другой Л-набор с активными байтами в тех же позициях, что и у исходного. Функция *ShiftRows()* сместит эти байты соответственно заданным в ней смещениям в строках массивов *State*. После функции *MixColumnsQ* Л-набор в общем случае необязательно останется Л-набором (т. е. результат преобразования может перестать удовлетворять определению Л-набора). Но поскольку каждый байт результата функции *MixColumnsQ* является линейной комбинацией (с обратимыми коэффициентами) четырех входных байт того же столбца

$$by = 2a_{ij} \oplus 3a_{(i+1)j} \oplus a_{(i+2)j} \oplus a_{(i+3)j},$$

столбец с единственным активным байтом на входе даст в результате на выходе столбец со всеми четырьмя байтами - активными.

2.6.4.2. Базовая атака "Квадрат" на 4 раунда

Рассмотрим Л-набор, во всех блоках которого активен только один байт. Иначе говоря, значение этого байта различно во всех 256 блоках, а остальные байты одинаковы (скажем, равны нулю). Проследим эволюцию этого байта на протяжении трех раундов. В первом раунде функция *MixColumnsQ* преобразует один активный байт в столбец из 4 активных байт. Во втором раунде эти 4 байта разойдутся по 4 различным столбцам в результате преоб-

разования функцией *ShiftRows()*. Функция же *MixColumnsQ* следующего, третьего раунда преобразует эти байты в 4 столбца, содержащие активные байты. Этот набор все еще остается Л-набором до того самого момента, когда он поступает на вход функции *MixColumnsQ* третьего раунда.

Основное свойство Л-набора, используемое здесь, то, что поразрядная сумма по модулю 2 всех блоков такого набора всегда равна нулю. Действительно, поразрядная сумма неактивных (с одинаковыми значениями) байт равна нулю по определению операции поразрядного XOR, а активные байты, пробегаая все 256 значений, также при поразрядном суммировании дадут нуль. Рассмотрим теперь результат преобразования функцией *MixColumnsQ* в третьем раунде байтов входного массива данных *a* в байты выходного массива данных *b*. Покажем, что и в этом случае поразрядная сумма всех блоков выходного набора будет равна нулю, то есть:

$$\begin{aligned} b = \text{MixColumns}(a), a \in \Lambda \quad b_{ij} &= \bigoplus_{a \in \Lambda} (2a_{ij} \oplus 3a_{(i+1)j} \oplus a_{(i+2)j} \oplus a_{(i+3)j}) = \\ &= 2 \bigoplus_{a \in \Lambda} a_{ij} \oplus 3 \bigoplus_{a \in \Lambda} a_{(i+1)j} \oplus \bigoplus_{a \in \Lambda} a_{(i+2)j} \oplus \bigoplus_{a \in \Lambda} a_{(i+3)j}. \end{aligned}$$

Таким образом, все данные на входе четвертого раунда сбалансированы (т. е. их полная сумма равна нулю). Этот баланс в общем случае нарушается последующим преобразованием данных функцией *SubBytesQ*.

Мы предполагаем далее, что четвертый раунд является последним, то есть в нем нет функции *MixColumnsQ*. Тогда каждый байт выходных данных этого раунда зависит только от одного байта входных данных. Если обозначить через *a* байт выходных данных четвертого раунда, через *b* байт входных данных и через *k* - соответствующий байт раундового ключа, то можно записать:

$$a_{ij} = \text{SubBytes}(b_{ij}) \oplus k_{ij}.$$

Отсюда, предполагая значение *kg*, можно по известному *ay* вычислить *b_{ij}*, а затем проверить правильность догадки о значении *k_{ij}*: если значения байта *by*, полученные при данном *k_{ij}* не будут сбалансированы по всем блокам (то есть не дадут при поразрядном суммировании нулевой результат), значит догадка неверна.

Перебрав максимум 2^8 вариантов байта раундового ключа, мы найдем его истинное значение.

По такому же принципу могут быть определены и другие байты раундового ключа. За счет того, что поиск может производиться отдельно (читай - параллельно) для каждого байта ключа, скорость подбора всего значения раундового ключа весьма велика. А по значению полного раундового ключа, при известном алгоритме его развертывания, не составляет труда восстановить сам начальный ключ шифрования.

2.6.4.3. Добавление пятого раунда в конец базовой атаки "Квадрат"

Если будет добавлен пятый раунд, то значения b_{ij} придется вычислять уже на основании выходных данных не четвертого, а пятого раунда. И дополнительно кроме байта раундового ключа четвертого раунда перебирать еще значения четырех байт столбца раундового ключа для пятого раунда. Только так мы сможем выйти на значения сбалансированных байт b_{ij} входных данных четвертого раунда.

Таким образом, теперь нам нужно перебрать 2^{40} значений - 2^{32} вариантов для 4 байт столбца раундового ключа пятого раунда и для каждого из них 2^8 вариантов для одного байта четвертого раунда. Эту процедуру нужно будет повторить для всех четырех столбцов пятого раунда. Поскольку при подборе "верного"² значения байта раундового ключа четвертого раунда количество "неверных"³ ключей уменьшается в 2^8 раз, то работая одновременно с пятью Л-наборами, можно с большой степенью вероятности правильно подобрать все 2^{40} бита. Теперь поиск может производиться отдельно (т. е. параллельно) для каждого столбца каждого из пяти Л-наборов, что опять же гораздо быстрее полного перебора всех возможных значений ключа.

² Кавычки здесь означают то, что *верным* это значение может быть названо лишь с некоторой (но довольно большой) степенью вероятности.

³ То же.

2.6.4.4. Добавление шестого раунда в начало базовой атаки "Квадрат"

Основная идея заключается в том, чтобы подобрать такой набор блоков открытого текста, который на выходе после первого раунда давал бы Л-набор с одним активным байтом. Это требует предположения о значении четырех байт ключа, используемых функцией *AddRoundKey()* перед первым раундом.

Для того чтобы на входе второго раунда был только один активный байт достаточно, чтобы в первом раунде один активный байт оставался на выходе функции *MixColumns()*. Это означает, что на входе *MixColumns()* первого раунда должен быть такой столбец, байты a которого для набора из 256 блоков в результате линейного преобразования

$$b_i = 2a_i \oplus 3a_{i+1} \oplus a_{i+2} \oplus a_{i+3}, \quad 0 \leq i < 3,$$

где i - номер строки, для одного определенного i давали 256 различных значений, в то время как для каждого из остальных трех значений i результат этого преобразования должен оставаться постоянным. Следуя обратно по порядку приложения функций преобразования в первом раунде, к *ShiftRows()* данное условие нужно применить к соответственно разнесенным по столбцам 4 байтам. С учетом применения функции *SubBytes()* и сложения с предполагаемым значением 4-байтового раундового ключа можно смело составлять уравнения и подбирать нужные значения байт открытого текста, подаваемых на зашифрование для последующего анализа результата:

$$b_y = 2\text{SubBytes}(a_{ij} \oplus k_{ij}) \oplus 3\text{SubBytes}(a_{(i+1)(j+1)} \oplus k_{(i+1)(j+1)}) \oplus \\ \oplus \text{SubBytes}(a_{(i+2)(j+2)} \oplus k_{(i+2)(j+2)}) \oplus \text{SubBytes}(a_{(i+3)(j+3)} \oplus k_{(i+3)(j+3)}), \\ 0 \leq i, j \leq 3.$$

Таким образом, получаем следующий алгоритм взлома. Имеем всего 2^2 различных значений a для определенных i и j . Остальные байты для всех блоков одинаковы (пассивные байты). Предположив значения четырех байт k ключа первого раунда, подбираем (исходя из вышеописанного условия) набор из 256 блоков. Эти 256 блоков станут Л-набором после первого раунда. К этому Л-набору применима базовая атака для 4 раундов. Подобранный с ее помощью один байт ключа последнего раунда фиксируется.

Теперь подбирается новый набор из 256 блоков для того же значения 4 байт k ключа первого раунда. Опять осуществляется базовая атака, дающая один байт ключа последнего раунда. Если после нескольких попыток значение этого байта не меняется, значит мы на верном пути. В противном случае нужно менять предположение о значении 4 байт k ключа первого раунда. Такой алгоритм действий достаточно быстро приведет к полному восстановлению всех байт ключа последнего раунда.

2.6.4.5. Эффективность и требования к памяти

Таким образом атака "Квадрат" может быть применена к 6 раундам шифра *RIJNDAEL*, являясь при этом более эффективной, чем полный перебор по всему ключевому пространству. Трудоемкость и требования к памяти для атаки "Квадрат" обобщены в табл. 2.14. Любое известное продолжение атаки "Квадрат" на 7 и более раундов становится более трудоемким, чем даже обычный полный перебор значений ключа.

Таблица 2.14. Ресурсы, необходимые для проведения атаки "Квадрат"

Тип атаки	Необходимое количество блоков открытых данных	Число повторений процедуры зашифрования	Требования к памяти
Базовая 4-раундовая атака	29	29	Небольшие
Расширенная, с добавлением раунда в конце	211	240	Небольшие
Расширенная, с добавлением раунда в начале	232	240	232
Расширенная, с добавлением раундов в начале и в конце	232	272	232

2.6.5. Атака методом интерполяций

В работе [10] Т. Якобсен и Л. Кнудсен представили новый вид атаки на блочные шифры. Суть ее состоит в том, что атакующий пытается получить многочлен из известных ему пар входных/выходных данных. Это становится возможным лишь в тех случаях, когда функции, входящие в состав преобразования, могут быть достаточно компактно выражены алгебраически, а все преобразование - представлено в виде алгебраического выражения разрешимой сложности. Атака основывается на следующем

факте: если полученный многочлен (или рациональное выражение) имеет небольшую степень, то достаточно нескольких пар входных/выходных данных для того, чтобы подобрать коэффициенты многочлена, значения которых прямо связаны со значением ключа. Сложность алгебраического представления подстановки *SubBytes()* в поле $GF(2^8)$ в сочетании с рассеиванием функцией *MixColumns()* делает невозможным применение этого типа атак к многораундовому шифру *RIJNDAEL*. Вот, к примеру, алгебраическое выражение для функции *SubBytesQ*:

$$\begin{aligned} &\{63\} + \{8f\}x^{127} + \{b5\}x^{191} + \{01\}x^{223} + \{f4\}x^{239} + \\ &+ \{25\}x^{247} + \{f9\}x^{251} + \{09\}x^{253} + \{05\}x^{254} \end{aligned}$$

2.6.6. О существовании слабых ключей

Слабыми считаются ключи, использование которых ведет к плохому преобразованию входных данных (недостаточное рассеивание или перемешивание). Наиболее известный случай существования слабых ключей для шифра *IDEA* описан в [5]. Как правило, такой недостаток свойственен шифрам, в которых имеются нелинейные операции, зависящие от значения ключа. Это не так в случае *RIJNDAEL* - добавление ключа в процедуру шифрования осуществляется с использованием операции *XOR*, а нелинейное преобразования реализуют 5-блоки с фиксированными таблицами замен. Таким образом, в *RIJNDAEL* отсутствуют ограничения на выбор ключей.

2.6.7. Атака "эквивалентных ключей"

В [3] Э. Бихэм описал атаку "эквивалентных ключей". Позже Д. Келси, Б. Шнейер и Д. Вагнер показали в работе [12], что некоторые шифры являются недостаточно стойкими к этой атаке.

В атаке "эквивалентных ключей" криптоаналитик выполняет шифрование, используя различные (полностью или частично неизвестные) ключи, которые имеют заданные взаимозависимости. Учитывая высокую степень рассеивания при разворачивании ключа, возможность этой атаки на *RIJNDAEL* маловероятна.

2.7. Заключение

Исследования, проведенные различными сторонами, показали высокое быстродействие *RIJNDAEL* на различных платформах. Ценным свойством этого шифра является его байт-ориентированная структура, что обещает хорошие перспективы при его реализации в будущих процессорах и специальных схемах. Некоторым недостатком можно считать то, что режим обратного расшифрования отличается от режима зашифрования не только порядком следования функций, но и сами эти функции отличаются своими параметрами от применяемых в режиме зашифрования. Данный факт сказывается на эффективности аппаратной реализации шифра. Этому недостатка лишены шифры Фейстеля (*DES*, ГОСТ 28147-89), в которых при расшифровании меняется лишь порядок использования раундовых ключей. Однако *RIJNDAEL* имеет более цельную структуру, за один раунд в нем преобразуются все биты входного блока, в отличие от шифров Фейстеля, где за один раунд изменяется, как правило, лишь половина бит входного блока. По этой причине *RIJNDAEL* может позволить себе меньшее число раундов преобразования, что можно рассматривать как некоторую компенсацию за потери при реализации режима обратного шифрования. К тому же благодаря некоторым описанным выше манипуляциям можно все же (хотя и не полностью) оптимизировать алгоритм расшифрования так, что в режиме прямого расшифрования становится возможным разделение части аппаратных ресурсов с зашифрованием. Шифр полностью самодостаточен в том смысле, что он не использует никаких частей, заимствованных у других (пусть даже и хорошо зарекомендовавших себя) шифров, имеет четкую и ясную структуру, т. е. его стойкость не основана на каких-то сложных и не вполне понятных преобразованиях. Последнее свойство также уменьшает вероятность существования каких-либо "потайных ходов". К тому же гибкость, заложенная в архитектуре *RIJNDAEL*, позволяет варьировать не только длину ключа, что используется в стандарте *AES*, но и размер блока преобразуемых данных, что хотя и не нашло применения в стандарте, авторами рассматривается как вполне нормальное расширение этого шифра.

Литература к главе 2

1. **Киви Берд.** Конкурс на новый криптостандарт *AES*. - Системы безопасности связи и телекоммуникаций, 1999, № 27-28.
2. **Винокуров А.** Страничка классических блочных шифров.
<http://www.enlight.ru/crypto/>
3. **Biham E.** New types of cryptanalytic attacks using related keys. *Advances in Cryptology, Proceedings Eurocrypt'93*, LNCS 765, T. Helleseeth, Ed., Springer-Verlag, 1993, pp. 398-409.
4. **Biham E., Shamir A.** Differential cryptanalysis of DES-like cryptosystems. *Journal of Cryptology*, Vol. 4, No. 1, 1991, pp. 3-72.
5. **Daemen J.** Cipher and hash function design strategies based on linear and differential cryptanalysis. Doctoral Dissertation, March 1995, K.U.Leuven.
6. **Daemen J., Knudsen L., Rijmen V.** The block cipher Square. *Fast Software Encryption*, LNCS 1267, E. Biham, Ed., Springer-Verlag, 1997, pp. 149-165. Also available at <http://www.esat.kuleuven.ac.be/rijmen/square/fse.ps.gz>.
7. **Daemen J., Clapp C.** Fast hashing and stream Encryption with PANAMA. *Fast Software Encryption*, LNCS 1372, S. Vaudenay, Ed., Springer-Verlag, 1998, pp. 60-74.
8. **Daemen J., Rijmen V.** AES Proposal: Rijndael.
<http://csrc.nist.gov/encryption/aes/>
9. **Federal Information Processing Standards Publication.** Announcing the Advanced Encryption Standard (AES). <http://csrc.nist.gov/encryption/aes/index.html>
10. **Jakobsen T., Knudsen L.** The interpolation attack on block ciphers. *Fast Software Encryption*, LNCS 1267, E. Biham, Ed., Springer-Verlag, 1997, pp. 28-40.
11. **Jelly Andrey.** Криптографический стандарт в новом тысячелетии. - BYTE, Россия, 6.06.1999.
12. **Kelsey J., B. Schneier B., Wagner D.** Key-schedule cryptanalysis of IDEA, GDES, GOST, SAFER, and Triple-DES. *Advances in Cryptology, Proceedings Crypto '96*, LNCS 1109, N. Koblitz, Ed., Springer-Verlag, 1996, pp. 237-252.
13. **Knudsen L.** Truncated and higher order differentials. *Fast Software Encryption*, LNCS 1008, B. Preneel, Ed., Springer-Verlag, 1995, pp. 196-211.
14. **Knudsen L.** A key-schedule weakness in SAFER-K64. *Advances in Cryptology, Proceedings Crypto'95*, LNCS 963, D. Coppersmith, Ed., Springer-Verlag, 1995, pp. 274-286.
15. **Lai X., Massey J.L., Murphy S.** Markov ciphers and differential cryptanalysis. *Advances in Cryptology, Proceedings Eurocrypt'91*, LNCS 547, D.W. Davies, Ed., Springer-Verlag, 1991, pp. 17-38.
16. **Matsui M.** Linear cryptanalysis method for DES cipher. *Advances in Cryptology, Proceedings Eurocrypt'93*, LNCS 765, T. Helleseeth, Ed., Springer-Verlag, 1994, pp. 386-397.
17. **National Information Systems Security Conference materials.**
<http://csrc.nist.gov/encryption/aes/round2/conf3/aes3conf.htm>
18. **Nyberg K.** Differentially uniform mappings for cryptography. *Advances in Cryptology, Proceedings Eurocrypt'93*, LNCS 765, T. Helleseeth, Ed., Springer-Verlag, 1994, pp. 55-64.
19. **Overview** of the AES Development Effort.
<http://csrc.nist.gov/encryption/aes/index2.html#overview>

ГЛАВА 3

Конечные поля

3.1. Введение

Широкое распространение в системах защиты получили *регистры сдвига с линейными обратными связями* - *LFSR (Linear Feedback Shift Register)* [1, 4–7]. Эти устройства являются идеальным средством защиты от *случайных деструктивных воздействий*. Основными достоинствами *LFSR* являются:

- * простота аппаратной и программной реализации;
- * высокое быстродействие;
- * хорошие статистические свойства формируемых последовательностей в режиме генерации псевдослучайных последовательностей (ПСП);
- * возможность построения на их основе генераторов, обладающих свойствами, ценными при решении специфических задач защиты информации (формирование последовательностей произвольной длины, формирование последовательностей с *предпериодом*, формирование ПСП с произвольным законом распределения, построение генераторов, обладающих свойством самоконтроля и т. п.).

Генераторы ПСП на основе *LFSR*, к сожалению, не являются криптостойкими, что исключает возможность их использования для защиты от *умышленных деструктивных воздействий*. Они применяются при решении таких задач лишь в качестве строительных блоков.

Используемый при анализе схем на *LFSR* математический аппарат - *теория линейных последовательностных машин* и *теория конечных полей* (полей Галуа) [1-7].

Наиболее известные примеры использования *LFSR* и математического аппарата полей Галуа:

- * разработка и исследование свойств кодов, обнаруживающих и исправляющих ошибки [6];

- построение *CRC*-генераторов и исследование свойств *CRC*-кодов, которые являются всеми признанным идеальным средством контроля целостности информации при случайных искажениях информации [4];
- разработка и реализация поточных криптоалгоритмов, наиболее известные примеры - шифры *A5*, *SOBER*, *PANAMA*, *SNOW* и др.;
- в разработке и реализация блочных криптоалгоритмов, наиболее известный пример - блочный шифр *RIJNDAEL*, принятый в 2000 г. в качестве стандарта криптографической защиты XX1 века - *AES*;
- построении криптографических протоколов, наиболее известный пример - протокол выработки общего секретного ключа Диффи-Хеллмана;
- » построение криптосистем с открытым ключом, например криптосистема, основанной на свойствах эллиптических кривых - *ECCS*.

3.2. Регистры сдвига с линейными обратными связями

В состав двоичного *LFSR* входят *D*-триггеры, выполняющие функцию задержки на один такт и сумматоры по модулю два в цепи обратной связи. Исходная информация для построения *LFSR* - так называемый *образующий многочлен*. Степень этого многочлена определяет разрядность регистра сдвига, а ненулевые коэффициенты - характер обратных связей. Так например, многочлену

$$\Phi(x) = x^8 + x^7 + x^5 + x^3 + 1$$

соответствуют два устройства, показанные на рис. 3.1 и 3.2. В общем случае двоичному образующему многочлену $\Phi(x)$ степени N

$$\Phi(x) = \sum_{i=0}^N a_i x^i, \quad a_N = a_0 = 1, \quad a_j \in \{0, 1\}, \quad j = 1, (N-1)$$

соответствуют устройства, показанные на рис. 3.3, а (*LFSR1* - схема Фибоначчи) и б (*LFSR2* - схема Галуа).

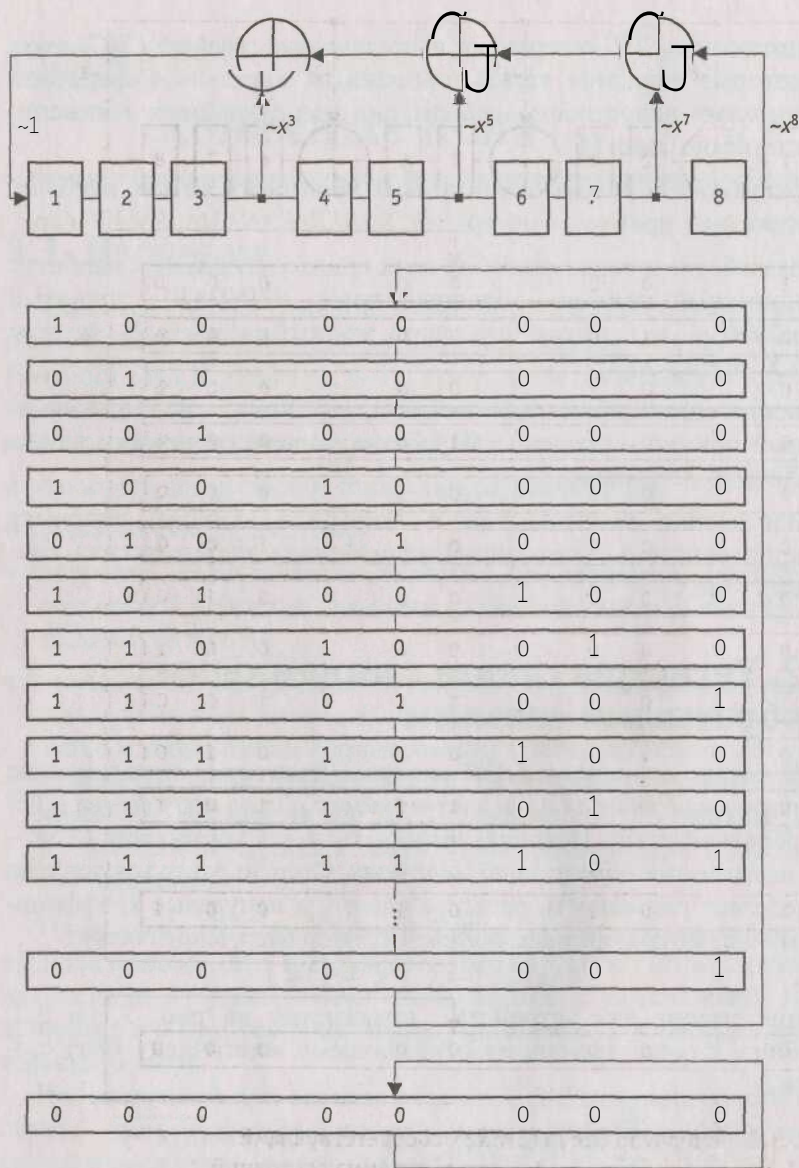


Рис. 3.1. Генератор Фибоначчи (*LFSR*), соответствующий $\Phi(x) = x^8 + x^7 + x^5 + x^3 + 1$, и его диаграмма состояний

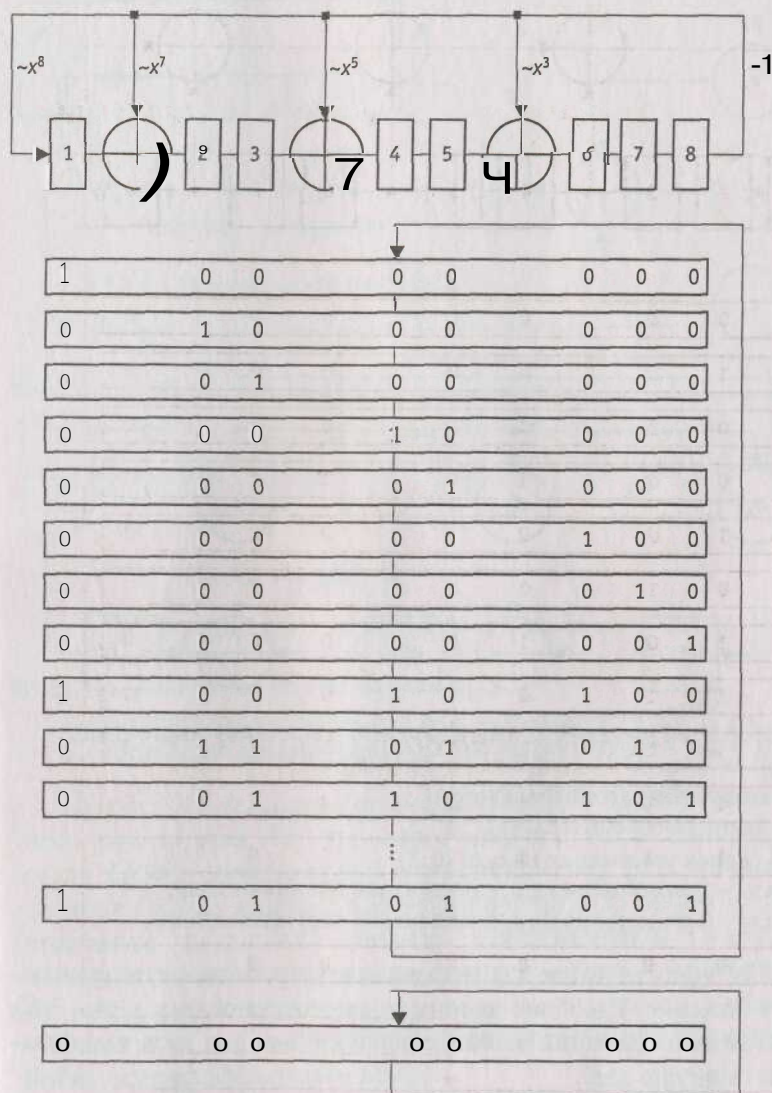


Рис. 3.2. Генератор Галуа (LFSR2), соответствующий $\Phi(x) = x^8 + x^7 + x^5 + x^3 + 1$, и его диаграмма состояний

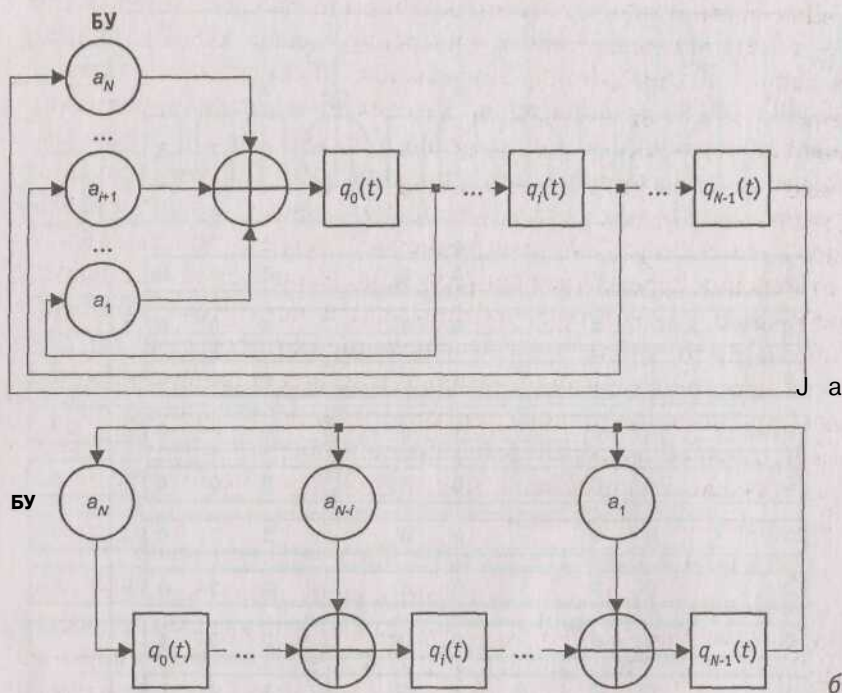


Рис. 3.3. Общий вид *LFSR*, соответствующих двоичному многочлену $\Phi(x) = x^N + a_{N-1}x^{N-1} + \dots + a_1x + 1$:

а - схема генератора Фибоначчи;

б - схема генератора Галуа.

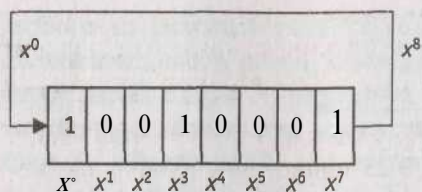
БУ - блоки умножения на $a_j \in \{0, 1\}$;

при $a_j = 1$ умножение на a_j равносильно наличию связи,

при $a_j = 0$ умножение на a_j равносильно отсутствию связи

LFSR2 используются для построения устройств, осуществляющих умножение и деление двоичных многочленов (рис. 3.4 - 3.6). При этом все операции приведения подобных членов выполняются по модулю два.

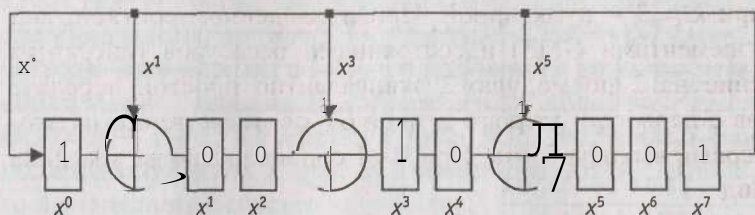
На рис. 3.4 показаны примеры схем, один такт работы которых суть умножение многочлена $A(x)$ степени не более 7 на x по модулю $\varphi(x)$ степени 8. При этом обратные связи *LFSR2* определяются коэффициентами $\varphi(x)$, а исходное состояние *LFSR2* - коэффициентами $A(x)$.



$$10010001 \rightarrow 11001000$$

$$(x^7 + x^3 + 1) \cdot x = (x^4 + x + 1) \bmod (x^8 + 1)$$

a



$$10010001 \rightarrow 10011100$$

$$(x^7 + x^3 + 1) \cdot x = (x^5 + x^4 + x^3 + 1) \bmod (x^8 + x^5 + x^3 + x + 1)$$

b

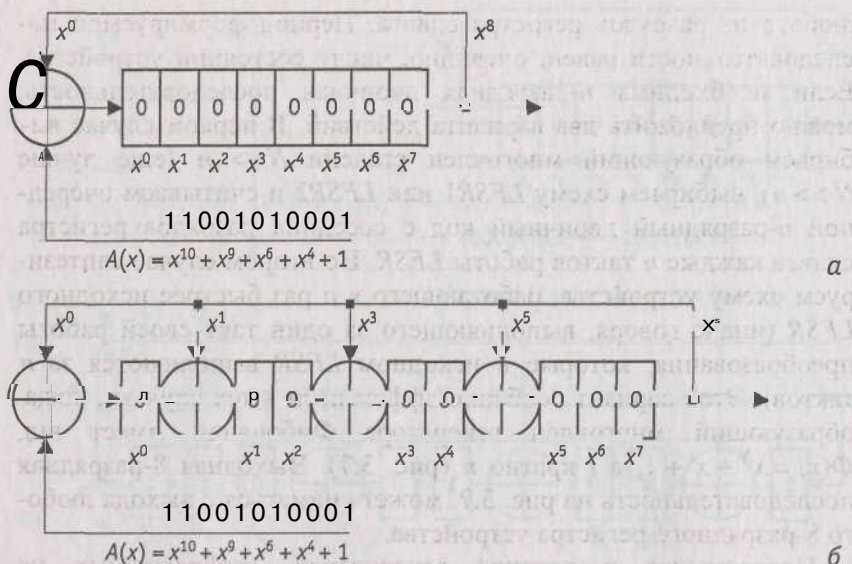
Рис. 3.4. Умножение на x по модулю $\phi(x)$:

a - устройство для умножения на x по модулю $\phi(x) = x^8 + 1$,

b - устройство для умножения на x по модулю $\phi(x) = x^8 + x^5 + x^3 + x + 1$

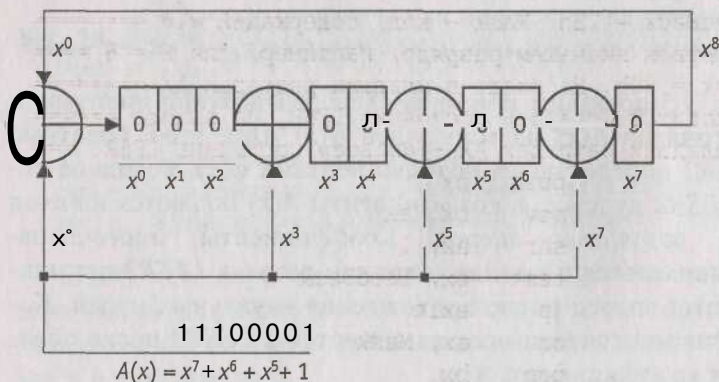
На рис. 3.5 показаны примеры схем для деления произвольного многочлена $A(x)$ на многочлен $\phi(x)$. При этом обратные связи $LFSR2$ определяются коэффициентами $\phi(x)$, исходное состояние $LFSR2$ нулевое, а коэффициенты $A(x)$ подаются на вход старшими разрядами вперед. Коэффициенты многочлена-частного снимаются с выхода старшего разряда $LFSR2$ старшими разрядами вперед после прохождения начальных нулей. Коэффициенты многочлена-остатка остаются в $LFSR2$ после обработки всех коэффициентов $A(x)$.

На рис. 3.6 показан пример схемы для умножения многочлена $A(x)$ на многочлен $g(x)$ по модулю многочлена $\phi(x)$. При этом коэффициенты $A(x)$ подаются на вход старшими разрядами вперед, обратные связи $LFSR2$ определяются коэффициентами $\phi(x)$, а точки ввода входной последовательности - коэффициентами $g(x)$.

Рис. 3.5. Деление на $\phi(x)$:

(г - устройство для деления на $\phi(x) = x^8 + 1$,

б - устройство для деления на $\phi(x) = x^8 + x^5 + x^3 + x + 1$

Рис. 3.6. Устройство для умножения на $d(x) = x^7 + x^5 + x^3 + 1$ по модулю $\phi(x) = x^8 + 1$

LFSR могут использоваться для генерации двоичной последовательности, которая в этом случае может сниматься с выхода

любого из разрядов регистра сдвига. Период формируемой последовательности равен, очевидно, числу состояний устройства. Если необходима n -разрядная двоичная последовательность, можно предложить два варианта действий. В первом случае выбираем образующий многочлен степени $N > n$ (еще лучше $N \gg n$), выбираем схему *LFSR1* или *LFSR2* и считываем очередной n -разрядный двоичный код с соседних разрядов регистра сдвига каждые n тактов работы *LFSR*. Во втором случае синтезируем схему устройства, работающего в n раз быстрее исходного *LFSR* (иначе говоря, выполняющего за один такт своей работы преобразования, которые в исходном *LFSR* выполняются за n тактов). Этот вариант особенно эффективен в тех случаях, когда образующий многочлен генератора Фибоначчи имеет вид $\Phi(x) = x^N + x^i + 1$, а i кратно n (рис. 3.7). Выходная 8-разрядная последовательность на рис. 3.7. может сниматься с выхода любого 8-разрядного регистра устройства.

Программная реализация генераторов, изображенных на рис. 3.3, при $N < 16$ имеет вид

```
; ===== Генератор Фибоначчи. =====
; ===== FeedBack - вектор обратных связей, например, =====
; ===== для  $\Phi(x) = x^8 + x^7 + x^5 + x^3 + 1$  =====
; ===== FeedBack = 2Bh. Mask - код, содержащий «1» =====
; ===== в первом значащем разряде, например, для  $N = 8$  =====
; ===== Mask = 80h. На входе в младших разрядах AX =====
; ===== находится «старое» состояние LFSR, на выходе =====
; ===== в младших разрядах AX - «новое» состояние LFSR. =====
lfsr1proc:      push    bx
                raov    bx,    ax
                shr     ax,    1
                test    bx,    FeedBack
                jp      exit
                or      ax,    Mask
exit:          pop     bx
                ret

; ===== Генератор Галуа. =====
; ===== FeedBack - вектор обратных связей, =====
; ===== например, для  $\Phi(x) = x^8 + x^7 + x^5 + x^3 + 1$  =====
; ===== FeedBack = 0D4h. На входе в младших разрядах AX =====
; ===== находится «старое» состояние LFSR, на выходе =====
```



```

; ===== в младших разрядах AX - «новое» состояние LFSR. ===
lfsr2proc:      shr     ax, 1
                jnc     exit
                xor     ax, FeedBack
exit:           ret
; =====

```

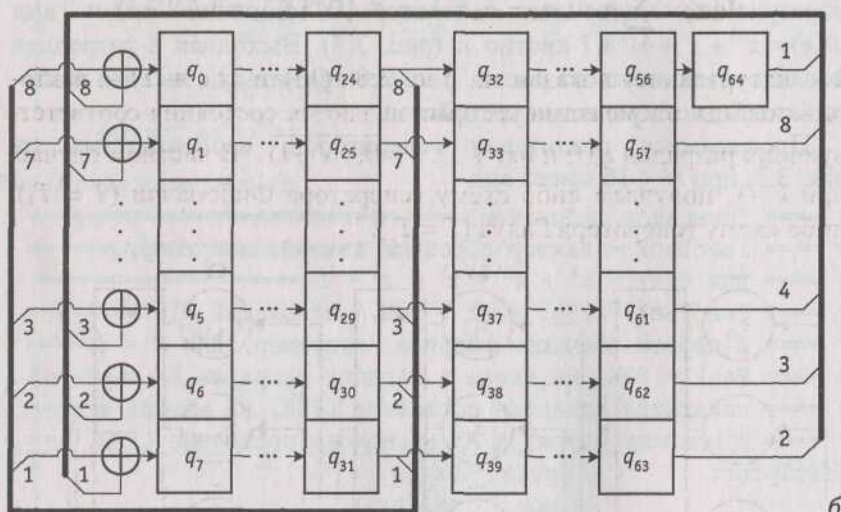
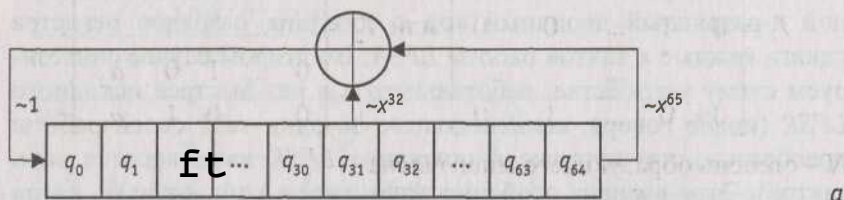


Рис. 3.7. Байтовый генератор ПСП:

а - битовый генератор Фибоначчи, соответствующий многочлену $\Phi(x) = x^{65} + x^{32} + 1$;

б - байтовый генератор Фибоначчи, соответствующий $\Phi(x) = x^{65} + x^{32} + 1$;

q_i - состояние i -го разряда $LFSR1$, $i = 0, 64$.

Общий вид генератора двоичных последовательностей, соответствующего уравнению

$$Q(i+1) = T^k Q(i),$$

где $Q(t)$ и $Q(t+1)$ - состояния регистра генератора ПСП соответственно в моменты времени t и $t+1$ (до и после прихода синхроимпульса); T - квадратная матрица порядка N вида

$$T_1 = \begin{bmatrix} a_1 & \ll 2 & \dots & a_{N-1} & a_N \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ & & \dots & & \\ 0 & 0 & \dots & 1 & 0 \end{bmatrix} \text{ или } T_2 = \begin{bmatrix} 0 & \dots & 0 & 0 & a_N \\ 1 & \dots & 0 & 0 & a_{N-1} \\ \dots & & & & \\ 0 & \dots & 1 & 0 & a_2 \\ 0 & \dots & 0 & 1 & a_1 \end{bmatrix},$$

N - степень образующего многочлена

$$\Phi(x) = \sum_{i=0}^N a_i x^i, \quad a_N = a_0 = 1, \quad a_j \in \{0, 1\}, \quad j = \overline{1, (N-1)},$$

$\mathbb{F}$ - натуральное, показан на рис. 3.8. $Q(t)$ и $Q(t+1)$ - векторы-столбцы, элементами которых являются состояния соответствующих разрядов $q_i(t)$ и $q_i(t+1)$, $i = \overline{0, (N-1)}$. В частном случае, при $k=1$, получаем либо схему генератора Фибоначчи ($T = T_1$), либо схему генератора Галуа ($T = T_2$).

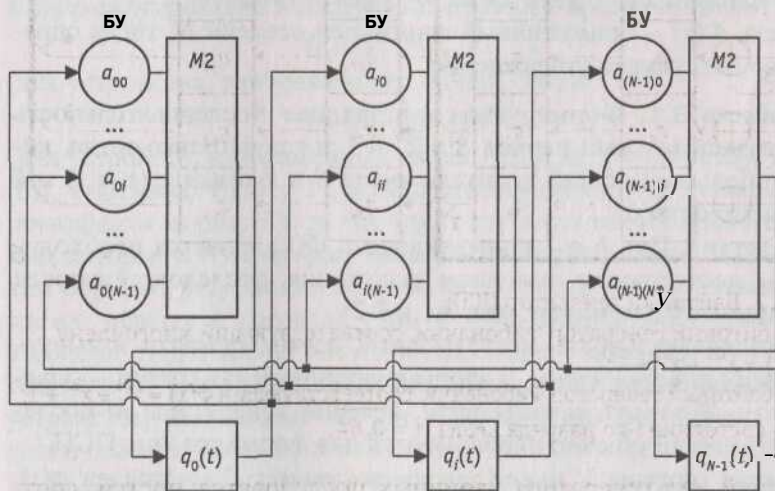


Рис. 3.8. Генератор двоичных последовательностей, соответствующий уравнению $Q(t+1) = I * Q(t)$

Величина, на которую происходит умножение в каждом блоке умножения (БУ), определяется соответствующим коэффициентом $a_{ij} \in \{0,1\}$ ($i = 0, (N-1), j = 0, (N-1)$) *сопровождающей матрицы*

$$V = T^k.$$

Если $a_{ij} = 0$, это эквивалентно отсутствию связи между выходом i -го разряда регистра генератора и входом j -го сумматора по модулю два. Если $a_{ij} = 1$, это эквивалентно наличию связи между выходом i -го разряда регистра генератора и входом j -го сумматора по модулю два.

Так как нулевое состояние регистра ГПК является запрещенным, максимально возможное число состояний устройства, а значит, и максимально возможная длина формируемой двоичной последовательности, снимаемой с выхода любого из триггеров, равны $2^N - 1$. В этом случае диаграмма состояний генератора состоит из одного тривиального цикла и цикла максимальной длины $2^N - 1$.

Многочлен $\Phi(x)$ степени N называется *примитивным*, если он не делит нацело ни один многочлен вида $x^S - 1$, где $S < 2^N - 1$. Примитивные многочлены существуют для любого N . *Показателем* многочлена $\Phi(x)$ называется наименьшее натуральное число e , при котором $x^e - 1$ делится на $\Phi(x)$ без остатка.

Пусть $\Phi(x)$ - примитивный многочлен степени N , тогда справедливо следующее утверждение.

Свойство 3.1. Формируемая k -разрядная последовательность имеет максимальный период $5 = 2^k - 1$ тогда и только тогда, когда наибольший общий делитель чисел S и k равен 1 (т. е. 5 и k взаимно просты).

Следствие. При $k = 1$ примитивность $\Phi(x)$ является необходимым и достаточным условием получения последовательности максимальной длины (см. рис. 3.1 и 3.2).

Последовательность максимальной длины принято называть *М-последовательностью*, а формирующий ее генератор - *генератором М-последовательности*. Именно генераторы *М-последовательностей* обычно используются для формирования ПСП.

Каждая матрица V имеет *характеристический* многочлен $\phi(x)$, которым является определитель матрицы $V - xE$, т. е. $\phi(x) = |V - xE|$, где E - единичная матрица. Многочлен $\Phi(x)$ определяет только структуру генератора, свойства же последнего зависят именно от $\phi(x)$.

Свойство 3.2. Каждая квадратная матрица удовлетворяет своему характеристическому уравнению, т. е. $\Phi(V) = 0$.

Свойство 3.3. Характеристический и образующий многочлен генератора связаны следующим соотношением

$$\Phi(x) = \Phi(x^{-1}Y,$$

т. е. являются *взаимно обратными*.

Примитивность $\Phi(x)$ автоматически означает примитивность $\Phi(x)$ и наоборот.

Децимацией последовательности $\{q_i(t)\}$ по индексу k называется формирование новой последовательности $\{q_i^*(t)\}$, состоящей из k -х элементов $\{q_i(t)\}$ т. е. $q_i^*(t) = q_i(kt)$. Если период последовательности, полученной в результате децимации M -последовательности, равен максимальному, децимация называется *собственной* или *нормальной*.

Последовательность, снимаемая с выхода z' -го разряда генератора, изображенного на рис. 3.8, является децимацией по индексу k последовательности, снимаемой с выхода i -го разряда генераторов, изображенных на рис. 3.3. Если Q - начальное состояние генератора, то последовательности состояний, в которых будут находиться устройства в следующие моменты времени, имеют вид

$$Q, QV, QV^2, QV^3, \dots$$

(для устройства, изображенного на рис. 3.8) и

$$Q, QT, QT^2, QT^3, \dots$$

(для устройств, изображенных на рис. 3.3, где $T - T \setminus (a)$ или $T = T_2(b)$). Учитывая, что $V = T^k$, можно сделать вывод, что в генераторе, показанном на рис. 3.8, за один такт осуществляются преобразования, которые в генераторах, показанных на рис. 3.3 за k тактов. Таким образом, устройство, показанное на рис. 3.8, в котором содержимое первых k разрядов (при $k < M$) полностью обновляется в каждом такте, может использоваться для генерации последовательности k -разрядных двоичных кодов, что нельзя сказать про устройства, показанные на рис. 3.3, которые формируют лишь сдвинутые копии одной и той же двоичной последовательности.

Пример 3.1. Пусть

$$N=4, \Phi(x)=x^4+x^3+1, T=T_1, k=1.$$

Этой ситуации соответствует генератор, схема и диаграмма состояний которого показаны на рис. 3.9. Если начальное состоя-

ние устройства равно 1000, на выходе первого триггера q_0 формируется периодическая последовательность

1 0 0 1 1 0 1 0 1 1 1 1 0 0 0 ...,

удовлетворяющая рекуррентному соотношению

$$q_0(t+4) = q_0(t+1) \oplus q_0(t).$$

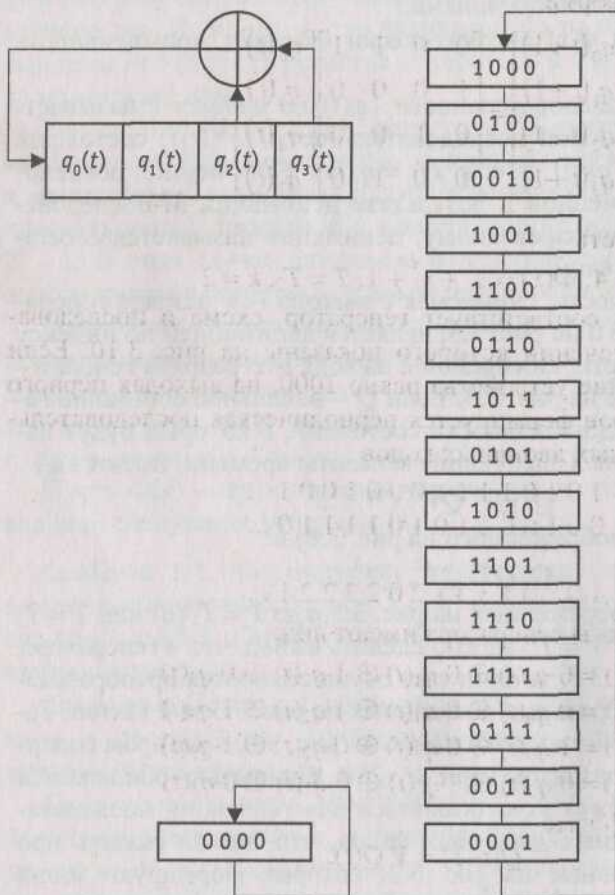


Рис. 3.9. Генератор двоичной M -последовательности, соответствующий $\Phi(x) = x^4 + x^3 + 1$, $T = T_1$, $k = 1$; иегодиаграмма состояний

С выходов триггеров q_1 , q_2 и q_3 снимаются сдвинутые копии той же последовательности.

Уравнения работы генератора имеют вид:

$$q_0(t+1) = 0 \cdot q_0(t) \oplus 0 \cdot q_1(t) \oplus 1 \cdot q_2(t) \oplus 1 \cdot q_3(t)$$

$$q_1(t+1) = 1 \cdot q_0(t) \oplus 0 \cdot q_1(t) \oplus 0 \cdot q_2(t) \oplus 0 \cdot q_3(t)$$

$$q_2(t+1) = 0 \cdot q_0(t) \oplus 1 \cdot q_1(t) \oplus 0 \cdot q_2(t) \oplus 0 \cdot q_3(t)$$

$$q_3(t+1) = 0 \cdot q_0(t) \oplus 0 \cdot q_1(t) \oplus 1 \cdot q_2(t) \oplus 0 \cdot q_3(t)$$

или в матричной форме

$$\begin{bmatrix} q_0(t+1) \\ q_1(t+1) \\ q_2(t+1) \\ q_3(t+1) \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} q_0(t) \\ q_1(t) \\ q_2(t) \\ q_3(t) \end{bmatrix}.$$

Пример 3.2. Пусть

$$N = 4, \Phi(x) = x^4 + x^3 + 1, T = T_2, k = 2.$$

Этой ситуации соответствует генератор, схема и последовательность переключений которого показаны на рис. 3.10. Если начальное состояние устройства равно 1000, на выходах первого и второго триггеров формируется периодическая последовательность двухразрядных двоичных кодов

$$\begin{array}{l} 101011110001001... \\ 001001101011110... \end{array}$$

или

$$103013312023221...$$

Уравнения работы генератора имеют вид:

$$q_0(t+1) = 0 \cdot q_0(t) \oplus 0 \cdot q_1(t) \oplus 1 \cdot q_2(t) \oplus 0 \cdot q_3(t)$$

$$q_1(t+1) = 0 \cdot q_0(t) \oplus 0 \cdot q_1(t) \oplus 1 \cdot q_2(t) \oplus 1 \cdot q_3(t)$$

$$q_2(t+1) = 1 \cdot q_0(t) \oplus 0 \cdot q_1(t) \oplus 0 \cdot q_2(t) \oplus 1 \cdot q_3(t)$$

$$q_3(t+1) = 0 \cdot q_0(t) \oplus 1 \cdot q_1(t) \oplus 0 \cdot q_2(t) \oplus 0 \cdot q_3(t)$$

или в матричной форме

$$Q(t+1) = V Q(t),$$

где

$$V = T_2^2 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

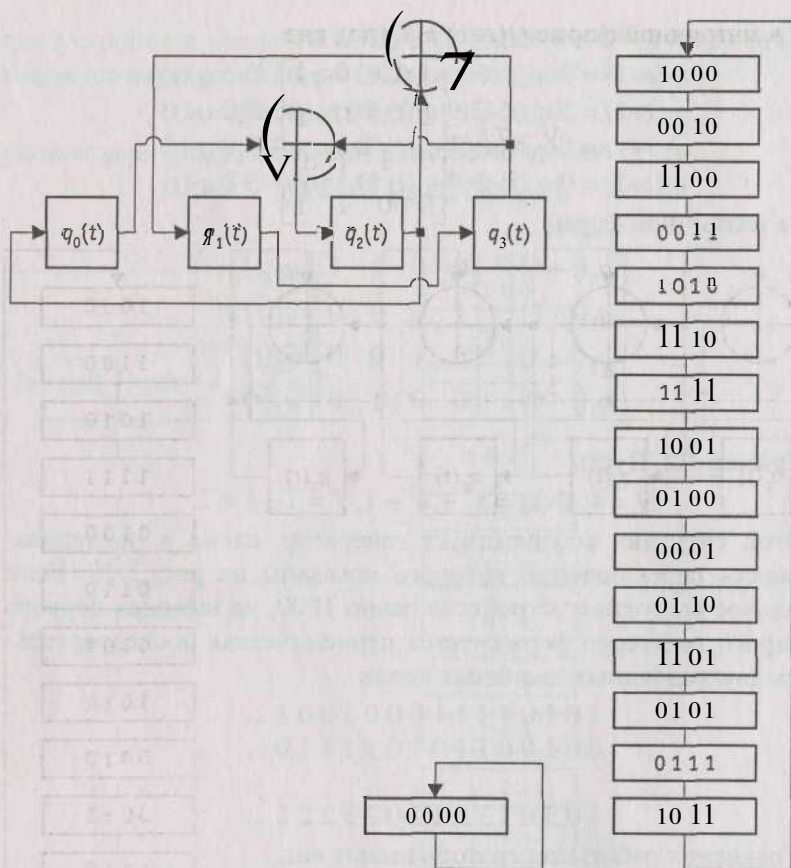


Рис. 3.10. Генератор двоичной M -последовательности, соответствующий $\Phi(x) = x^4 + x^3 + 1$, $T = T_2$, $k = 2$; и его диаграмма состояний

Пример 3.3. Пусть

$$N = 4, \Phi(x) = x^4 + x^3 + 1, T = T_2, k = 4.$$

Этой ситуации соответствует генератор, схема и диаграмма состояний которого показаны на рис. 3.11.

Уравнения работы генератора имеют вид:

$$q_0(t+1) = 1 \cdot q_0(t) \oplus 0 \cdot q_1(t) \oplus 0 \cdot q_2(t) \oplus 1 \cdot q_3(t)$$

$$q_1(t+1) = 1 \cdot q_0(t) \oplus 1 \cdot q_1(t) \oplus 0 \cdot q_2(t) \oplus 1 \cdot q_3(t)$$

$$q_2(t+1) = 0 \cdot q_0(t) \oplus 1 \cdot q_1(t) \oplus 1 \cdot q_2(t) \oplus 0 \cdot q_3(t)$$

$$q_3(t+1) = 0 \cdot q_0(t) \oplus 0 \cdot q_1(t) \oplus 1 \cdot q_2(t) \oplus 1 \cdot q_3(t)$$

или в матричной форме $Q(t+1) = VQ(t)$, где

$$V = T^* = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}.$$

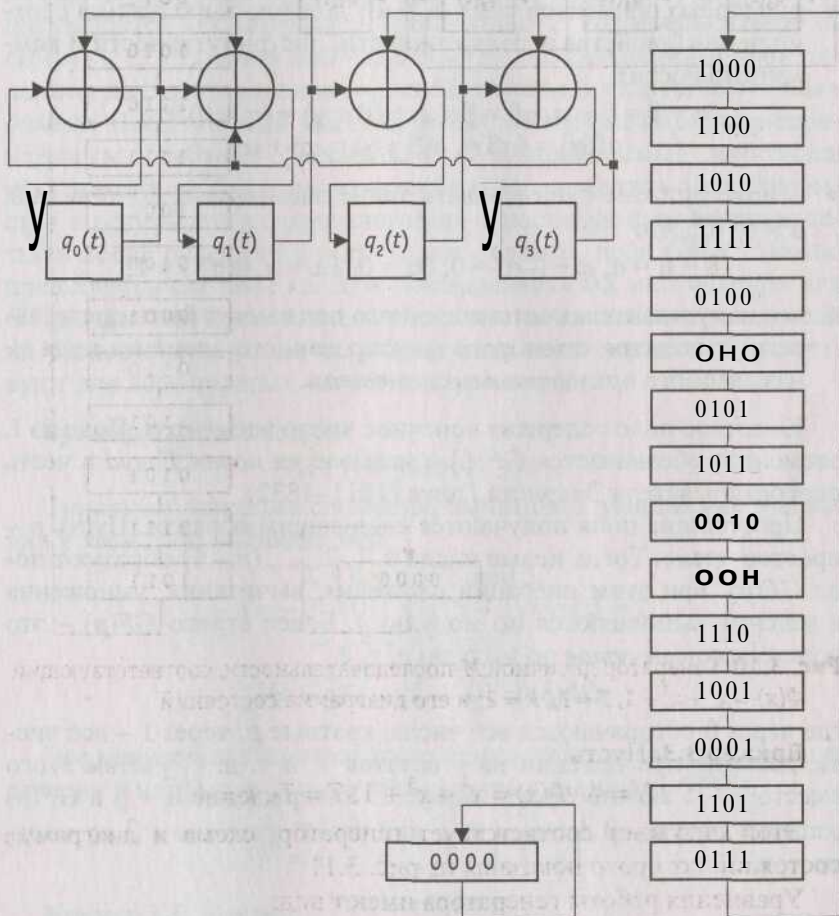


Рис. 3.11. Генератор двоичной M -последовательности, соответствующий $\Phi(x) = x^4 + x^3 + 1$, $T = T_2$, $k = 4$; и его диаграмма состояний

3.3. Основы теории конечных полей

Поле – это множество элементов, обладающее следующими свойствами:

- * в нем определены операции сложения, вычитания, умножения и деления;
- * для любых элементов поля α , β и γ должны выполняться соотношения (свойства ассоциативности, дистрибутивности и коммутативности)

$$\alpha + \beta = \beta + \alpha, \alpha\beta = \beta\alpha, \alpha + (\beta + \gamma) = (\alpha + \beta) + \gamma, \\ \alpha(\beta\gamma) = (\alpha\beta)\gamma, \alpha(\beta + \gamma) = \alpha\beta + \alpha\gamma;$$

в поле должны существовать такие элементы 0 , 1 , $-a$ и (для $a \neq 0$) a^{-1} , что

$$0 + a = a, a + (-a) = 0, 0a = 0, 1a = a, a(a^{-1}) = 1;$$

- * все ненулевые элементы конечного поля могут быть представлены в степени некоторого фиксированного элемента поля α , называемого *примитивным элементом*.

Конечное поле содержит конечное число элементов. Поле из L элементов обозначается $GF(L)$ и называется *полем Галуа* в честь первооткрывателя Эвариста Галуа (1811–1832).

Простейшие поля получаются следующим образом. Пусть p – простое число. Тогда целые числа $0, 1, 2, \dots, (p-1)$ образуют поле $GF(p)$, при этом операции сложения, вычитания, умножения и деления выполняются по модулю p . Более строго $GF(p)$ – это поле *классов вычетов* по модулю p , т. е.

$$GF(p) = \{0, 1, 2, \dots, (p-1)\},$$

где через 0 обозначаются все числа, кратные p , через 1 – все числа, дающие при делении на p остаток 1 , и т. д. С учетом этого вместо $p-1$ можно писать минус 1 . Утверждение $a = p$ в $GF(p)$ означает, что $a - p$ делится на p или что $\alpha \equiv p \pmod p$, т. е.

$$a \equiv p \pmod p.$$

Поле, содержащее $L = p^n$ элементов, где p – простое число, а n – натуральное, не может быть образовано из совокупности целых чисел по модулю L . Например, в множестве классов вычетов по модулю 4 элемент 2 не имеет обратного, так как $2 \cdot 2 = 0$.

Таким образом, хотя это множество состоит из 4 элементов, оно совсем не похоже на поле $GF(L)$. Чтобы подчеркнуть это различие, обычно вместо $GF(4)$ пишут $GF(2^2)$.

Элементами поля из p^n элементов являются все многочлены степени не более $n - 1$ с коэффициентами из поля $GF(p)$. Сложение в $GF(p^n)$ выполняется по обычным правилам сложения многочленов, при этом операции приведения подобных членов осуществляются по модулю p . Многочлен с коэффициентами из $GF(p)$ (т. е. *многочлен над полем $GF(p)$*), не являющийся произведением двух многочленов меньшей степени, называется *неприводимым*. Прimitивный многочлен автоматически является неприводимым. Выберем фиксированный неприводимый многочлен $\varphi(x)$ степени n . Тогда произведение двух элементов поля получается в результате их перемножения с последующим взятием остатка после деления на $\varphi(x)$. Таким образом, поле $GF(p^n)$ можно представить как поле *классов эквивалентности* многочленов над $GF(p)$. Два таких многочлена объявляются эквивалентными, если их разность делится на $\varphi(x)$. Конечные поля порядка p^n существуют для всех простых p и всех натуральных n .

Пример 3.4. Рассмотрим

$$GF(5) = \{0, 1, 2, 3, 4\}.$$

Типичные операции сложения, вычитания, умножения и деления в этом поле выглядят так:

$$\begin{aligned} 2 + 4 &= 6 = 1 \bmod 5, \\ 1 - 4 &= -3 = 0 - 3 = (5 - 3) \bmod 5 = 2, \\ 4 \cdot 2 &= 8 = 3 \bmod 5, \\ \frac{3}{2} &= \frac{(0+3)}{2} = \frac{(5+3) \bmod 5}{2} = 4. \end{aligned}$$

Все ненулевые элементы этого поля можно представить в виде степени 2 или 3, т. е. 2 и 3 - примитивные элементы $GF(5)$:

$$\begin{aligned} 2^0 &= 1, 2^1 = 2, 2^2 = 4, 2^3 = 3 \\ 3^0 &= 1, 3^1 = 3, 3^2 = 4, 3^3 = 2. \end{aligned}$$

Пример 3.5. Пусть

$$p = 2, n = 4, \varphi(x) = x^4 + x + 1$$

- примитивный над $GF(2)$. Элементы поля $GF(2^4)$ имеют вид

$$0, 1, x, x + 1, \dots, x^3 + x^2 + x + 1.$$

Так как $\varphi(x)$ - примитивный, ему соответствует устройство, диаграмма состояний которого состоит из цикла максимальной длины $2^4 - 1$ и одного тривиального цикла, включающего состояние 0000, переходящее само в себя (рис. 3.12).

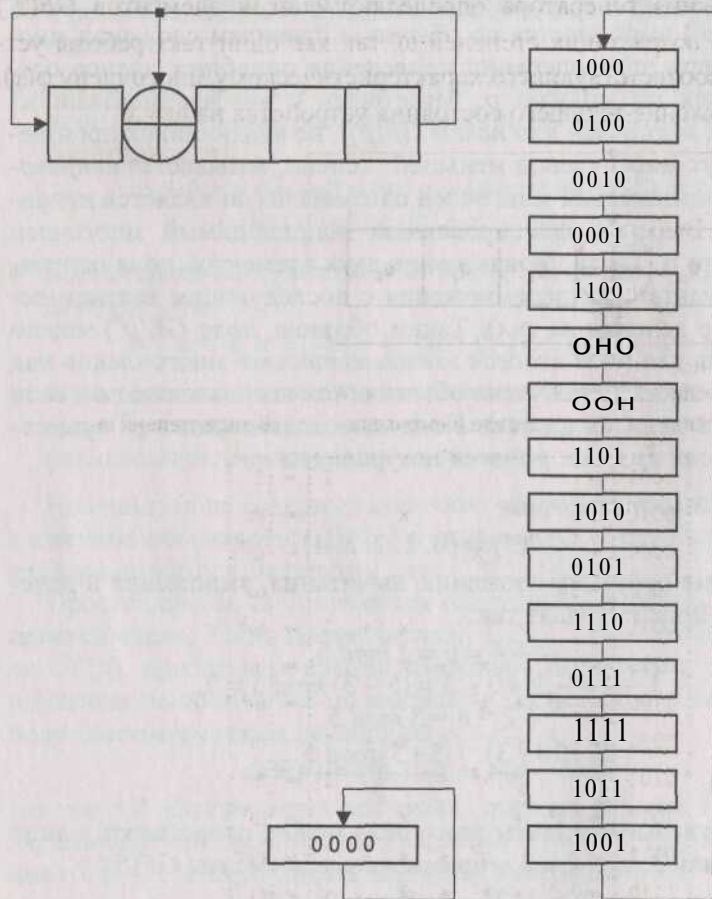


Рис. 3.12. *LFSR*, соответствующий характеристическому многочлену $\varphi(x) = x^4 + x + 1$, и его диаграмма состояний

Таким образом, в качестве α можно взять корень $\varphi(x)$, а устройство, для которого $\varphi(x) = x^4 + x + 1$ является характеристическим многочленом, объявить *генератором ненулевых элементов поля*. В результате соответствие между различными представле-

ниями элементов поля имеет вид, представленный на рис. 3.13. Показано представление в виде $q_3q_2q_1q_0$, иными словами набора двоичных коэффициентов многочлена; в виде двоичного многочлена степени не более 3 и в виде степеней примитивного элемента. Состояния генератора определяют список элементов $GF(2^4)$ в порядке возрастания степеней ω , так как один такт работы устройства, соответствующего характеристическому многочлену $\phi(x)$, суть умножение текущего состояния устройства на $\omega = x$.

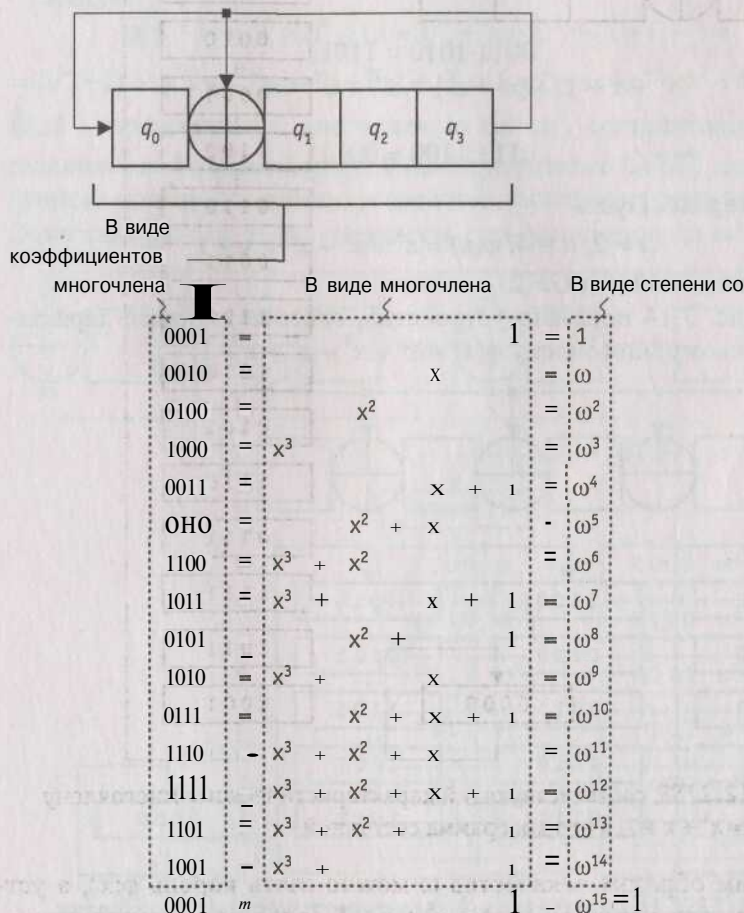


Рис. 3.13. Соответствие между различными формами представления элементов поля $GF(2^4)$

Типичные операции сложения, умножения и деления в поле $GF(2^4)$ в рассматриваемом случае выглядят следующим образом:

$$(x^3 + x^2 + 1) + (x^2 + x + 1) = x^3 + x$$

или

$$1101 + 0111 = 1010;$$

$$(x + 1)(x^3 + x) = \omega^4 \cdot \omega^9 = \omega^{13} = x^3 + x^2 + 1$$

или

$$(x + 1)(x^3 + x) = x^4 + x^3 + x^2 + x \pmod{\varphi(x)} = x^3 + x^2 + 1$$

или

$$0011 \cdot 1010 = 1101;$$

$$(x^2 + x + 1) : (x^3 + x^2) = \omega^{10} : \omega^6 = \omega^4 = x + 1$$

или

$$0111 : 1100 = 0011.$$

Пример 3.6. Пусть

$$p = 2, n = 4, \varphi(x) = x^4 + x^3 + x^2 + x + 1$$

- неприводимый над $GF(2)$.

На рис. 3.14 показано устройство, соответствующее характеристическому многочлену $\varphi(x) = x^4 + x^3 + x^2 + x + 1$.

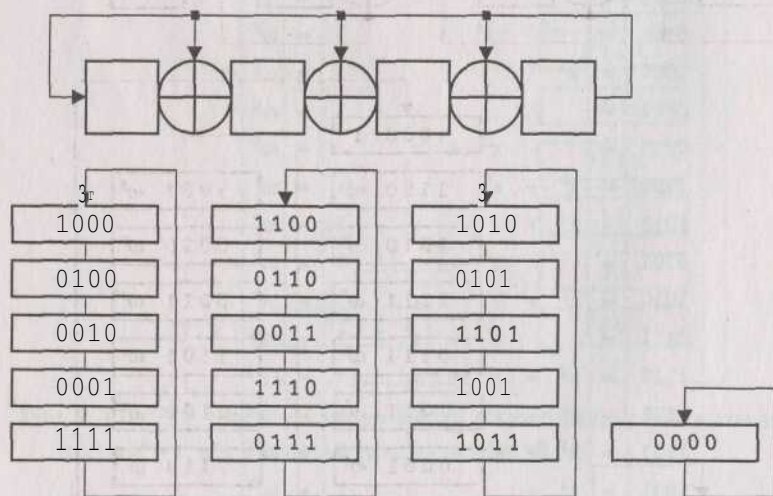


Рис. 3.14. LFSR соответствующий характеристическому многочлену $\varphi(x) = x^4 + x^3 + x^2 + x + 1$, и его диаграмма состояний

Диаграмма состояний устройства состоит из трех циклов длиной 5 и одного тривиального цикла, а значит, данное устройство, один такт которого суть умножение на x по модулю $\varphi(x)$, не может использоваться в качестве генератора элементов поля. Такая ситуация всегда имеет место, когда $\varphi(x)$ - неприводимый. Определим структуру устройства (генератора элементов поля), позволяющего сопоставить каждому ненулевому элементу поля соответствующую степень примитивного элемента.

Имеем

$$\begin{aligned}\varphi(x+1) &= (x+1)^4 + (x+1)^3 + (x+1)^2 + (x+1) + 1 = \\ &= (x^4 + 1) + (x^3 + x^2 + x + 1) + (x^2 + 1) + (x + 1) + 1 = x^4 + x^3 + 1 = \tilde{\varphi}(x).\end{aligned}$$

$\tilde{\varphi}(x)$ - примитивный многочлен, а значит, соответствие между различными формами представления элементов $GF(2^4)$ можно получить, моделируя работу устройства, показанного на рис. 3.15. Один такт работы этого устройства суть умножение на $\alpha = x + 1$.

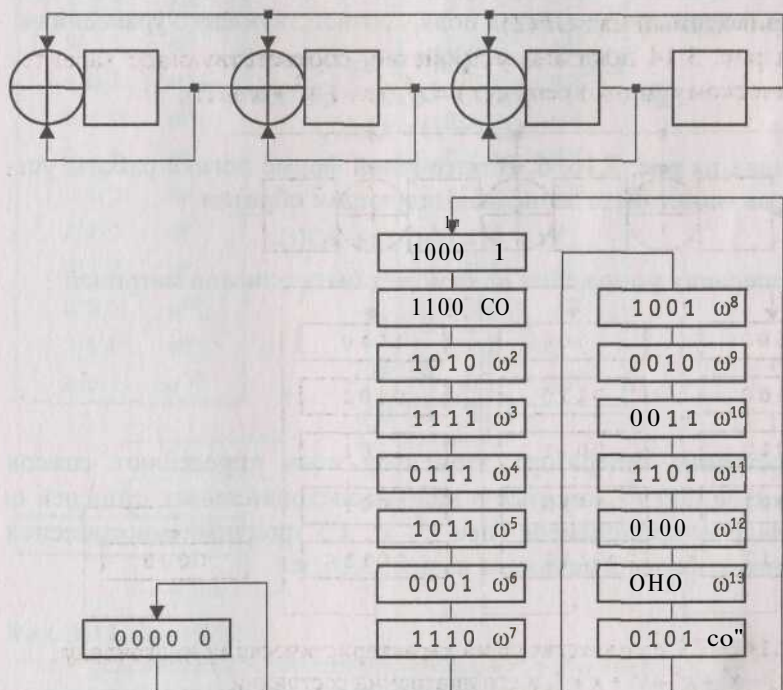


Рис. 3.15. Генератор элементов поля $GF(2^4)$

Пример 3.7. Пусть $p = 3$, $n = 3$, $\varphi(x) = 2x^3 + x^2 + x + 1$ - неприводимый над $GF(3)$. $\Phi(x) = x^3 + x^2 + x + 2$, а значит, соответствующий троичный генератор имеет вид, показанный на рис. 3.16, а. Элементы поля $GF(3^3)$ имеют вид

$$0, 1, 2, x, x + 1, x + 2, 2x, 2x + 1, 2x + 2, \dots, 2x^2 + 2x + 2.$$

Характеристический многочлен $\varphi(x)$ не является примитивным над $GF(3)$ в результате диаграмма состояний устройства состоит из двух кодовых колец по 13 состояний в каждом и одного вырожденного тривиального цикла, соответствующего состоянию 000, переходящему само в себя.

Имеем

$$\Phi(x+2) = (x+2)^3 + (x+2)^2 + (x+2) + 2 = x^3 + x^2 + 2x + 1 = \tilde{\Phi}(x).$$

$\tilde{\Phi}(x)$ является примитивным над $GF(3)$, а значит, в качестве генератора элементов поля $GF(2^4)$ можно взять устройство, такт работы которого эквивалентен умножению на $x + 2$ по модулю $\varphi(x)$. Схема генератора элементов поля, соответствующего уравнениям

$$\begin{aligned} Q_0(t+1) &= 2Q_0(t) + Q_2(t) \pmod{3}, \\ Q_1(t+1) &= Q_0(t) + 2Q_1(t) + Q_2(t) \pmod{3}, \\ Q_2(t+1) &= Q_1(t), \end{aligned}$$

показана на рис. 3.16, б. В матричной форме логика работы устройства может быть записана следующим образом

$$Q(t+1) = T_2 Q(t) + 2Q(t),$$

т. е. операция умножения на со может быть описана матрицей

$$\begin{bmatrix} 2 & 0 & 1 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Состояния генератора элементов поля определяют список элементов $GF(3^3)$, которые в виде последовательных степеней со и в виде коэффициентов (при x^0, x^1, x^2) троичных многочленов степени не более 2 показаны на рис. 3.16, в.

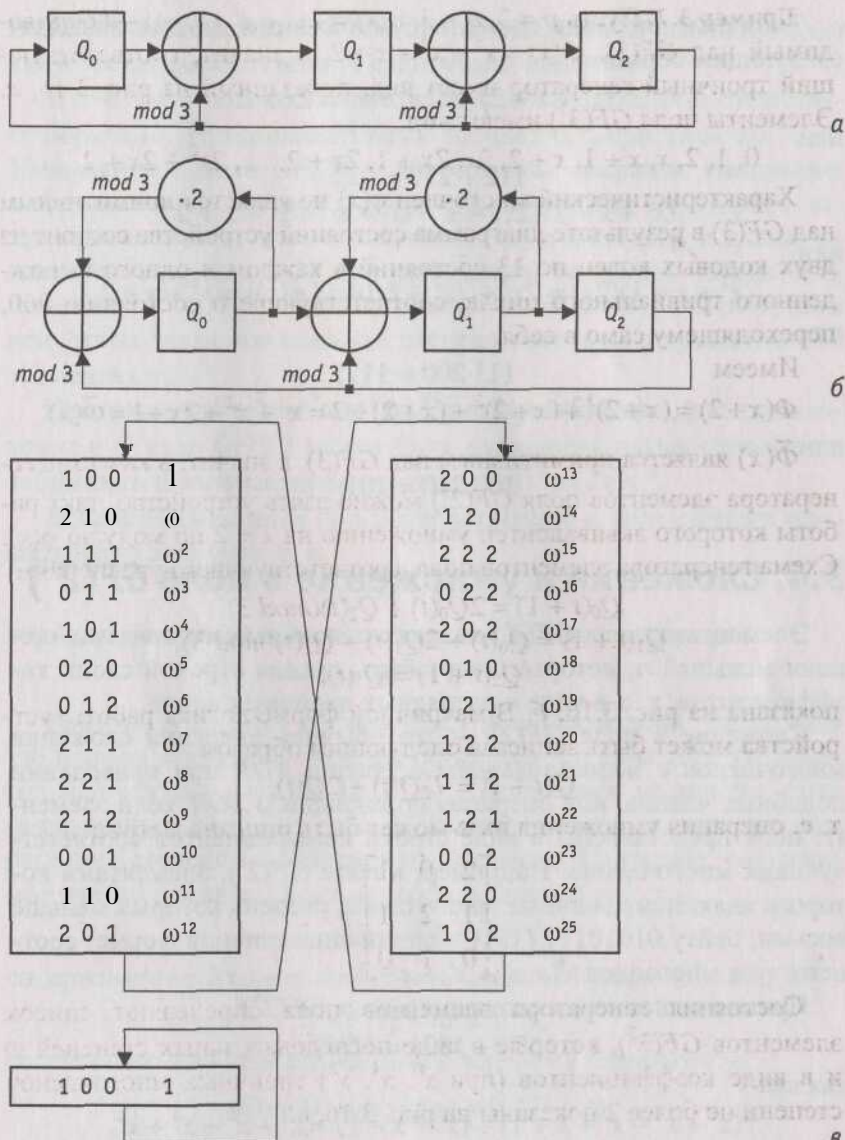


Рис. 3.16. Поле $GF(3^3)$:

а - устройство, соответствующее $\Phi(x) = x^3 + x^2 + x + 2$;

б - генератор элементов поля $GF(3^3)$;

в - диаграмма состояний генератора элементов поля $GF(3^3)$

Типичные операции сложения, умножения и деления выглядят следующим образом:

$$(x^2 + x + 2) + (2x^2 + x) = 2x + 2$$

или

$$112 + 210 = 022;$$

$$(x^2 + x + 1)(2x) = \omega^2 \cdot \omega^5 = \omega^7 = x^2 + x + 2$$

или

$$(x^2 + x + 1)(2x) = 2x^3 + 2x^2 + 2x \pmod{\phi(x)} = x^2 + x + 2$$

или

$$111 \cdot 200 = 112;$$

$$(x^2) : (2x^2 + x) = \omega^{10} : \omega^6 = \omega^4 = x^2 + 1$$

или

$$100 : 210 = 101.$$

3.4. Сложение и умножение в поле $GF(2^n)$

Элементами поля $GF(2^n)$ являются двоичные многочлены степени меньшей n , которые могут быть заданы строкой своих коэффициентов, т. е. в виде n -разрядных двоичных кодов.

Сложение в поле $GF(2^n)$ - это обычная операция сложения многочленов с использованием операции XOR при приведении подобных членов; или операция поразрядного XOR , если элементы поля представлены в виде строки коэффициентов соответствующих многочленов. Например, в поле $GF(2^8)$, элементами которого являются двоичные многочлены, степень которых меньше восьми, байту 01010111 ($\{57\}$ в шестнадцатеричной форме) соответствует многочлен $x^6 + x^4 + x^2 + x + 1$.

Пример выполнения сложения в поле $GF(2^8)$:

$$\{57\} + \{83\} = \{D4\},$$

так как

$$(x^6 + x^4 + x^2 + x + 1) + (x^7 + x + 1) = x^7 + x^6 + x^4 + x^2$$

или

$$01010111 + 10000011 = 11010100.$$

В конечном поле для любого ненулевого элемента a существует обратный аддитивный элемент $-a$, при этом $a + (-a) = 0$.

В $GF(2^n)$ справедливо $a + 0a = 0$, т. е. каждый ненулевой элемент является своей собственной аддитивной инверсией.

В конечном поле для любого ненулевого элемента a существует обратный мультипликативный элемент a^{-1} , при этом $aa^{-1} = 1$. Умножение в поле $GF(2^n)$ — это обычная операция умножения многочленов со взятием результата по модулю некоторого неприводимого двоичного многочлена $\phi(x)$ n -й степени и с использованием операции XOR при приведении подобных членов. Умножение в $GF(2^n)$ также можно выполнять, рассматривая ненулевые элементы поля как степени некоторого примитивного элемента ω .

Программная реализация операции умножения двух элементов α и β поля $GF(2^n)$ может быть выполнена двумя способами: табличным и вычислением результата $\alpha\beta$ "на лету".

Если элементы поля α и β (3) представлены в виде степени примитивного элемента, т. е.

$$\alpha = \omega^i \text{ и } \beta = \omega^j,$$

то их произведение $\alpha\beta$ может быть вычислено по формуле

$$\alpha\beta = \omega^{(i+j) \bmod (2^n-1)}.$$

Именно этот факт используется при реализации умножения табличным способом. Формируются два массива *Elem* и *Addr*. Первый массив состоит из $2^n - 1$ ненулевых элементов поля (n -разрядных двоичных кодов), расположенных в порядке возрастания степеней примитивного элемента. Например, содержимое ячейки массива *Elem* с адресом 0 равно $\omega^0 = 1$, т. е.

$$Elem[0] = \omega^0;$$

содержимое ячейки массива *Elem* с адресом 1 равно $\omega^1 = \omega$, т. е.

$$Elem[1] = \omega^1 \text{ и т. д.,}$$

содержимое ячейки массива *Elem* с адресом i равно ω^i , т. е.

$$Elem[i] = \omega^i; \text{ где } i = 0, (2^n - 2).$$

Второй массив формируется для ускорения поиска нужного элемента в массиве *Elem*, в каждой ячейке массива *Addr* с адресом j содержится адрес элемента поля j в массиве *Elem*, $j = 1, (2^n - 1)$, т. е.

$$Elem[i] = a \Leftrightarrow Addr[\alpha] = i, a \in GF(2^n), a \neq 0.$$

Процедура определения результата $\alpha\beta$ умножения двух элементов поля

$$a = \omega^i \text{ и } P = \omega^j$$

с использованием массивов *Elem* и *Addr* основана на нахождении в массиве *Elem* элемента a и считывании результата умножения из ячейки массива *Elem*, циклически смещенной в сторону старших адресов относительно ячейки, содержащей α , на; позиций, т. е.

$$ap = Elem[(i + j) \bmod T - 1] = Elem[(Addr[\alpha] + Addr[\beta]) \bmod 2^n - 1].$$

Рассмотрим поле $GF(2^4)$, порожденное многочленом $\varphi(x) = x^4 + x + 1$. На рис. 3.17 показан пример умножения двух элементов поля $\alpha = 0111$ и $P = 0010$:

$$0111 \cdot 0010 = 1010.$$

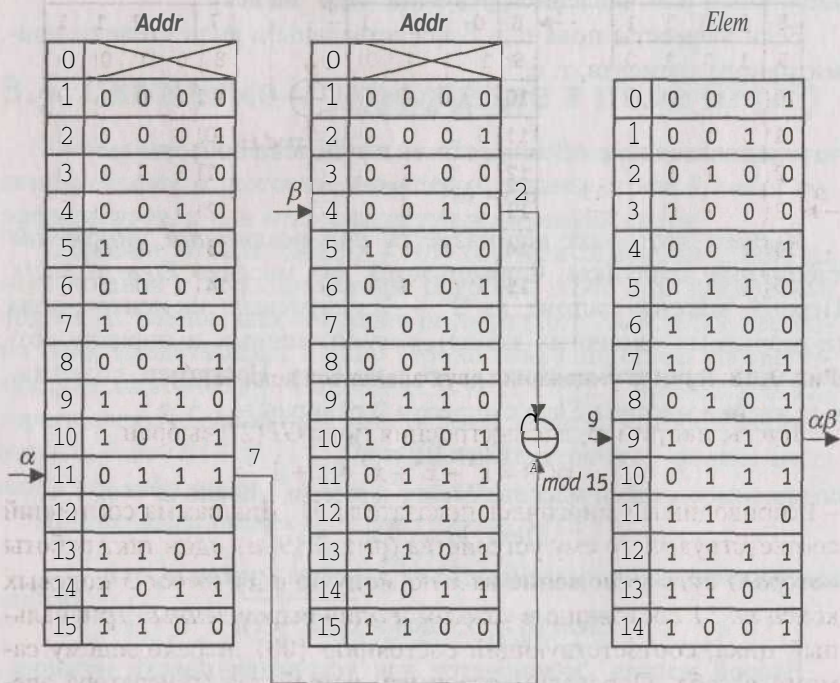


Рис. 3.17. Пример умножения двух элементов поля $GF(2^4)$

На рис. 3.18 показан пример умножения двух элементов поля $a = 1101$ и $(3 = 0011)$:

$$1101 \cdot 0011 = 0010.$$

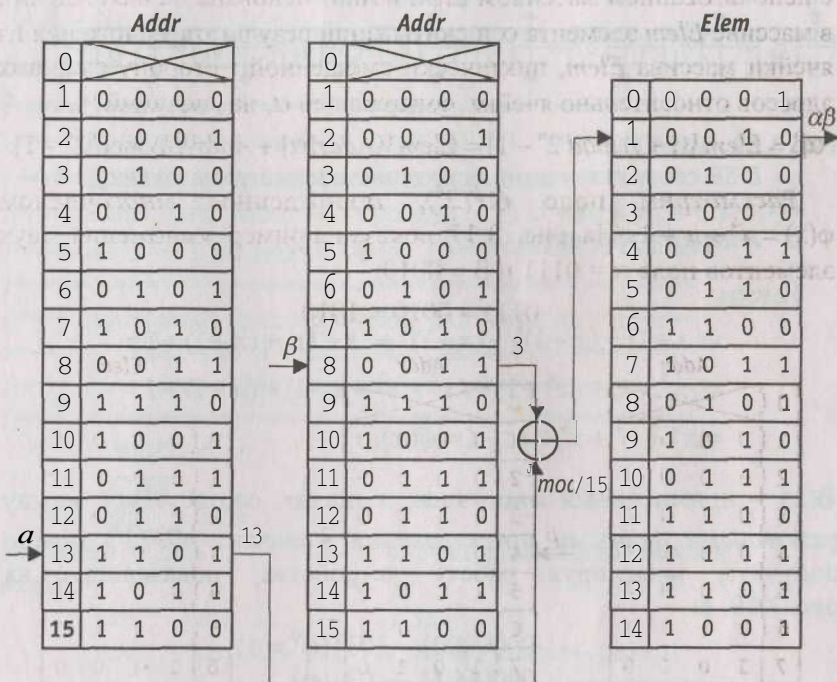


Рис. 3.18. Пример умножения двух элементов поля $GF(2^4)$

Пусть, например, для построения поля $GF(2^8)$ выбран

$$\phi(x) = x^8 + x^4 + x^3 + x + 1$$

- неприводимый многочлен показателя 51. Диаграмма состояний соответствующего ему устройства (рис. 3.19, а), один такт работы которого суть умножение на x по модулю $\phi(x)$, имеет 5 кодовых колец по 51 состоянию в каждом и один вырожденный тривиальный цикл, соответствующий состоянию $\{00\}$, переходящему самому в себя. Определим структуру устройства (генератора элементов поля), позволяющего сопоставить каждому ненулевому элементу поля соответствующую степень примитивного элемента.

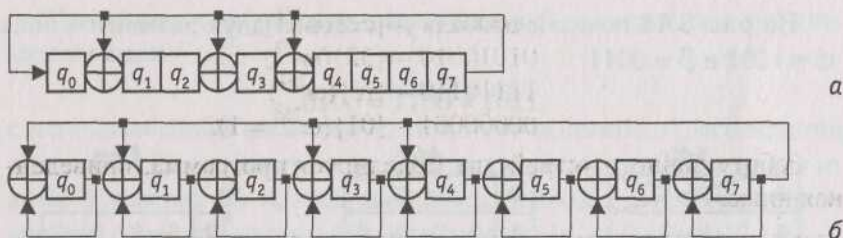


Рис. 3.19. Поле $GF(2^8)$:

а - LFSR, соответствующий характеристическому многочлену

$$\varphi(x) = x^8 + x^4 + x^3 + x + 1,$$

б - генератор элементов $GF(2^8)$

Имеем

$$\begin{aligned} \varphi(x+1) &= (x+1)^8 + (x+1)^4 + (x+1)^3 + (x+1) + 1 = \\ &= (x^8 + 1) + (x^4 + 1) + (x^3 + x^2 + x + 1) + (x+1) + 1 = \\ &= x^8 + x^4 + x^3 + x^2 + 1 = \bar{\varphi}(x). \end{aligned}$$

$\bar{\varphi}(x)$ - примитивный многочлен, а значит, соответствие между различными формами представления элементов $GF(2^8)$ можно получить, моделируя работу устройства, показанного на рис. 3.19, б:

$$00000001 - \{01\}(\omega^0 = 1)$$

$$00000011 - \{03\}(\omega)$$

$$00000101 - \{05\}(\omega^2)$$

$$00001111 - \{0f\}(\omega^3)$$

$$00010001 - \{11\}(\omega^4)$$

$$00110011 - \{33\}(\omega^5)$$

$$01010101 - \{55\}(\omega^6)$$

$$11111111 - \{ff\}(\omega^7)$$

$$00011010 - \{1a\}(\omega^8)$$

$$00101110 - \{2e\}(\omega^9)$$

...

$$11110111 - \{f7\}(\omega^{25})$$

$$00000010 - \{02\}(\omega^{26})$$

$$00000110 - \{06\}(\omega^{27})$$

...

11000111 - $\{c7\}(\omega^{252})$
 01010010 - $\{52\}(\omega^{253})$
 11110110 - $\{f6\}(\omega^{254})$
 00000001 - $\{01\}(\omega^{255} = 1)$.

Работу данного устройства моделирует программа, приведенная ниже.

```

;=====
;==== Генератор элементов поля  $GF(2^8)$ , =====
;==== если  $\phi(x)$  не является примитивным. =====
;==== Один такт работы устройства суть умножение
;==== на  $x + 1$  по модулю  $\phi(x)$ . =====
;=====
;==== Программа предназначена для запуска =====
;==== в отладчике в пошаговом режиме. =====
;==== AL - состояние генератора. =====
;=====

.MODEL tiny
.CODE
ORG 100h
FeedBack = 1Bh ; Вектор обратных связей
Begin:
    mov al, 1 ; AL = '01'
    mov cx, 255 ; Число циклов равно
                ; количеству ненулевых
                ; элементов поля

Step:
    mov ah, al ; Копия «старого»
                ; состояния генератора
    rcl al, 1 ; Сдвиг LFSR и анализ
                ; бита обратной связи
    jnc Next ; Если бит обратной связи
                ; равен нулю, коррекции
                ; результата умножения
                ; на  $x$  не требуется

    xor al, FeedBack
; Действие единичного
; бита обратной связи
  
```

Next:

```

xor    al, ah      ; AL - «новое» состояние
                    ; генератора
loop   Step        ; Если CX не равно 0, переход
                    ; на начало следующего такта
                    ; работы генератора

mov     ax, 4c00h
int     21h
END     Begin

```

; =====

Пример выполнения операции умножения в рассматриваемом поле:

$$\begin{aligned}
 & (x^6 + x^4 + x^2 + x + 1)(x^7 + x + 1) = \\
 & = x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1 = \\
 & (x^7 + x^6 + 1) \bmod \varphi(x)
 \end{aligned}$$

или

$$\omega^{98} \cdot \omega^{80} = \omega^{178},$$

а значит,

$$\{57\} \cdot \{83\} = \{c1\}.$$

В $GF(2^8)$ справедливо

$$\omega^{255} = 1,$$

а значит, ненулевые элементы ω^i и ω^j ($i \neq j$) являются взаимно обратными тогда и только тогда, когда

$$i + j = 255.$$

Например,

$$\omega^4 \cdot \omega^{251} = \omega^{255}$$

т. е.

$$\{11\} \cdot \{b4\} = 1$$

или

$$(x^4 + 1)(x^7 + x^5 + x^4 + x^2) = 1.$$

Умножая произвольный многочлен

$$a(x) = a_7x^7 + a_6x^6 + a_5x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

на x , получим в общем случае многочлен 8-й степени

$$ax^8 + a_6x^7 + a_5x^6 + a_4x^5 + a_3x^4 + a_2x^3 + a_1x^2 + a_0x.$$

Если $a_7 = 0$, мы сразу получаем результат умножения; если $a_7 = 1$, необходимо взять результат по модулю $\phi(x)$, что в рассматриваемой ситуации, очевидно, эквивалентно вычитанию из результата $\phi(x)$. Таким образом, умножение $a(x)$ на x на байтовом уровне это либо результат циклического сдвига в сторону старших разрядов байта

$$(a_7a_6a_5a_4a_3a_2a_1a_0),$$

если $a_7 = 0$; либо поразрядный XOR результата циклического сдвига байта

$$(a_7a_6a_5a_4a_3a_2a_1a_0)$$

в сторону старших разрядов с вектором обратной связи {1b} (рис. 3.19, а), если $a_7 = 1$.

Пусть

$$xTime(\alpha)$$

- операция умножения элемента поля a на x . Тогда умножение на x^n можно осуществить путем n -кратного повторения операции $xTime(\alpha)$.

Например,

$$\{57\} \cdot \{13\} = \{fe\},$$

так как

$$\{57\} \cdot \{02\} = xTime(57) = \{ae\},$$

$$\{57\} \cdot \{04\} = xTime(ae) = \{47\},$$

$$\{57\} \cdot \{08\} = xTime(47) = \{8e\},$$

$$\{57\} \cdot \{10\} = xTime(8e) = \{07\},$$

$$\{57\} \cdot \{13\} = \{57\} \cdot (\{01\} \oplus \{02\} \oplus \{10\}) = \{57\} \oplus \{ae\} \oplus \{07\} = \{fe\}.$$

Ниже приведен пример вычисления "на лету" произведения двух элементов $GF(2^8)$ по приведенному выше алгоритму.

```
;=====
;==== Mulgf - процедура умножения =====
;==== двух элементов поля GF(2^8) . =====
;=====
;==== при вызове: =====
;==== АН - первый сомножитель, =====
;==== АЛ - второй сомножитель. =====
```



```

;==== DL - вектор обратных связей =====
;==== генератора элементов поля. =====
;==== При возврате: =====
;==== AL - результат умножения. =====
;=====
Mulgf ' PROC
    test    al, al        ; Первый сомножитель равен 0 ?
    jz      Result       ; Если да, на выход -
                                ; результат равен 0
    xchg    ah, al
    test    al, al        ; Второй сомножитель равен 0 ?
    jz      Result       ; Если да, на выход -
                                ; результат равен 0
    push    bx cx
    mov     cx, 8          ; Число повторений
    xor     БИ, БИ        ; Инициализация регистра,
                                ; в котором формируется
                                ; результат
NextShift:
    shr     ah, 1          ; Анализ очередного
                                ; бита 1-го сомножителя
    jnc     xTime          ; Если выдвигаемый бит
                                ; равен нулю, готовимся
                                ; к следующему циклу
    xor     bl, al         ; Формирование в BL
                                ; промежуточного результата
    ; ==== xTime ( $\alpha$ ), AL = a =====
xTime:
    shl     al, 1
    jnc     EndL
    xor     al, dl
    ; =====
EndL:
    loop    NextShift
    mov     al, БИ        ; Результат умножения в AL
    pop     cx bx
Result:
    ret
Mulgf     ENDP
=====

```

3.5. Устройства, функционирующие в $GF(L)$, $L > 2$

Пусть $L > 2$ - степень простого числа, $GF(L)$ - поле Галуа из L элементов. Общий вид генератора L -ричных последовательностей, соответствующего уравнению

$$Q(t+1) = T^k Q(t),$$

где $Q(t)$ и $Q(t+1)$ - состояния генератора соответственно в моменты времени t и $t + 1$ (до и после прихода синхроимпульса); T - квадратная матрица порядка N вида

$$T_1 = \begin{bmatrix} -\frac{a_1}{a_0} & -\frac{a_2}{a_0} & \dots & -\frac{a_{N-1}}{a_0} & -\frac{a_N}{a_0} \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ & & \dots & & \\ 0 & 0 & \dots & 1 & 0 \end{bmatrix}$$

или

$$T_2 = \begin{bmatrix} 0 & \dots & 0 & 0 & -\frac{a_N}{a_0} \\ 1 & \dots & 0 & 0 & -\frac{a_{N-1}}{a_0} \\ \dots & & & & \\ 0 & \dots & 1 & 0 & -\frac{a_2}{a_0} \\ 0 & \dots & 0 & 1 & -\frac{a_1}{a_0} \end{bmatrix},$$

N - степень образующего многочлена

$$\Phi(x) = \sum_{i=0}^N a_i x^i, \quad a_0 \neq 0, \quad a_i \in GF(L), \quad i = 0, N,$$

k - натуральное, показан на рис. 3.20, где $Q_i(t)$ - состояние i -го элемента памяти ($\lfloor \log_2 L \rfloor$ - разрядного регистра) генератора в момент времени t .

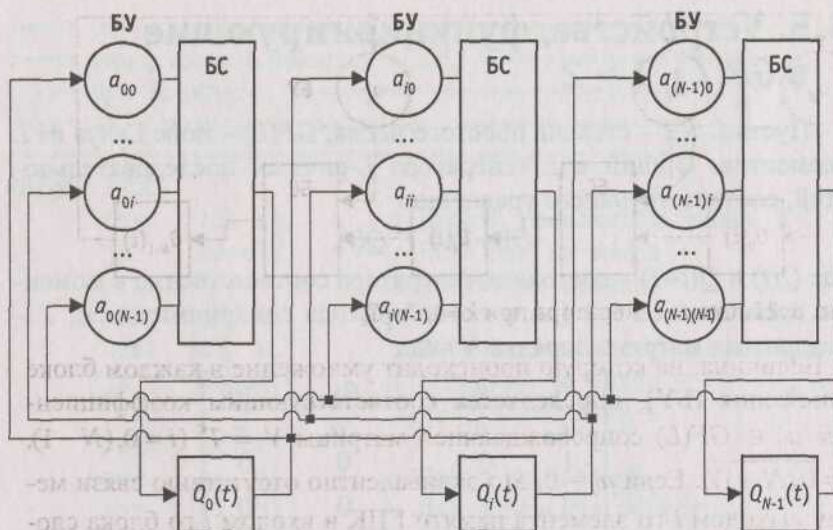


Рис. 3.20. Устройство, функционирующее в поле $GF(L)$ - генератор L -ричных последовательностей, соответствующий уравнению $Q(t+1) - T^k Q(t)$

При $k=1$ генератор имеет вид, показанный на рис. 3.21 ($T = T_1$) или рис. 3.22 ($T = T_2$).

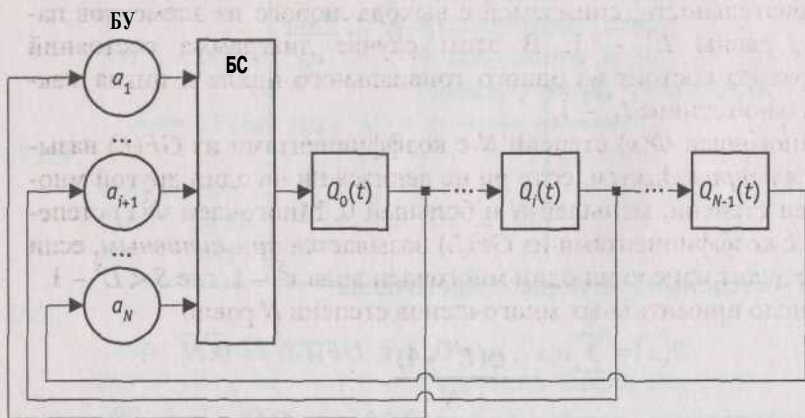
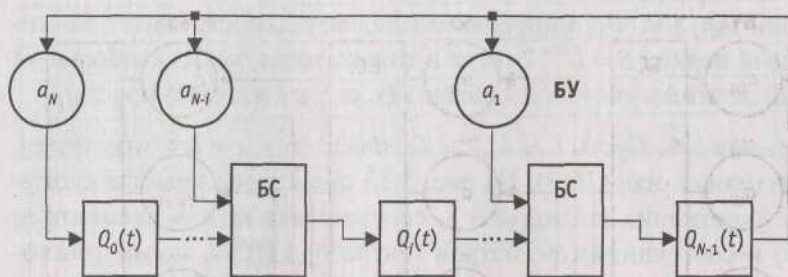


Рис. 3.21. Схема генератора при $k=1, T=T_1$


 Рис. 3.22. Схема генератора при $k=1$, $T=T_2$

Величина, на которую происходит умножение в каждом блоке умножения (БУ), определяется соответствующим коэффициентом $a_{ij} \in GF(L)$ сопровождающей матрицы $V = T^k$ ($i = 0, (N-1)$, $j = 0, (N-1)$). Если $a_{ij} = 0$, это эквивалентно отсутствию связи между выходом i -го элемента памяти ГПК и входом j -го блока сложения (БС). Если $a_{ij} = 1$, это эквивалентно наличию связи между выходом i -го элемента памяти генератора и входом j -го блока сложения. При $L = 2^n$ блоки сложения и умножения в поле $GF(2^n)$ реализуются на сумматорах по модулю два.

Максимально возможное число состояний устройства, а значит, и максимально возможная длина формируемой L -ричной последовательности, снимаемой с выхода любого из элементов памяти, равны $L^N - 1$. В этом случае диаграмма состояний генератора состоит из одного тривиального цикла и цикла максимальной длины $L^N - 1$.

Многочлен $\Phi(x)$ степени N с коэффициентами из $GF(L)$ называется *неприводимым*, если он не делится ни на один другой многочлен степени, меньшей N и большей 0. Многочлен $\Phi(x)$ степени N с коэффициентами из $GF(L)$ называется *примитивным*, если он не делит нацело ни один многочлен вида $x^S - 1$, где $S < L^N - 1$.

Число примитивных многочленов степени N равно

$$\frac{\varphi(L^N - 1)}{N},$$

где $\varphi(n)$ - функция Эйлера (число натуральных чисел, меньших n и взаимно простых с n).

Пусть $\Phi(x)$ - образующий многочлен степени N , примитивный над $GF(L)$, тогда справедливо следующее утверждение.

Свойство 3.4. Формируемая последовательность имеет максимальный период $S = L^{N-1} - 1$ тогда и только тогда, когда наибольший общий делитель чисел 5 и k равен 1 (т. е. 5 и k взаимно просты).

Пример 3.8. Пусть $L = 3$, $T = T_1$, $\Phi(x) - x^2 + x + 2$ - многочлен, примитивный над $GF(3)$. На рис. 3.23 приведены правила сложения и умножения по модулю 3, соответствие между элементами $GF(3)$ и состояниями регистров генератора ПСП, схемы генераторов и диаграммы состояний при $k = 1$ и $k = 3$. Последовательность переключения устройства при $k = 1$ показана сплошной линией, при $k = 3$ - пунктирной. При приведенном соответствии между элементами $GF(3)$ и состояниями регистров генератора умножение на 2 по модулю 3 эквивалентно простой передаче сигналов с первого и второго входов БУ соответственно на второй и первый выходы блока. При $k = 1$ сопровождающая матрица имеет вид

$$T_1 = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix},$$

а уравнения работы

$$\begin{aligned} Q_0(t+1) &= Q_0(t) + Q_1(t) \pmod{3} \\ Q_1(t+1) &= Q_0(t). \end{aligned}$$

При $k = 3$ сопровождающая матрица имеет вид

$$V = T_1^3 = \begin{bmatrix} 0 & 2 \\ 2 & 1 \end{bmatrix},$$

а уравнения работы

$$\begin{aligned} Q_0(t+1) &= 2Q_1(t) \pmod{3} \\ Q_1(t+1) &= 2Q_0(t) + Q_1(t) \pmod{3}. \end{aligned}$$

Пример 3.9. Пусть $L = 2^2$, $T = T_1$, $\Phi(x) = x^8 + x^3 + x + \omega$ - многочлен, примитивный над

$$GF(2^2) = \{0, 1, \omega, \omega^2\}, \omega^2 + \omega + 1 = 0, \omega^3 = 1.$$

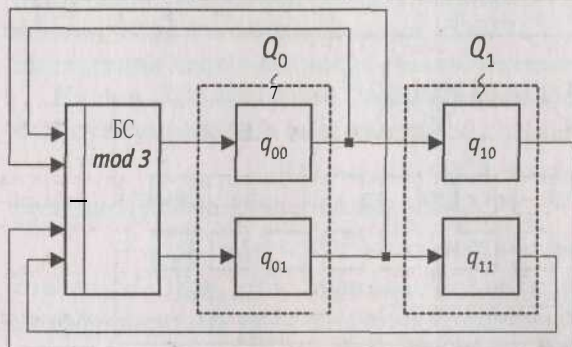
На рис. 3.24 приведены соответствие между элементами $GF(2^2)$ и состояниями регистров ГПК, схема генератора M -последовательности.

+	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

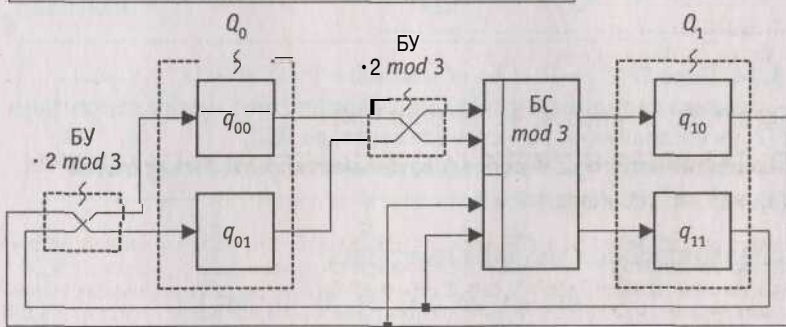
·	0	1	2
0	0	0	0
1	0	1	2
2	0	2	1

Элемент $GF(3)$	$Q_i = [q_{i1} q_{i0}]$
0	00
1	01
2	10

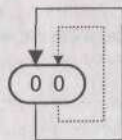
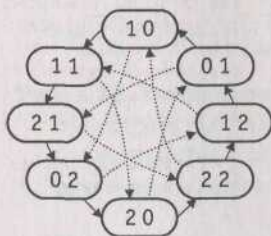
а



б



в



г

 Рис. 3.23. Поле $GF(3)$:

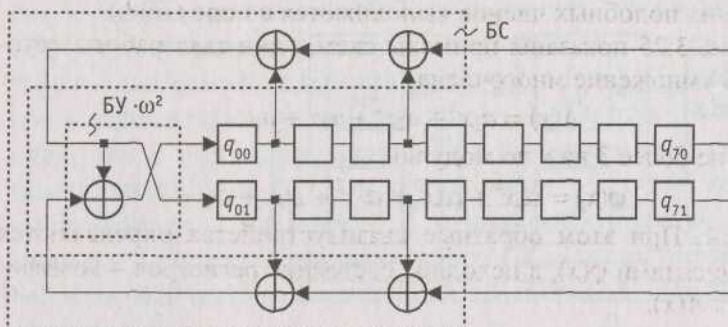
а - правила сложения и умножения по модулю 3, соответствие между элементами $GF(3)$ и состояниями регистров генератора ПСП;
 б и в - схемы генераторов троичной M -последовательности ($T = T_1$, $\Phi(x) = x^2 + x + 2$) соответственно при $k = 1$ и $k = 3$;
 г - диаграммы состояний генераторов

+	0	1	∞	ω^2
0	0	1	∞	ω^2
1	1	0	ω^2	∞
∞	∞	ω^2	0	1
ω^2	ω^2	∞	1	0

·	0	1	∞	ω^2
0	0	0	0	0
1	0	1	∞	ω^2
∞	∞	0	ω^2	1
ω^2	ω^2	0	1	∞

Элемент $GF(2^2)$	$q_i = [q_{i1} \ q_{i0}]$
0	00
1	01
∞	10
ω^2	11

a



6

Рис. 3.24. Поле $GF(2^2) = \{0, 1, \infty, \omega^2\}$, $\omega^2 + \infty + 1 = 0$, $\omega^3 = 1$:

а - правила сложения и умножения, соответствие между элементами $GF(2^2)$ и состояниями регистров генератора ПСП,

б - схема генератора M -последовательности, соответствующего $\Phi(x) = X^8 + x^3 + x + \infty$ при $\Gamma = T_1$

Сопровождающая матрица имеет вид

$$T_1 = \begin{bmatrix} \omega^2 & 0 & \omega^2 & 0 & 0 & 0 & 0 & \omega^2 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

а уравнения работы с учетом равенства $\infty^2 = \frac{1}{\infty}$

$$Q_0(t+1) = \frac{1}{\infty} (Q_0(t) + Q_2(t) + Q_7(t)) (GF(2^2))$$

$$Q_j(t+1) = Q_{j-1}(t), j = \overline{1, 7}.$$

3.6. Умножение и деление многочленов над $GF(L)$

Генераторы, функционирующие в $GF(L)$, используются для построения устройств, осуществляющих умножение и деление многочленов с коэффициентами из $GF(L)$. При этом все операции приведения подобных членов выполняются в поле $GF(L)$.

На рис. 3.25 показаны примеры схем, один такт работы которых суть умножение многочлена

$$A(x) = a_3x^3 + a_2x^2 + a_1x + a_0$$

степени не более 3 на x по модулю

$$\varphi(x) = \alpha_4x^4 + \alpha_3x^3 + \alpha_2x^2 + \alpha_1x + \alpha_0$$

степени 4. При этом обратные связи устройства определяются коэффициентами $\varphi(x)$, а исходное состояние регистров - коэффициентами $A(x)$.

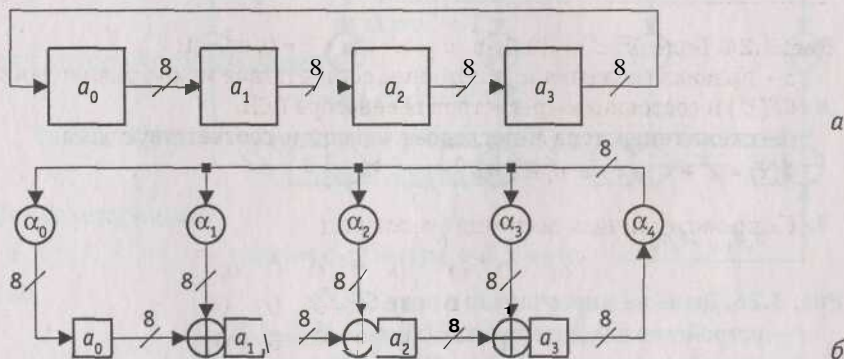


Рис. 3.25. Умножение многочленов в поле $GF(2^8)$:

а - устройство для умножения на x по модулю $\varphi(x) = x^4 + 1$,

б - устройство для умножения на x по модулю

$$\varphi(x) = \alpha_4x^4 + \alpha_3x^3 + \alpha_2x^2 + \alpha_1x + \alpha_0, \alpha_i \in GF(2^8), i = 0, 4$$

На рис. 3.26 показаны примеры схем для деления произвольного многочлена

$$A(x) = a_nx^n + a_{n-1}x^{n-1} + a_1x + a_0$$

степени n на многочлен

$$\varphi(x) = \alpha_4x^4 + \alpha_3x^3 + \alpha_2x^2 + \alpha_1x + \alpha_0$$

степени 4. При этом обратные связи устройства определяются коэффициентами $\varphi(x)$, исходное состояние регистров нулевое, а коэффициенты $A(x)$ подаются на вход старшими значениями вперед. Коэффициенты многочлена-частного снимаются с выхода старшего регистра старшими значениями вперед после прохождения начальных нулей. Коэффициенты многочлена-остатка остаются в регистрах устройства после обработки всех коэффициентов $A(x)$.

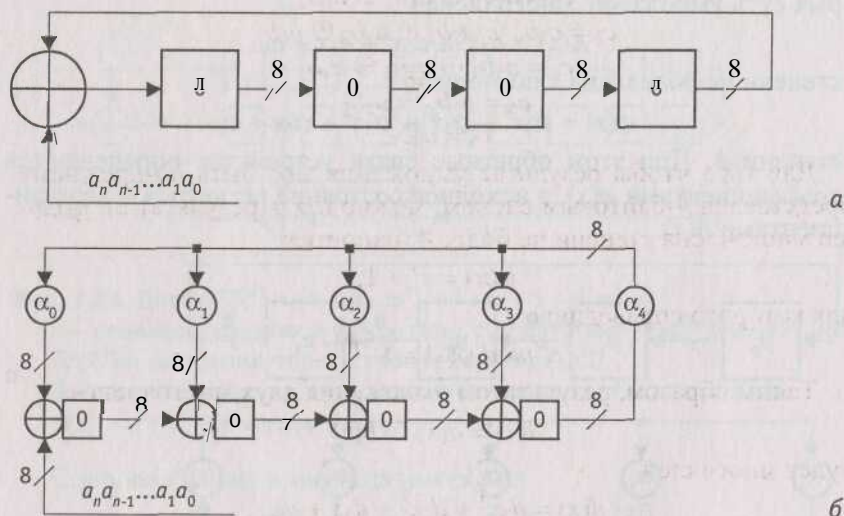


Рис. 3.26. Деление многочленов в поле $GF(2^8)$:

а - устройство для деления на $\varphi(x) = x^4 + 1$,

б - устройство для деления на $\varphi(x) = \alpha_4 x^4 + \alpha_3 x^3 + \alpha_2 x^2 + \alpha_1 x + \alpha_0$,

$\alpha_i \in GF(2^8)$, $i = \overline{0, 4}$

Четырехбайтовому слову может быть поставлен в соответствие многочлен с коэффициентами из $GF(2^8)$ степени не более 3. Сумма двух многочленов с коэффициентами из $GF(2^8)$ - это обычная операция сложения многочленов с приведением подобных членов в поле $GF(2^8)$. Таким образом, сложение двух четырехбайтовых слов суть операция поразрядного XOR.

Умножение - более сложная операция. Предположим, мы перемножаем два многочлена

$$a(x) = a_3 x^3 + a_2 x^2 + a_1 x + a_0 \text{ и } b(x) = b_3 x^3 + b_2 x^2 + b_1 x + b_0.$$

Результатом умножения

$$c(x) = a(x)b(x)$$

будет многочлен

$$c(x) = c_6x^3 + c_5x^2 + c_4x + c_3x^3 + c_2x^2 + c_1x + c_0,$$

где

$$c_0 = a_0b_0$$

$$c_1 = a_1b_0 \oplus a_0b_1$$

$$c_2 = a_2b_0 \oplus a_1b_1 \oplus a_0b_2$$

$$c_3 = a_3b_0 \oplus a_2b_1 \oplus a_1b_2 \oplus a_0b_3$$

$$c_4 = a_3b_1 \oplus a_2b_2 \oplus a_1b_3$$

$$c_5 = a_3b_2 \oplus a_2b_3$$

$$c_6 = a_3b_3.$$

Для того чтобы результат умножения мог быть по-прежнему представлен 4-байтовым словом, можно взять результат по модулю многочлена степени не более 4, например

$$\varphi(x) = x^4 + 1,$$

для которого справедливо

$$x^i \bmod \varphi(x) = x^{i \bmod 4}.$$

Таким образом, результатом умножения двух многочленов

$$d(x) = a(x) \otimes b(x)$$

будет многочлен

$$d(x) = d_3x^3 + d_2x^2 + d_1x + d_0,$$

где

$$d_0 = a_0b_0 \oplus a_3b_1 \oplus a_2b_2 \oplus a_1b_3$$

$$d_1 = a_1b_0 \oplus a_0b_1 \oplus a_3b_2 \oplus a_2b_3$$

$$d_2 = a_2b_0 \oplus a_1b_1 \oplus a_0b_2 \oplus a_3b_3$$

$$d_3 = a_3b_0 \oplus a_2b_1 \oplus a_1b_2 \oplus a_0b_3.$$

В матричной форме это может быть записано следующим образом

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}.$$

На рис. 3.27 показано устройство для одновременного умножения многочлена

$$b(x) = b_3x + b_2x^2 + b_1x + b_0$$

степени не более 3 на многочлен

$$a(x) = a_3x^3 + a_2x^2 + a_1x + a_0$$

степени не более 3 и деления на многочлен 4-й степени

$$\varphi(x) = x^4 + 1.$$

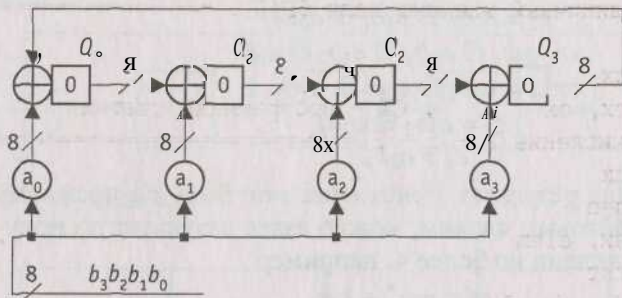


Рис. 3.27. Устройство для умножения многочлена

$$b(x) = b_3x^3 + b_2x^2 + b_1x + b_0 \text{ на многочлен } a(x) = a_3x^3 + a_2x^2 + a_1x + a_0$$

$$a_i \in GF(2^8), i = 0, 3, \text{ по модулю } \varphi(x) = x^4 + 1$$

При этом коэффициенты $b(x)$ подаются на вход старшими разрядами вперед, обратные связи устройства определяются коэффициентами $\varphi(x)$, а коэффициенты ввода входной последовательности равны коэффициентам $a(x)$.

3.7. Инверсия в $GF(L)$

В $GF(L)$ справедливо

$$\omega^{L-1} = 1,$$

а значит, ненулевые элементы ω^i и ω^j ($i \neq j$) являются взаимно обратными тогда и только тогда, когда

$$i + j = L - 1.$$

Данный факт может быть использован для осуществления инверсии в $GF(L)$.

```

;==== invgf - процедура определения =====
;==== обратного элемента в поле GF(L), L < 2^16. =====
;=====

```

```

;==== При вызове: AX - входной ненулевой =====
;==== элемент a =  $\omega^i$  поля GF(L). =====
;==== при возврате: AX =  $\omega^{L-i-1} = \alpha^{-1}$ . =====
;=====
;==== gen - один такт работы генератора элементов =====
;==== поля GF(L). При вызове gen в AX - «старое», =====
;==== при возврате в AX - «новое» состояние генератора. ==
;==== elem - единичный элемент поля GF(L). =====
invgf PROC
    push    cx
    xor     cx, cx          ; CX - программный счетчик
    ;==== Вычисление L - i - 1 =====
fst:    inc     cx
        call    gen
        cmp     ax, elem
        jnz     fst
        ; CX = L - i - 1
        ;==== Вычисление  $\omega^L$  =====
scnd:   call    gen
        loop    scnd
        pop     cx
        ; AX =  $\omega^{L-i-1}$ 
invgf   ENDP
;=====

```

Литература к главе 3

1. Алексеев А. И., Шереметьев А. Г., Тузов Г. И., Глазов В. И. Теория и применение псевдослучайных сигналов. - М.: Наука, 1969.
2. Аршинов М. Н., Садовский Л. Е. Коды и математика (рассказы о кодировании). - М.: Наука. Главн. ред. физ.- мат. лит., 1983.
3. Болл У., Коксетер Г. Математические эссе и развлечения: Пер с англ. / Под ред. И.М.Яглома. - М.: Мир, 1986.
4. Бурдаев О. В., Иванов М. А., Тетерин И. И. Ассемблер в задачах защиты информации / Под ред. И. Ю. Жукова. - М.: КУДИЦ-ОБРАЗ, 2002.
5. Макуильямс Ф. Дж., Слоан Н. Дж. А. Псевдослучайные последовательно-сти и таблицы. // ТИИЭР. - 1976. № 12. С. 80-95.
6. Питерсон У., Уэлдон Э. Коды, исправляющие ошибки. - М.: Мир, 1976.
7. Элспас Б. Теория автономных линейных последовательных сетей // Кибернетический сборник. - 1963. Вып. 7. С. 90-128.

Приложение 1

Пример процедуры разворачивания ключа

Пусть $N_k = 4$ и ключ шифра имеет вид

2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c,

т. е.

$w_0 = 2b7e1516$, $w_1 = 28aed2a6$, $w_2 = abf71588$, $w_3 = 09cf4f3c$.

Значения, формируемые в процессе разворачивания ключа, в шестнадцатеричном формате приведены в табл. П1.1, где i - индекс столбца.

Таблица. П1.1. Значения, формируемые в процессе выполнения процедуры Key Expansion

i	temp	После RotWord	После SubBytes	Rcon [i/Nk]	После XOR с Rcon	w[i-Nk]	w[i]
4	09cf4f3c	cf4f3c09	8a84eb01	01000000	8b84eb01	2b7e1516	a0fafe17
5	a0fafe17					28aed2a6	88542cb1
6	88542cb1					abf71588	23a33939
7	23a33939					09cf4f3c	2a6c7605
8	2a6c7605	6c76052a	50386be5	02000000	52386be5	a0fafe17	f2c295f2
9	f2c295f2					88542cb1	7a96b943
10	7a96b943					23a33939	5935807a
11	5935807a					2a6c7605	7359f67f
12	7359f67f	59f67f73	cb42d28f	04000000	cf42d28f	f2c295f2	3d80477d
13	3d80477d					7a96b943	4716fe3e
14	4716fe3e					5935807a	1e237e44
15	1e237e44					7359f67f	6d7a883b
16	6d7a883b	7a883b6d	dac4e23c	08000000	d2c4e23c	3d80477d	ef44a541
17	ef44a541					4716fe3e	a8525b7f
18	a8525b7f					1e237e44	b671253b
19	b671253b					6d7a883b	db0bad00
20	db0bad00	0bad00db	2b9563b9	10000000	3b9563b9	ef44a541	d4d1c6f8
21	d4d1c6f8					a8525b7f	7c839d87
22	7c839d87					b671253b	caf2b8bc

i	temp	После RotWord	После SubBytes	Rcon [i/Nk]	После XOR с Rcon	w[i-Nk]	w[i]
23	caf2b8bc					db0bad00	11f915bc
24	11f915bc	f915bc11	99596582	20000000	b9596582	d4d1c6f8	6d88a37a
25	6d88a37a					7c839d87	110b3efd
26	110b3efd					caf2b8bc	dbf98641
27	dbf98641					11f915bc	ca0093fd
28	ca0093fd	0093fdca	63dc5474	40000000	23dc5474	6d88a37a	4e54f70e
29	4e54f70e					110b3efd	5f5fc9f3
30	5f5fc9f3					dbf98641	84a64fb2
31	84a64fb2					ca0093fd	4ea6dc4f
32	4ea6dc4f	a6dc4f4e	2486842f	80000000	a486842f	4e54f70e	ead27321
33	ead27321					5f5fc9f3	b58dbad2
34	b58dbad2					84a64fb2	312bf560
35	312bf560					4ea6dc4f	7f8d292f
36	7f8d292f	8d2927ff	5da515d2	1b000000	46a515d2	ead27321	ac7766f3
37	ac7766f3					b58dbad2	19fadc21
38	19fadc21					312bf560	28d12941
39	28d12941					7f8d292f	575c006e
40	575c006e	5c006e57	4a639f5b	36000000	7c639f5b	ac7766f3	d014f9a8
41	d014f9a8					19fadc21	c9ee2589
42	c9ee2589					28d12941	e13f0cc8
43	e13f0cc8					575c006e	b6630ca6

Приложение 2

Пример работы криптоалгоритма *AES-128*

Рассматриваются последовательные состояния (рис. П2.1 – П2.4) при зашифровании для случая, когда входной блок равен 32 43 f6 a8 88 5a 30 8d 31 31 98 a2 e0 37 07 34,

а ключ шифра имеет вид

2b 7e 15 16 28 ae d2 ab И 15 88 09 cf 4f 3c.

Значения раундовых ключей, формируемые в процессе выполнения процедуры Key Expansion, приведены в приложении 1.

Номер раунда	Начало раунда	После SubBytes	После ShiftRows	После MixColumns	Раундовый ключ
Входной блок	32 88 31 e0				2b 28 ab 09
	43 5a 31 37				7e ae f7 cf
	K 30 98 07				15 d2 15 4f
	a8 8d a2 34				16 a6 88 3c
					⊕ =
1	19 a0 9a e9	d4 e0 b8 1e	d4 e0 b8 1e	04 e0 48 28	a0 88 23 2a
	3d f4 c6 f8	27 bf b4 41	bf b4 41 27	66 cb f8 06	fa 54 a3 6c
	e3 e2 8d 48	11 98 5d 52	5d 52 11 98	81 19 d3 26	fe 2c 39 76
	be 2b 2a 08	ae f1 e5 30	30 ae f1 e5	e5 9a 7a 4c	17 b1 39 05
					⊕ =
2	a4 68 6b 02	49 45 7f 77	49 45 7f 77	58 1b db 1b	f2 7a 59 73
	9c 9f 5b 6a	de db 39 02	db 39 02 de	4d 4b e7 6b	c2 96 35 59
	7f 35 ea 50	d2 96 87 53	87 53 d2 96	ca 5a ca b0	95 b9 80 f6
	f2 2b 43 49	89 f1 1a 3b	3b 89 f1 1a	f1 ac a8 e5	f2 43 7a 7f
					⊕ =

Рис. П2.1. Последовательные значения блоков *State* при зашифровании с использованием *AES-128* (начало)

Номер раунда	Начало раунда	После SubBytes	После ShiftRows	После MixColumns	Раундовый ключ
3	aa 61 82 68	ac ef 13 45	ac ef 13 45	75 20 53 bb	3d 47 1e 6d
	8f dd d2 32	73 c1 b5 23	c1 b5 23 73	ec 0b c0 25	80 16 23 7a
	5f e3 4a 46	cf 11 d6 5a	d6 5a cf 11	09 63 cf d0	47 fe 7e 88
	03 ef d2 9a	7b df b5 b8	b8 7b df b5	93 33 7c dc	7d 3e 44 3b
					$\oplus$ =
4	48 67 4d d6	52 85 e3 M	52 85 e3 K	0f 60 6f 5e	ef a8 b6 db
	6c 1d e3 5f	50 a4 11 cf	a4 11 cf 50	d6 31 c0 b3	44 52 71 0b
	4e 9d b1 58	2f 5e c8 6a	c8 6a 2f 5e	da 38 10 13	a5 5b 25 ad
	ee 0d 38 e7	28 d7 07 94	94 28 d7 07	a9 bf 6b 01	41 7f 3b 00
					$\oplus$ =
5	e0 c8 d9 85	e1 e8 35 97	e1 e8 35 97	25 bd b6 4c	d4 7c ca 11
	92 63 b1 b8	4f fb c8 5c	fb c8 6c 4f	d1 11 3a 4c	d1 83 f2 f9
	7f 63 35 be	d2 fb 96 ae	96 ae d2 fb	a9 d1 33 c0	c6 9d b8 15
	e8 c0 50 01	9b ba 53 7c	7c 9b ba 53	ad 68 8e b0	f8 87 bc bc
					$\oplus$ =

Рис. П2.2. Последовательные значения блоков *State* при зашифровании с использованием *AES-128* (продолжение)

Номер раунда	Начало раунда	После SubBytes	После ShiftRows	После MixColumns	Раундовый ключ
6	f1 c1 7c 5d	a1 78 10 4c	a1 78 10 4c	4b 2c 33 37	6d 11 db ca
	00 92 c8 b5	63 4f e8 d5	4f e8 d5 63	86 4a 9d d2	88 0b f9 00
	6f 4c 8b d5	a8 29 3d 03	3d 03 a8 29	8d 89 f4 18	a3 3e 86 93
	55 ef 32 0c	fc df 23 fe	fe fc df 23	6d 80 e8 d8	7a fd 41 fd
					$\oplus$ =
7	26 3d e8 fd	f7 27 9b 54	f7 27 9b 54	14 46 27 34	4e 5f 84 4e
	0e 41 64 d2	ab 83 43 b5	83 43 b5 ab	15 16 46 2a	54 5f a6 a6
	2e b7 72 8b	31 a9 40 3d	40 3d 31 a9	b5 15 56 d8	f7 c9 4f dc
	17 7d a9 25	f0 ff d3 3f	3f f0 ff d3	bf ec d7 43	0e f3 b2 4f
					$\oplus$ =
8	5a 19 a3 7a	be d4 0a da	be d4 0a da	00 b1 54 fa	ea b5 31 7f
	41 49 e0 8c	83 3b e1 64	3b e1 64 83	51 c8 76 1b	d2 8d 2b 8d
	42 dc 19 04	2c 86 d4 f2	d4 f2 2c 86	2f 89 6d 99	73 ba f5 29
	b1 1f 65 0c	c8 c0 4d fe	fe c8 c0 4d	d1 ff cd ea	21 d2 60 2f
					$\oplus$ =

Рис. П2.3. Последовательные значения блоков *State* при зашифровании с использованием *AES-128* (продолжение)

Рис. П2.4. Последовательные значения блоков *State* при зашифровании с использованием *AES-128* (окончание)

Приложение 3

Тестовые примеры

Данное приложение содержит примеры реализации криптоалгоритма *AES* при разрядности ключа, равной 128, 192 и 256 битам ($Nk = 4, 6, \text{ и } 8$). При этом рассматриваются процедуры зашифрования (Cipher), расшифрования (Inverse Cipher) и эквивалентного расшифрования (Equivalent Inverse Cipher).

Все тестовые вектора представлены в шестнадцатеричном формате.

Обозначения.

- * CIPHER (зашифрование) (номера раундов от $r = 0$ до 10, 12 или 14):

input: входной блок;
start: состояние перед началом раунда с номером r ;
s_box: состояние после SubBytes();
s_row: состояние после ShiftRows();
m_col: состояние после MixColumns();
k_sch: значение раундового ключа для раунда с номером r ;
output: выходной блок.

- в INVERSE CIPHER (расшифрование) (номера раундов от $r = 0$ до 10, 12 или 14):

iinput: входной блок;
istart: состояние перед началом раунда с номером r ;
is_box: состояние после InvSubBytes();
is_row: состояние после InvShiftRows();
ik_sch: значение раундового ключа для раунда с номером r ;
ik_add: состояние после AddRoundKey();
ioutput: выходной блок.

- * EQUIVALENT INVERSE CIPHER (расшифрование) (номера раундов от $r = 0$ до 10, 12 или 14):

iinput: входной блок;
istart: состояние перед началом раунда с номером r ;
is_box: состояние после InvSubBytes();

is_row: состояние после InvShiftRows();
 im_col: состояние после InvMixColumns()
 ik_sch: значение раундового ключа для раунда с номером r ;
 ioutput: выходной блок.

3.1. AES-128 ($N_k = 4, N_r = 10$)

PLAINTEXT: 00112233445566778899aabbccddeeff
 KEY: 000102030405060708090a0b0c0d0e0f

CIPHER (зашифрование):

```
round[ 0].input      00112233445566778899aabbccddeeff
round[ 0].k_sch      000102030405060708090a0b0c0d0e0f
round[ 1].start      00102030405060708090a0b0c0d0e0f0
round[ 1].s_box      63cab7040953d051cd60e0e7ba70el8c
round[ 1].s_row      6353e08c0960e104cd70b751bacad0e7
round[ 1].m_col      5f72641557f5bc92f7be3b291db9f91a
round[ 1].k_sch      d6aa74fdd2af72fadaa678fld6ab76fe
round[ 2].start      89d810e8855ace682d1843d8cbl28fe4
round[ 2].s_box      a761ca9b97be8b45d8adla611fc97369
round[ 2].s_row      a7bela6997ad739bd8c9ca451f618b61
round[ 2].m_col      ff87968431d86a51645151fa773ad009
round[ 2].k_sch      b692cf0b643dbdf1be9bc5006830b3fe
round[ 3].start      4915598f55e5d7aOdaca94fal f0a63f7
round[ 3].s_box      3b59cb73fcd90ee05774222dc067fb68
round! 3].s_row      3bd92268fc74fb735767cbe0c0590e2d
round[ 3].m_col      4c9cle66f771f0762c3f868e534df256
round[ 3].k_sch      b6ff744ed2c2c9bf6c590cbf0469bf41
round[ 4].start      fa636a2825b339c940668a3157244d17
round[ 4].s_box      2dfb02343f6d12dd09337ec75b36e3f0
round[ 4].s_row      2d6d7ef03f33e334093602dd5bfb12c7
round[ 4].m_col      6385b79ffc538df997be478e7547d691
round[ 4].k_sch      47f7f7bc95353e03f96c32bcfd058dfd
round[ 5].start      247240236966b3fa6ed2753288425b6c
round[ 5].s_box      36400926f9336d2d9fb59d23c42c3950
round[ 5].s_row      36339d50f9b539269f2c092dc4406d23
round[ 5].m_col      f4bcd45432e554d075fld6c51dd03b3c
round[ 5].k_sch      3caaa3e8a99f9deb50f3af57adf622aa
round[ 6].start      C81677bc9b7ac93b25027992b0261996
```

```

round[ 6].s_box      e847f56514dadde23f77b64fe7f7d490
round[ 6].s_row      e8dab6901477d4653ff7f5e2e747dd4f
round[ 6].m_col      9816ee7400f87f556b2c049c8e5ad036
round[ 6].k_sch      5e390f7df7a69296a7553dcl0aa31f6b
round[ 7].start      C62fel09f75eedc3cc79395d84f9cf5d
round[ 7].s_box      b415f8016858552e4bb6124c5f998a4c
round[ 7].s_row      b458124c68b68a014b99f82e5f15554c
round[ 7].m_col      C57elc159a9bd286f05f4be098c63439
round[ 7].k_sch      14f9701ae35fe28c440adf4d4ea9c026
round[ 8].start      d1876c0f79c4300ab45594add66ff41f
round[ 8].s_box      3el75076b61c04678dfc2295f6a8bfc0
round[ 8].s_row      3elc22c0b6fcbf768da85067f6170495
round[ 8].m_col      baa03de7a1f9b56ed5512cba5f414d23
round[ 8].k_sch      47438735a41c65b9e016baf4aebf7ad2
round[ 9].start      fde3bad205e5d0d73547964ef1fe37f1
round[ 9].s_box      5411f4b56bd9700e96a0902falbb9aal
round[ 9].s_row      54d990a16ba09ab596bbf40ea111702f
round[ 9].m_col      e9f74eec023020f61bf2ccf2353c21c7
round[ 9].k_sch      549932dlf08557681093ed9cbe2c974e
round[10].start      bd6e7c3df2b5779e0b61216e8b10b689
round[10].s_box      7a9f102789d5f50b2beffd9f3dca4ea7
round[10].s_row      7ad5fda789ef4e272bcal00b3d9ff59f
round[10].k_sch      13111d7fe3944a17f307a78b4d2b30c5
round[10].output     69c4e0d86a7b0430d8cdb78070b4c55a

```

INVERSE CIPHER (расшифрование):

```

round[ 0].iinput     69c4e0d86a7b0430d8cdb78070b4c55a
round[ 0].ik_sch     13111d7fe3944a17f307a78b4d2b30c5
round[ 1].istart     7ad5fda789ef4e272bcal00b3d9ff59f
round[ 1].is_row     7a9f102789d5f50b2beffd9f3dca4ea7
round[ 1].is_box     bd6e7c3df2b5779e0b61216e8b10b689
round[ 1].ik_sch     549932dlf08557681093ed9cbe2c974e
round[ 1].ik_add     e9f74eec023020f61bf2ccf2353c21c7
round[ 2].istart     54d990a16ba09ab596bbf40ea111702f
round[ 2].is_row     5411f4b56bd9700e96a0902falbb9aal
round[ 2].is_box     fde3bad205e5d0d73547964ef1fe37f1
round[ 2].ik_sch     47438735a41c65b9e016baf4aebf7ad2
round[ 2].ik_add     baa03de7a1f9b56ed5512cba5f414d23
round[ 3].istart     3e1c22c0b6fcbf768da85067f6170495
round[ 3].is_row     3el75076b61c04678dfc2295f6a8bfc0

```

```
round[ 3].isbox      d1876c0f79c4300ab45594add66ff41f
round[ 3].ik_sch     14f9701ae35fe28c440adf4d4ea9c026
round[ 3].ik_add     C57elc159a9bd286f05f4be098c63439
round[ 4].istart     b458124c68b68a014b99f82e5f15554c
round[ 4].is_row     b415f8016858552e4bb6124c5f998a4c
round[ 4].is_box     C62fel09f75eedc3cc79395d84f9cf5d
round[ 4].ik_sch     5e390f7df7a69296a7553dcl0aa31f6b
round[ 4].ik_add     9816ee7400f87f556b2c049c8e5ad036
round[ 5].istart     e8dab6901477d4653ff7f5e2e747dd4f
round[ 5].is_row     e847f56514dadde23f77b64fe7f7d490
round[ 5].is_box     C81677bc9b7ac93b25027992b0261996
round[ 5].ik_sch     3caaa3e8a99f9deb50f3af57adf622aa
round[ 5].ik_add     f4bcd45432e554d075fld6c51dd03b3c
round[ 6].istart     36339d50f9b539269f2c092dc4406d23
round[ 6].is_row     36400926f9336d2d9fb59d23c42c3950
round[ 6].is_box     247240236966b3fa6ed2753288425b6c
round[ 6].ik_sch     47f7f7bc95353e03f96c32bcfd058dfd
round[ 6].ik_add     6385b79ffc538df997be478e7547d691
round[ 7].istart     2d6d7ef03f33e334093602dd5bfb12c7
round[ 7].is_row     2dfb02343f6d12dd09337ec75b36e3f0
round[ 7].is_box     fa636a2825b339c940668a3157244d17
round[ 7].ik_sch     b6ff744ed2c2c9bf6c590cbf0469bf41
round[ 7].ik_add     4c9c1e66f771f0762c3f868e534df256
round[ 8].istart     3bd92268fc74fb735767cbe0c0590e2d
round[ 8].is_row     3b59cb73fcd90ee05774222dc067fb68
round[ 8].is_box     4915598f55e5d7a0daca94fa1f0a63f7
round[ 8].ik_sch     b692cf0b643dbdf1be9bc5006830b3fe
round[ 8].ik_add     ff87968431d86a51645151fa773ad009
round[ 9].istart     a7bela6997ad739bd8c9ca451f618b61
round[ 9].is_row     a761ca9b97be8b45d8adla611fc97369
round[ 9].is_box     89d810e8855ace682d1843d8cb128fe4
round[ 9].ik_sch     d6aa74fdd2af72fadaa678fld6ab76fe
round[ 9].ik_add     5f72641557f5bc92f7be3b291db9f91a
round[10].istart     6353e08c0960e104cd70b751bacad0e7
round[10].is_row     63cab7040953d051cd60e0e7ba70e18c
round[10].is_box     00102030405060708090a0b0c0d0e0f0
round[10].ik_sch     000102030405060708090a0b0c0d0e0f
round[10].ioutput    00112233445566778899aabbccddeeff
```


EQUIVALENT INVERSE CIPHER (расшифрование):

```

round[ 0].iinput      69c4e0d86a7b0430d8cdb78070b4c55a
round[ 0].ik_sch      13111d7fe3944a17f307a78b4d2b30c5
round[ 1].istart      7ad5fda789ef4e272bcal00b3d9ff59f
roundt 1].is_box      bdb52189f261b63d0b107c9e8b6e776e
round[ 1].is_row      bd6e7c3df2b5779e0b61216e8b10b689
roundt 1].im_col      4773b91ff72f354361cb018eale6cf2c
round[ 1].ik_sch      13aa29be9c8faff6f770f58000f7bf03
round[ 2].istart      54d990a16ba09ab596bbf40ea111702f
round[ 2].is_box      fde596f1054737d235febad7f1e3d04e
roundt 2].is_row      fde3bad205e5d0d73547964ef1fe37f1
round[ 2].im_col      2d7e86a339d9393ee6570a1101904e16
roundt 2].ik_sch      1362a4638f2586486bff5a76f7874a83
round[ 3].istart      3elc22c0b6fcbf768da85067f6170495
round[ 3].is_box      dlc4941f7955f40fb46f6c0ad68730ad
roundt 3].is_row      d1876c0f79c4300ab45594add66ff41f
round[ 3].im_col      39daee38f4fla82aaf432410c36d45b9
round[ 3].ik_sch      8d82fc749c47222be4dad3e9c7810f5
round[ 4].istart      b458124c68b68a014b99f82e5f15554c
round[ 4].is_box      c65e395df779cf09ccf9e1c3842fed5d
round[ 4].is_row      c62fel09f75eedc3cc79395d84f9cf5d
round[ 4].im_col      9a39bfl05b20a3a476a0bf79fe51184
round[ 4].ik_sch      72e3098d11c5de5f789dfe1578a2cccb
round[ 5].istart      e8dab6901477d4653ff7f5e2e747dd4f
round[ 5].is_box      c87a79969b0219bc2526773bb016c992
roundt 5].is_row      C81677bc9b7ac93b25027992b0261996
roundt 5].im_col      18f78d779a93eef4f6742967c47f5ffd
round[ 5].ik_sch      2ec410276326d7d26958204a003f32de
roundt 6].istart      36339d50f9b539269f2c092dc4406d23
round[ 6].is_box      2466756c69d25b236e4240fa8872b332
round[ 6].is_row      247240236966b3fa6ed2753288425b6c
round[ 6].im_col      85cf8bf472d124c10348f545329c0053
roundt 6].ik_sch      a8a2f5044de2c7f50a7ef79869671294
round[ 7].istart      2d6d7ef03f33e334093602dd5bfb12c7
round[ 7].is_box      fab38a1725664d2840246ac957633931
roundt 7].is_row      fa636a2825b339c940668a3157244d17
round[ 7].im_col      fc1fc1f91934c98210fbfb8da340eb21
round[ 7].ik_sch      C7c6e391e54032f1479c306d6319e50c
round[ 8].istart      3bd92268fc74fb735767cbe0c0590e2d
round[ 8].is_box      49e594f755ca638fda0a59a01fl5d7fa
round[ 8].is_row      4915598f55e5d7a0daca94fal0a63f7
round[ 8].im_col      076518f0b52ba2fb7a15c8d93be45e00

```

```

round[ 8].iksch a0db02992286d160a2dc029c2485d561
round[ 9].istart a7bela6997ad739bd8c9ca451f618b61
round[ 9].isbox 895a43e485188fe82d121068cbd8ced8
round[ 9].isrow 89d810e8855ace682d1843d8cbl28fe4
round[ 9].imcol ef053f7c8b3d32fd4d2a64ad3c93071a
round[ 9].iksch 8c56dff0825dd3f9805ad3fc8659d7fd
round[10].istart 6353e08c0960e104cd70b751bacad0e7
round[10].isbox 0050a0f04090e03080d02070c01060b0
round[10].isrow 00102030405060708090a0b0c0d0e0f0
round[10].iksch 000102030405060708090a0b0c0d0e0f
round[10].ioutput 00112233445566778899aabbccddeeff

```

3.2. AES-192 ($N_k = 6$, $N_r = 12$)

```

PLAINTEXT: 00112233445566778899aabbccddeeff
KEY: 000102030405060708090a0b0c0d0e0f1011121314151617

```

CIPHER (зашифрование):

```

round[ 0].input 00112233445566778899aabbccddeeff
round[ 0].k_sch 000102030405060708090a0b0c0d0e0f
round[ 1].start 00102030405060708090a0b0c0d0e0f0
round[ 1].s_box 63cab7040953d051cd60e0e7ba70e18c
round[ 1].s_row 6353e08c0960e104cd70b751bacad0e7
round[ 1].m_col 5f72641557f5bc92f7be3b291db9f91a
round[ 1].k_sch 10111213141516175846f2f95c43f4fe
round[ 2].start 4f63760643e0aa85aff8c9d041fa0de4
round[ 2].s_box 84fb386f1ae1ac977941dd70832dd769
round[ 2].s_row 84e1dd691a1d76f792d389783fbac70
round[ 2].m_col 9f487f794f955f662afc86abd7flab29
round[ 2].k_sch 544afef55847f0fa4856e2e95c43f4fe
round[ 3].start Cb02818cl7d2af9c62aa64428bb25fd7
round[ 3].s_box 1f770c64f0b579deaaac432c3d37cf0e
round[ 3].s_row 1fb5430ef0accf64aa370cde3d77792c
round[ 3].m_col b7a53ecbbf9d75a0c40efc79b674ccll
round[ 3].k_sch 40f949b31cbabd4d48f043b810b7b342
round[ 4].start f75c7778a327c8ed8cfefbfc1a6c37f53
round[ 4].s_box 684af5bc0acce85564bb0878242ed2ed
round[ 4].s_row 68cc08ed0abbd2bc642ef555244ae878
round[ 4].m_col 7ale98bdacb6d1141a6944dd06eb2d3e
round[ 4].k_sch 58e151ab04a2a5557effb5416245080c

```

```
round[ 5].start 22ffc916a81474416496f19c64ae2532
round[ 5].s_box 9316dd47c2fa92834390a1de43e43f23
round[ 5].s_row 93faal23c2903f4743e4dd83431692de
round[ 5].m_col aaa755b34cffe57cef6f98e1f01c13e6
round[ 5].k_sch 2ab54bb43a02f8f662e3a95d66410c08
round[ 6].start 80121e0776fdld8a8d8c31bc965dlfee
round[ 6].s_box Cdc972c53854a47e5d64c765904cc028
round[ 6].s_row Cd54c7283864c0c55d4c727e90c9a465
round[ 6].m_col 921f748fd96e937d622d7725ba8ba50c
round[ 6].k_sch f501857297448d7ebdf1c6ca87f33e3c
round[ 7].start 671ef1fd4e2ale03dfdcblf3d789b30
round[ 7].s_box 8572al542fe5727b9e86c8df27bcl404
round[ 7].s_row 85e5c8042f8614549ebcal7b277272df
round[ 7].m_col e913e7b18f507d4b227ef652758acbcc
round[ 7].k_sch e510976183519b6934157c9ea351fle0
round[ 8].start 0c0370d00c01e622166b8accd6db3a2c
round[ 8].s_box fe7b5170fe7c8e93477f7e4bf6b98071
round[ 8].s_row fe7c7e71fe7f807047b95193f67b8e4b
round[ 8].m_col 6cf5edf996eb0a069c4ef21cbfc25762
round[ 8].k_sch 1ea0372a995309167c439e77ff12051e
round[ 9].start 7255dad30fb80310e00d6c6b40d0527c
round[ 9].s_box 40fc5766766c7bcae1d7507f09700010
round[ 9].s_row 406c501076d70066e17057ca09fc7b7f
round[ 9].m_col 7478bcdce8a50b81d4327a9009188262
round[ 9].k_sch dd7e0e887e2fff68608fc842f9dcc154
round[10].start a906b254968af4e9b4bdb2d2f0c44336
round[10].s_box d36f3720907ebf1e8d7a37b58clcla05
round[10].s_row d37e3705907ala208dlc371e8c6fbfb5
round[10].m_col Od73cc2d8f6abe8b0cf2dd9bb83d422e
round[10].k_sch 859f5f237a8d5a3dc0c02952beefd63a
round[11].start 88ec930ef5e7e4b6cc32f4c906d29414
round[11].s_box c4cedcabe694694e4b23bfdd6fb522fa
round[11].s_row C494bffae62322ab4bb5dc4e6fce69dd
round[11].m_col 71d720933b6d677dc00b8f28238e0fb7
round[11].k_sch de601e7827bcdff2ca223800fd8aeda32
round[12].start afb73eeblcd1b85162280f27fb20d585
round[12].s_box 79a9b2e99c3e6cdlaa3476cc0fb70397
round[12].s_row 793e76979c3403e9aab7b2dl0fa96ccc
round[12].k_sch a4970a331a78dc09c418c271e3a41d5d
round[12].output dda97ca4864cdf0e6eaf70a0ec0d7191
```


INVERSE CIPHER (расшифрование) :

```

round[ 0] .iinput  dda97ca4864cdfe06eaf70a0ec0d7191
round[ 0] .ik_sch  a4970a331a78dc09c418c271e3a41d5d
round[ 1] .istart  793e76979c3403e9aab7b2d10fa96ccc
round[ 1] .is_row  79a9b2e99c3e6cdlaa3476cc0fb70397
round[ 1] .is_box  afb73eeblcd1b85162280f27fb20d585
round[ 1] .ik_sch  de601e7827bcdcf2ca223800fd8aeda32
round[ 1] .ik_add  71d720933b6d677dc00b8f28238e0fb7
round[ 2] .istart  C494bffaef62322ab4bb5dc4e6fce69dd
round[ 2] .is_row  C4cedcabe694694e4b23bfdd6fb522fa
round[ 2] .is_box  88ec930ef5e7e4b6cc32f4c906d29414
round[ 2] .ik_sch  859f5f237a8d5a3dc0c02952beefd63a
round[ 2] .ik_add  Od73cc2d8f6abe8b0cf2dd9bb83d422e
round[ 3] .istart  d37e3705907ala208dlc371e8c6fbfb5
round[ 3] .is_row  d36f3720907ebf1e8d7a37b58clcla05
round[ 3] .is_box  a906b254968af4e9b4bdb2d2f0c44336
round[ 3] .ik_sch  dd7e0e887e2fff68608fc842f9dcc154
round[ 3] .ik_add  7478bcdce8a50b81d4327a9009188262
round[ 4] .istart  406c501076d70066e17057ca09fc7b7f
round[ 4] .is_row  40fc5766766c7bcae1d7507f09700010
round[ 4] .is_box  7255dad30fb80310e00d6c6b40d0527c
round[ 4] .ik_sch  1ea0372a995309167c439e77ff12051e
round[ 4] .ik_add  6cf5edf996eb0a069c4ef21cbfc25762
round[ 5] .istart  fe7c7e71fe7f807047b95193f67b8e4b
round[ 5] .is_row  fe7b5170fe7c8e93477f7e4bf6b98071
round[ 5] .is_box  Oc0370d00c01e622166b8accd6db3a2c
round[ 5] .ik_sch  e510976183519b6934157c9ea351fle0
round[ 5] .ik_add  e913e7b18f507d4b227ef652758acbcc
round[ 6] .istart  85e5c8042f8614549ebca17b277272df
round[ 6] .is_row  8572a1542fe5727b9e86c8df27bc1404
round[ 6] .is_box  671ef1fd4e2a1e03dfdcblf3d789b30
round[ 6] .ik_sch  f501857297448d7ebdf1c6ca87f33e3c
round[ 6] .ik_add  921f748fd96e937d622d7725ba8ba50c
round[ 7] .istart  Cd54c7283864c0c55d4c727e90c9a465
round[ 7] .is_row  Cdc972c53854a47e5d64c765904cc028
round[ 7] .is_box  80121e0776fd1d8a8d8c31bc965dlfee
round[ 7] .ik_sch  2ab54bb43a02f8f662e3a95d66410c08
round[ 7] .ik_add  aaa755b34cffe57cef6f98e1f01c13e6
round[ 8] .istart  93faal23c2903f4743e4dd83431692de
round[ 8] .is_row  9316dd47c2fa92834390a1de43e43f23
round[ 8] .is_box  22ffc916a81474416496f19c64ae2532
round[ 8] .ik_sch  58e151ab04a2a5557effb5416245080c
round[ 8] .ik_add  7ale98bdacb6dl141a6944dd06eb2d3e

```

```

round[ 9].istart 68cc08ed0abbd2bc642ef555244ae878
round[ 9].is_row 684af5bc0acce85564bb0878242ed2ed
round[ 9].is_box f75c7778a327c8ed8cfebfclac637f53
round[ 9].ik_sch 40f949b31cbabd4d48f043b810b7b342
round[ 9].ik_add b7a53ecbbf9d75a0c40efc79b674cc11
round[10].istart 1fb5430ef0accf64aa370cde3d77792c
round[10].is_row 1f770c64f0b579deaaac432c3d37cf0e
round[10].is_box cb02818c17d2af9c62aa64428bb25fd7
round[10].ik_sch 544afef55847f0fa4856e2e95c43f4fe
round[10].ik_add 9f487f794f955f662afc86abd7f1ab29
round[11].istart 84eldd691a41d76f792d389783fbac70
round[11].is_row 84fb386flaelac977941dd70832dd769
round[11].is_box 4f63760643e0aa85aff8c9d041fa0de4
round[11].ik_sch 10111213141516175846f2f95c43f4fe
round[11].ik_add 5f72641557f5bc92f7be3b291db9f91a
round[12].istart 6353e08c0960e104cd70b751bacad0e7
round[12].is_row 63cab7040953d051cd60e0e7ba70e18c
round[12].is_box 00102030405060708090a0b0c0d0e0f0
round[12].ik_sch 000102030405060708090a0b0c0d0e0f
round[12].ioutput 00112233445566778899aabbccddeeff

```

EQUIVALENT INVERSE CIPHER (расшифрование):

```

round[ 0].iinput dda97ca4864cdfe06eaf70a0ec0d7191
round[ 0].ik_sch a4970a331a78dc09c418c271e3a41d5d
round[ 1].istart 793e76979c3403e9aab7b2d10fa96ccc
round[ 1].is_box afd10f851c28d5eb62203e51fbb7b827
round[ 1].is_row afb73eeblcd1b85162280f27fb20d585
round[ 1].im_col 122a02f7242ac8e20605afce51cc7264
round[ 1].ik_sch d6bebd0dc209ea494db073803e021bb9
round[ 2].istart c494bffaef62322ab4bb5dc4e6fce69dd
round[ 2].is_box 88e7f414f532940eccd293b606ece4c9
round[ 2].is_row 88ec930ef5e7e4b6cc32f4c906d29414
round[ 2].im_col 5cc7aecce3c872194ae5ef8309a933c7
round[ 2].ik_sch 8fb999c973b26839c7f9d89d85c68c72
round[ 3].istart d37e3705907ala208dlc371e8c6fbfb5
round[ 3].is_box a98ab23696bd4354b4c4b2e9f006f4d2
round[ 3].is_row a906b254968af4e9b4bdb2d2f0c44336
round[ 3].im_col b7113ed134e85489b20866b51d4b2c3b
round[ 3].ik_sch f77d6ec1423f54ef5378317f14b75744
round[ 4].istart 406c501076d70066el7057ca09fc7b7f
round[ 4].is_box 72b86c7c0f0d52d3e0d0dal04055036b
round[ 4].is_row 7255dad30fb80310e00d6c6b40d0527c

```

```

round[ 4] .im_col    ef3b1belb9b0e64bdcb79f1e0a707fbb
round[ 4] .ik_sch    1147659047cf663b9b0ece8dfc0bf1f0
round[ 5] .istart    fe7c7e71fe7f807047b95193f67b8e4b
round[ 5] .is_box    0c018a2c0c6b3ad016db7022d603e6cc
round[ 5] .is_row    0c0370d00c01e622166b8accd6db3a2c
round[ 5] .im_col    592460b248832b2952e0b831923048f1
round[ 5] .ik_sch    dcc1a8b667053f7dcc5c194ab5423a2e
round[ 6] .istart    85e5c8042f8614549ebcal7b277272df
round[ 6] .is_box    672abl304edc9bfddf78f1033dleleef
round[ 6] .is_row    671ef1fd4e2a1e03dfdcbl1ef3d789b30
round[ 6] .im_col    0b8a7783417ae3alf9492dc0c641a7ce
round[ 6] .ik_sch    C6deb0ab791e2364a4055f5be568803ab
round[ 7] .istart    Cd54c7283864c0c55d4c727e90c9a465
round[ 7] .is_box    80fd31ee768clf078d5dle8a96121dbc
round[ 7] .is_row    80121e0776fd1d8a8d8c31bc965d1fee
round[ 7] .im_col    4ee1ddf9301d6352c9ad769ef8d20515
round[ 7] .ik_sch    dd1b7cdaf28d5c158a49ab1dbbc497cb
round[ 8] .istart    93faal23c2903f4743e4dd83431692de
round[ 8] .is_box    2214f132a896251664aec94164ff749c
round[ 8] .is_row    22ffc916a81474416496f19c64ae2532
round[ 8] .im_col    1008ffe53b36ee6af27b42549b8a7bb7
round[ 8] .ik_sch    78c4f708318d3cd69655b701bfc093cf
round[ 9] .istart    68cc08ed0abb2bc642ef555244ae878
round[ 9] .is_box    f727bf53a3fe7f788cc377eda65cc8c1
round[ 9] .is_row    f75c7778a327c8ed8cfebfcl1a6c37f53
round[ 9] .im_col    7f69acled939ebaac8ece3cbl2el59e3
round[ 9] .ik_sch    60dcef10299524ce62dbef152f9620cf
round[10] .istart    1fb5430ef0accf64aa370cde3d77792c
round[10] .is_box    cbd264d717aa5f8c62b2819c8b02af42
round[10] .is_row    Cb02818cl7d2af9c62aa64428bb25fd7
round[10] .im_col    Cfaf16b2570cl8b52e7fef50cab267ae
round[10] .ik_sch    4b4ecbdb4d4dcfda5752d7c7f4949cbde
round[11] .istart    84e1dd691a41d76f792d389783fbac70
round[11] .is_box    4fe0c9e443f80d06affa76854163aad0
round[11] .is_row    4f63760643e0aa85aff8c9d041fa0de4
round[11] .im_col    794cf891177bfdld8a327086f3831b39
round[11] .ik_sch    1alf181d1elb1c194742c7d74949cbde
round[12] .istart    6353e08c0960el04cd70b751bacad0e7
round[12] .is_box    0050a0f04090e03080d02070c01060b0
round[12] .is_row    00102030405060708090a0b0c0d0e0f0
round[12] .ik_sch    000102030405060708090a0b0c0d0e0f
round[12] .ioutput   00112233445566778899aabbccddeeff

```


3.3. AES-256 ($N_k = 8, N_r = 14$)

PLAINTEXT: 00112233445566778899aabbccddeeff
 KEY: 000102030405060708090a0b0c0d0e0f101112131415
 161718191a1b1c1d1e1f

CIPHER (зашифрование):

```
round[ 0].input      00112233445566778899aabbccddeeff
round[ 0].k_sch      000102030405060708090a0b0c0d0e0f
round[ 1].start      00102030405060708090a0b0c0d0e0f0
round[ 1].s_box       63cab7040953d051cd60e0e7ba70e18c
round[ 1].s_row       6353e08c0960e104cd70b751bacad0e7
round[ 1].m_col       5f72641557f5bc92f7be3b291db9f91a
round[ 1].k_sch      101112131415161718191a1b1c1d1e1f
round[ 2].start      4f63760643e0aa85efa7213201a4e705
round[ 2].s_box       84fb386flaelac97df5cfd237c49946b
round[ 2].s_row       84elfd6bla5c946fdf4938977cfbac23
round[ 2].m_col       bd2a395d2b6ac438d192443e615da195
round[ 2].k_sch      a573c29fa176c498a97fce93a572c09c
round[ 3].start      1859fbc28alc00a078ed8aadca42f6109
round[ 3].s_box       adcb0f257e9c63e0bc557e951c15ef01
round[ 3].s_row       ad9c7e017e55ef25bc150fe01ccb6395
round[ 3].m_col       810dce0cc9db8172b3678c1e88alb5bd
round[ 3].k_sch      1651a8cd0244bedala5da4c10640bade
round[ 4].start      975c66c1cb9f3fa8a93a28df8ee10f63
round[ 4].s_box       884a33781fdb75c2d380349e19f876fb
round[ 4].s_row       88db34fb1f807678d3f833c2194a759e
round[ 4].m_col       b2822d81abe6fb275faf103a078c0033
round[ 4].k_sch      ae87dff00ff11b68a68ed5fb03fc1567
round[ 5].start      1c05f271a417e04ff921c5c104701554
round[ 5].s_box       9c6b89a349f0e18499fda678f2515920
round[ 5].s_row       9cf0a62049fd59a399518984f26bel78
round[ 5].m_col       aeb65ba974e0f822d73f567bdb64c877
round[ 5].k_sch      6delf1486fa54f9275f8eb5373b8518d
round[ 6].start      C357aaell1b45b7b0a2c7bd28a8dc99fa
round[ 6].s_box       2e5bacf8af6ea9e73ac67a34c286ee2d
round[ 6].s_row       2e6e7a2dafc6eef83a86ace7c25ba934
round[ 6].m_col       b951c33c02e9bd29ae25cdb1efa08cc7
round[ 6].k_sch      C656827fc9a799176f294cec6cd5598b
round[ 7].start      7f074143cb4e243ec10c815d8375d54c
```

```

round [ 7] .s_box      d2c5831a1f2f36b278fe0c4cec9d0329
round [ 7] .s_row      d22f0c291ffe031a789d83b2ecc5364c
round [ 7] .m_col      ebb19e1c3ee7c9e87d7535e9ed6b9144
round [ 7] .k_sch      3de23a75524775e727bf9eb45407cf39
round [ 8] .start      d653a4696ca0bc0f5acaab5db96c5e7d
round [ 8] .s_box      f6ed49f950e06576be74624c565058ff
round [ 8] .s_row      f6e062ff507458f9be50497656ed654c
round [ 8] .m_col      5174c8669da98435a8b3e62ca974a5ea
round [ 8] .k_sch      0bdc905fc27b0948ad5245a4cl871c2f
round [ 9] .start      5aa858395fd28d7d05e la38868f3b9c5
round [ 9] .s_box      bec26a12cfb55dff6bf80ac4450d56a6
round [ 9] .s_row      beb50aa6cff856126b0d6aff45c25dc4
round [ 9] .m_col      0f77ee31d2ccadc05430a83f4ef96ac3
round [ 9] .k_sch      45f5a66017b2d387300d4d33640a820a
round[10] .start      4a824851c57e7e47643de50c2af3e8c9
round[10] .s_box      d61352dla6f3f3a04327d9fee50d9bdd
round[10] .s_row      d6f3d9dda6279bd1430d52a0e513f3fe
round[10] .m_col      bd86f0ea748fc4f4630fllcle9331233
round[10] .k_sch      7ccff71cbeb4fe5413e6bbf0d261a7df
round[11] .start      Cl4907f6ca3b3aa070e9aa313b52b5ec
round[11] .s_box      783bc54274e280e0511eacc7e200d5ce
round[11] .s_row      78e2acce741ed5425100c5e0e23b80c7
round[11] .m_col      af8690415d6eldd387e5fbedd5c89013
round[11] .k_sch      f01afafee7a82979d7a5644ab3afe640
round[12] .start      5f9c6abfbac634aa50409fa766677653
round[12] .s_box      Cfde0208f4b418ac5309db5c338538ed
round[12] .s_row      Cfb4dbedf4093808538502ac33de185c
round[12] .m_col      7427fae4d8a695269ce83d315be0392b
round[12] .k_sch      2541fe719bf500258813bbd55a721c0a
round[13] .start      516604954353950314fb86e401922521
round[13] .s_box      dl33f22alaed2a7bfa0f44697c4f3ffd
round[13] .s_row      dled44fdla0f3f2afa4ff27b7c332a69
round[13] .m_col      2c21a820306fl54ab712c75eee0da04f
round[13] .k_sch      4e5a6699a9f24fe07e572baacdf8cdea
round[14] .start      627bceb9999d5aaac945ecf423f56da5
round[14] .s_box      aa218b56ee5ebeacdd6ecebf26e63c06
round[14] .s_row      aa5ece06ee6e3c56dde68bac2621bebf
round[14] .k_sch      24fc79ccb0f0979e9371ac23c6d.68de36
round[14] .output     8ea2b7ca516745bfeafc49904b496089

```

INVERSE CIPHER (расшифрование):

```
round[ 0].iinput      8ea2b7ca516745bfeafc49904b496089
round[ 0].ik_sch      24fc79ccbf0979e9371ac23c6d68de36
round[ 1].istart      aa5ece06ee6e3c56dde68bac2621bebf
round[ 1].is_row      aa218b56ee5ebeacdd6ecebf26e63c06
round[ 1].is_box      627bceb9999d5aaac945ecf423f56da5
round[ 1].ik_sch      4e5a6699a9f24fe07e572baacdf8cdea
round[ 1].ik_add      2c21a820306f154ab712c75eee0da04f
round[ 2].istart      dled44fdla0f3f2afa4ff27b7c332a69
round[ 2].is_row      d133f22a1aed2a7bfa0f44697c4f3ffd
round[ 2].is_box      516604954353950314fb86e401922521
round[ 2].ik_sch      2541fe719bf500258813bbd55a721c0a
round[ 2].ik_add      7427fae4d8a695269ce83d315be0392b
round[ 3].istart      Cfb4dbedf4093808538502ac33de185c
round[ 3].is_row      Cfde0208f4b418ac5309db5c338538ed
round[ 3].is_box      5f9c6abfbac634aa50409fa766677653
round[ 3].ik_sch      f01afafee7a82979d7a5644ab3afe640
round[ 3].ik_add      af8690415d6eldd387e5fbedd5c89013
round[ 4].istart      78e2acce741ed5425100c5e0e23b80c7
round[ 4].is_row      783bc54274e280e0511eacc7e200d5ce
round[ 4].is_box      Cl4907f6ca3b3aa070e9aa313b52b5ec
round[ 4].ik_sch      7ccff71cbeb4fe5413e6bbf0d261a7df
round[ 4].ik_add      bd86f0ea748fc4f4630f11cle9331233
round[ 5].istart      d6f3d9dda6279bd1430d52a0e513f3fe
round[ 5].is_row      d61352dla6f3f3a04327d9fee50d9bdd
round[ 5].is_box      4a824851c57e7e47643de50c2af3e8c9
round[ 5].ik_sch      45f5a66017b2d387300d4d33640a820a
round[ 5].ik_add      0f77ee31d2ccadc05430a83f4ef96ac3
round[ 6].istart      beb50aa6cff856126b0d6aff45c25dc4
round[ 6].is_row      bec26a12cfb55dff6bf80ac4450d56a6
round[ 6].is_box      5aa858395fd28d7d05ela38868f3b9c5
round[ 6].ik_sch      0bdc905fc27b0948ad5245a4c1871c2f
round[ 6].ik_add      5174c8669da98435a8b3e62ca974a5ea
round[ 7].istart      f6e062ff507458f9be50497656ed654c
round[ 7].is_row      f6ed49f950e06576be74624c565058ff
round[ 7].is_box      d653a4696ca0bc0f5acaab5db96c5e7d
round[ 7].ik_sch      3de23a75524775e727bf9eb45407cf39
round[ 7].ik_add      ebb19e1c3ee7c9e87d7535e9ed6b9144
```



```
round[ 8].istart d22f0c291ffe031a789d83b2ecc5364c
round[ 8].is_row d2c5831alf2f36b278fe0c4cec9d0329
round[ 8].is_box 7f074143cb4e243ec1Oc815d8375d54c
round[ 8].ik_sch c656827fc9a799176f294cec6cd5598b
round[ 8].ik_add b951c33c02e9bd29ae25cdblafa08cc7
round[ 9].istart 2e6e7a2dafc6eef83a86ace7c25ba934
round[ 9].is_row 2e5bacf8af6ea9e73ac67a34c286ee2d
round[ 9].is_box c357aae11b45b7b0a2c7bd28a8dc99fa
round[ 9].ik_sch 6delf1486fa54f9275f8eb5373b8518d
round[ 9].ik_add aeb65ba974e0f822d73f567bdb64c877
round[10].istart 9cf0a62049fd59a399518984f26bel78
round[10].is_row 9c6b89a349f0e18499fda678f2515920
round[10].is_box 1c05f271a417e04fff921c5c104701554
round[10].ik_sch ae87dff00ff11b68a68ed5fb03fcl567
round[10].ik_add b2822d81abe6fb275faf103a078c0033
round[11].istart 88db34fb1f807678d3f833c2194a759e
round[11].is_row 884a33781fdb75c2d380349e19f876fb
round[11].is_box 975c66c1cb9f3fa8a93a28df8ee10f63
round[11].ik_sch 1651a8cd0244bedala5da4cl0640bade
round[11].ik_add 810dce0cc9db8172b3678cle88alb5bd
round[12].istart ad9c7e017e55ef25bc150fe01ccb6395
round[12].is_row adcb0f257e9c63e0bc557e951c15ef01
round[12].is_box 1859fbc28alc00a078ed8aadcd42f6109
round[12].ik_sch a573c29fa176c498a97fce93a572c09c
round[12].ik_add bd2a395d2b6ac438d192443e615da195
round[13].istart 84elfd6bla5c946fdf4938977cfbac23
round[13].is_row 84fb386flaelac97df5cfd237c49946b
round[13].is_box 4f63760643e0aa85efa7213201a4e705
round[13].ik_sch 101112131415161718191a1b1clldelf
round[13].ik_add 5f72641557f5bc92f7be3b291db9f91a
round[14].istart 6353e08c0960e104cd70b751bacad0e7
round[14].is_row 63cab7040953d051cd60e0e7ba70el8c
round[14].is_box 00102030405060708090a0b0c0d0e0f0
round[14].ik_sch 000102030405060708090a0b0c0d0e0f
round[14].ioutput 00112233445566778899aabbccddeeff
```

EQUIVALENT INVERSE CIPHER (расшифрование):

```
round[ 0].iinput      8ea2b7ca516745bfeafc49904b496089
round[ 0].ik_sch      24fc79ccbf0979e9371ac23c6d68de36
round[ 1].istart      aa5ece06ee6e3c56dde68bac2621bebf
round[ 1].is_box      629deca599456db9c9f5ceaa237b5af4
round[ 1].is_row      627bceb9999d5aaac945ecf423f56da5
round[ 1].im_col      e51c9502a5c1950506a61024596b2b07
round[ 1].ik_sch      34fldlffbfceaa2ffce9e25f2558016e
round[ 2].istart      dled44fdla0f3f2afa4ff27b7c332a69
round[ 2].is_box      5153862143fb259514920403016695e4
round[ 2].is_row      516604954353950314fb86e401922521
round[ 2].im_col      91a29306cc450d0226f4b5eaf5efed8
round[ 2].ik_sch      5e1648eb384c350a7571b746dc80e684
round[ 3].istart      Cfb4dbedf4093808538502ac33del85c
round[ 3].is_box      5fc69f53ba4076bf50676aaa669c34a7
round[ 3].is_row      5f9c6abfbac634aa50409fa766677653
round[ 3].im_col      b041a94eff21ae9212278d903b8a63f6
round[ 3].ik_sch      C8a305808b3f7bd043274870d9ble331
round[ 4].istart      78e2acce741ed5425100c5e0e23b80c7
round[ 4].is_box      C13baaeccae9b5f6705207a03b493a31
round[ 4].is_row      c14907f6ca3b3aa070e9aa313b52b5ec
round[ 4].im_col      638357cec07de6300e30d0ec4ce2a23c
round[ 4].ik_sch      b5708e13665a7de14d3d824ca9f151c2
round[ 5].istart      d6f3d9dda6279bd1430d52a0e513f3fe
round[ 5].is_box      4a7ee5c9c53de85164f348472a827e0c
round[ 5].is_row      4a824851c57e7e47643de50c2af3e8c9
round[ 5].im_col      ca6f71058c642842a315595fdf54f685
round[ 5].ik_sch      74da7ba3439c7e50c81833a09a96ab41
round[ 6].istart      beb50aa6cff856126b0d6aff45c25dc4
round[ 6].is_box      5ad2a3c55felb93905f3587d68a88d88
round[ 6].is_row      5aa858395fd28d7d05e1a38868f3b9c5
round[ 6].im_col      Ca46f5ea835eab0b9537b6dbb221b6c2
round[ 6].ik_sch      3ca69715d32af3f22b67ffade4ccd38e
round[ 7].istart      f6e062ff507458f9be50497656ed654c
round[ 7].is_box      d6a0ab7d6cca5e695a6ca40fb953bc5d
round[ 7].is_row      d653a4696ca0bc0f5acaab5db96c5e7d
round[ 7].im_col      2a70c8da28b806e9f319ce42be4baead
round[ 7].ik_sch      f85fc4f3374605f38b844df0528e98e1
```

```
round[ 8].istart d22f0c291ffe031a789d83b2ecc5364c
round[ 8].is_box 7f4e814ccb0cd543c175413e8307245d
round[ 8].is_row 7f074143cb4e243ec10c815d8375d54c
round[ 8].im_col f0073ab7404a8a1fc2cba0b80df08517
round[ 8].ik_sch de69409aef8c64e7f84d0c5fcaf2c23
round[ 9].istart 2e6e7a2dafc6eef83a86ace7c25ba934
round[ 9].is_box C345bdfalbc799ela2dcaab0a857b728
round[ 9].is_row c357aae11b45b7b0a2c7bd28a8dc99fa
round[ 9].im_col 3225fe3686e498a32593c1872b613469
round[ 9].ik_sch aed55816cfl9c100bcc24803d90ad511
round[10].istart 9cf0a62049fd59a399518984f26bel78
round[10].is_box 1c17c554a4211571f970f24f0405e0c1
round[10].is_row 1c05f271a417e04ff921c5c104701554
round[10].im_col 9dld5c462e655205c4395b7a2eac55e2
round[10].ik_sch 15c668bd31e5247d17cl68b837e6207c
round[11].istart 88db34fblf807678d3f833c2194a759e
round[11].is_box 979f2863cb3a0fcla9el66a88e5c3fdf
round[11].is_row 975c66c1cb9f3fa8a93a28df8ee10f63
round[11].im_col d24bfbOelf997633cfce86e37903fe87
round[11].ik_sch 7fd7850f61cc991673db890365c89dl2
round[12].istart ad9c7e017e55ef25bc150fe01ccb6395
round[12].is_box 181c8a'098aed61c2782ffbaOc45900ad
round[12].is_row 1859fbc28a1c00a078ed8aadca2f6109
round[12].im_col aec9bda23e7fd8aff96d74525cdce4e7
round[12].ik_sch 2a2840c924234cc026244cc5202748c4
round[13].istart 84e1fd6b1a5c946fdf4938977cfbac23
round[13].is_box 4fe0210543a7e706efa476850163aa32
round[13].is_row 4f63760643e0aa85efa7213201a4e705
round[13].im_col 794cf891177bfd1ddf67a744acd9c4f6
round[13].ik_sch 1a1f181d1e1b1c191217101516131411
round[14].istart 6353e08c0960e104cd70b751bacad0e7
round[14].is_box 0050aOf04090e03080d02070c01060b0
round[14].is_row 00102030405060708090aObOcOdOeOf0
round[14].ik_sch 000102030405060708090aObOcOdOeOf
round[14].ioutput 00112233445566778899aabbccddeeff
```


Содержание

ВВЕДЕНИЕ.....	3
ОБОЗНАЧЕНИЯ.....	5
ГЛАВА 1 КРИПТОСИСТЕМЫ с СЕКРЕТНЫМ ключом.....	7
1.1. Основные термины и определения.....	7
1.2. ОЦЕНКА НАДЕЖНОСТИ криптоалгоритмов.....	8
1.5. КЛАССИФИКАЦИЯ МЕТОДОВ шифрования информации.....	9
1.4. БЛОЧНЫЕ СОСТАВНЫЕ шифры.....	12
1.5. Абсолютно стойкий шифр. Гаммирование.....	20
1.6. ПОТОЧНЫЕ шифры.....	25
1.7. МОДЕЛЬ СИММЕТРИЧНОЙ КРИПТОСИСТЕМЫ.....	24
1.8. КЛАССИФИКАЦИЯ угроз ПРОТИВНИКА. ОСНОВНЫЕ СВОЙСТВА КРИПТОСИСТЕМЫ.....	26
1.9. КЛАССИФИКАЦИЯ АТАК НА КРИПТОСИСТЕМУ с СЕКРЕТНЫМ ключом.....	27
1.10. РЕЖИМЫ ИСПОЛЬЗОВАНИЯ блочных шифров.....	28
Литература к ГЛАВЕ 1.....	40
ГЛАВА 2 СТАНДАРТ криптографической ЗАЩИТЫ XXI ВЕКА - Advanced Encryption Standard (AES).....	41
2.1. ИСТОРИЯ КОНКУРСА НА НОВЫЙ СТАНДАРТ КРИПТОЗАЩИТЫ.....	41
2.2. Блочный криптоалгоритм <i>RIJNDAEL</i> и СТАНДАРТ <i>AES</i>	47
2.2.1. МАТЕМАТИЧЕСКИЕ ПРЕДПОСЫЛКИ.....	48
2.2.2. ОПИСАНИЕ криптоалгоритма.....	51
2.2.3. ОСНОВНЫЕ ОСОБЕННОСТИ <i>RIJNDAEL</i>	69
2.5. АСПЕКТЫ РЕАЛИЗАЦИИ шифра.....	69
2.3.1. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ЗАШИФРОВАНИЯ.....	69
2.5.2. АППАРАТНАЯ РЕАЛИЗАЦИЯ ЗАШИФРОВАНИЯ.....	75
2.5.5. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ РАСШИФРОВАНИЯ.....	75
2.5.4. АППАРАТНАЯ РЕАЛИЗАЦИЯ прямого РАСШИФРОВАНИЯ.....	77
2.4. ХАРАКТЕРИСТИКИ ПРОИЗВОДИТЕЛЬНОСТИ.....	78
2.5. Обоснование выбранной структуры шифра.....	79
2.5.1. НЕПРИВОДИМЫЙ МНОГОЧЛЕН $f(x)$	79
2.5.2. S-блок, ИСПОЛЬЗУЕМЫЙ в преобразовании <i>SubBytes()</i>	79
2.5.5. Функция <i>MixColumns()</i>	82
2.5.4. СМЕЩЕНИЯ в функции <i>ShiftRows()</i>	83

2.5.5. Алгоритм РАЗВОРАЧИВАНИЯ КЛЮЧА.....	83
2.5.6. КОЛИЧЕСТВО раундов.....	85
2.6. Стойкость к ИЗВЕСТНЫМ АТАКАМ.....	87
2.6.1. СВОЙСТВА СИММЕТРИЧНОСТИ и СЛАБЫЕ КЛЮЧИ.....	87
2.6.2. ДИФФЕРЕНЦИАЛЬНЫЙ и ЛИНЕЙНЫЙ КРИПТОАНАЛИЗ.....	87
2.6.5. АТАКА МЕТОДОМ СОКРАЩЕННЫХ ДИФФЕРЕНЦИАЛОВ.....	99
2.6.4. АТАКА "КВАДРАТ".....	99
2.6.5. АТАКА МЕТОДОМ ИНТЕРПОЛЯЦИЙ.....	104
2.6.6. О СУЩЕСТВОВАНИИ СЛАБЫХ КЛЮЧЕЙ.....	105
2.6.7. АТАКА "ЭКВИВАЛЕНТНЫХ КЛЮЧЕЙ".....	105
2.7. ЗАКЛЮЧЕНИЕ.....	106
Литература к ГЛАВЕ 2.....	107
ГЛАВА 3 КОНЕЧНЫЕ ПОЛЯ.....	108
3.1. ВВЕДЕНИЕ.....	108
5.2. РЕГИСТРЫ СДВИГА с ЛИНЕЙНЫМИ ОБРАТНЫМИ СВЯЗЯМИ.....	109
5.5. ОСНОВЫ ТЕОРИИ КОНЕЧНЫХ ПОЛЕЙ.....	124
5.4. СЛОЖЕНИЕ и УМНОЖЕНИЕ в ПОЛЕ $GF(2^m)$	132
5.5. УСТРОЙСТВА, ФУНКЦИОНИРУЮЩИЕ в $GF(2)$, $l > 2$	141
5.6. УМНОЖЕНИЕ и ДЕЛЕНИЕ МНОГОЧЛЕНОВ НАД $GF(2)$	147
5.7. ИНВЕРСИЯ в $GF(2)$	150
Литература к ГЛАВЕ 5.....	151
ПРИЛОЖЕНИЕ 1. Пример процедуры РАЗВОРАЧИВАНИЯ КЛЮЧА.....	152
ПРИЛОЖЕНИЕ 2. Пример работы криптоалгоритма AES - 128.....	154
ПРИЛОЖЕНИЕ 3. ТЕСТОВЫЕ ПРИМЕРЫ.....	157
3.1. AES-128 ($N_k = 4$, $N_r = 10$).....	158
5.2. AES-192 ($N_k = 6$, $N_r = 12$).....	162
3.3. AES-256 ($N_k = 8$, $N_r = 14$)....	167